

Quantitative Simulation and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Lakshan N. Kumararathna

23038471
CB013366

A thesis submitted in partial fulfilment of the requirements of
Staffordshire University for the degree of

Computer Science

Staffordshire University
School of Digital, Technologies and Art
United Kingdom

Supervisor : Dr.Rajitha Navarathna
Assessor : Mrs. Sandali Pathirathne

August 2026

Thesis advisor

Author

Dr. Rajitha Navarathna

Lakshan N. Kumararathna

Quantitative Simulation and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Abstract

Modern supply chains operate under significant demand uncertainty, yet procurement and inventory allocation decisions are often supported by deterministic or manual approaches that may not adequately capture risk and variability. Existing approaches rarely integrate uncertainty modelling, optimisation, and risk evaluation within a single framework.

This research proposes *Optera*, a simulation–optimisation framework for inventory budget allocation under demand uncertainty. The framework combines Monte Carlo simulation, Particle Swarm Optimisation (PSO), portfolio inspired risk measures, and comparative optimisation within a hybrid Python–C++ architecture. Computationally intensive simulation components are implemented in C++ and integrated with Python through Pybind11.

The framework evaluates allocation strategies using profitability, Value at Risk (VaR), Conditional Value at Risk (CVaR), service level, lost revenue, and concentration measures. Experiments using open-source datasets to compare Optera IPSO with equal allocation, greedy allocation, random search, and Genetic Algorithm baselines across five demand shock scenarios. Results demonstrate the effects of demand uncertainty and disruption on profitability, risk, and service performance. Practitioner evaluation further indicated strong functional acceptance and usability of the framework.

Keywords: Supply Chain Optimisation, Inventory Budget Allocation, Monte Carlo Simulation, Particle Swarm Optimisation, Demand Uncertainty, Risk Analysis, Hybrid *Python/C++* Architecture.

Acknowledgment

I would like to express my sincere gratitude to my supervisor, **Dr. Rajitha Navarathna**, for his valuable guidance, constructive feedback, encouragement, and continuous support throughout this research project. His advice and direction were instrumental in shaping the research and completing the development and evaluation of the Optera framework.

I would also like to sincerely thank my assessor, **Mrs. Sandali Pathirathne**, for her valuable feedback and guidance throughout the research process. Her comments and recommendations helped improve the quality and direction of this work.

My sincere appreciation is extended to all the professionals who participated in the interviews and user evaluation activities. In particular, I would like to thank **Nevinka Wickramasinghe** of Deloitte, **D. Shashini Harischandra** of CBC Finance, **Sandeep Dissanayake** of Infinitum360, **R. G. Gihan Prasanna Rajapaksha** and **D. M. D. T. B. Dissanayaka** of Builtapps Business Solutions for sharing their professional experience, insights, and feedback. Their contributions provided valuable practical perspectives for the research and evaluation of the framework.

Finally, I would like to thank my family, friends, and everyone who supported and encouraged me throughout the completion of this research project. Their patience, motivation, and support were greatly appreciated.

CONTENT

CHAPTER 01: INTRODUCTION	1
1.1 Research Context	1
1.1.1 Inventory Budget Allocation under Demand Uncertainty	1
1.1.2 Challenges in Multi Category Inventory Allocation	2
1.1.3 Portfolio Optimisation Concepts to Inventory Management	3
1.2 Problem Statement	4
1.3 Research Motivation	4
1.4 Research Gap	5
1.4.1 Integration of Simulation and Optimization	5
1.4.2 Risk and Diversification in Inventory Budget Allocation	5
1.4.3 Computational Performance of Simulation–Optimization	5
1.4.4 Reusable Analytical Frameworks	6
1.5 Research Questions	6
1.6 Research Aim	6
1.7 Research Objectives	7
1.8 Research Contributions	7
1.8.1 Contribution to the Research Domain	8
1.8.2 Contribution to the Computational Domain	8
1.8.3 Contribution to Practical Inventory Analytics	8
1.9 Scope and Boundaries	8
1.10 Thesis Structure	9
CHAPTER 02: LITERATURE REVIEW	11
2.1 Introduction	11
2.2 Concept Map	11
2.3 Inventory Management under Demand Uncertainty	12
2.3.1 Inventory Allocation and Budget Constraints	12
2.3.2 Demand Uncertainty and Inventory Risk	13
2.3.3 Simulation Based Inventory Decision Making	13
2.4 Portfolio Optimisation and Cross Domain Applications	13
2.4.1 Portfolio Allocation Principles	13
2.4.2 Mean Variance Optimisation	14
2.4.3 Application of Portfolio Concepts Beyond Financial Markets	14
2.5 Risk and Diversification Modelling	14
2.5.1 Covariance Risk Measurement	14

2.5.2	Herfindahl-Hirschman Index and Concentration	14
2.6	Nature Inspired Optimisation	15
2.6.1	Particle Swarm Optimisation	15
2.6.2	Particle Representation and Search Behaviour	15
2.6.3	Genetic Algorithms	15
2.6.4	Random Search and Heuristic Baselines	15
2.7	Monte Carlo Simulation	16
2.7.1	Stochastic Demand Generation	16
2.7.2	Monte Carlo Performance Evaluation	16
2.7.3	Simulation Risk Analysis	16
2.8	Inventory Risk Metrics	17
2.8.1	Value at Risk (VaR)	17
2.8.2	Conditional Value at Risk (CVaR)	17
2.8.3	Service Level and Lost Revenue	17
2.8.4	Profit Efficiency and Return to Risk Measures	17
2.9	Existing Optimisation Approaches for Inventory Allocation	18
2.10	Comparative Analysis of Existing Approaches	18
2.11	Identified Research Gap	21
2.12	Chapter Summary	22
CHAPTER 03: METHODOLOGY		23
3.1	Chapter Overview	23
3.2	Research Methodology	23
3.2.1	Research Philosophy	23
3.2.2	Research Approach	23
3.2.3	Research Design	23
3.2.4	Research Strategy	24
3.2.5	Research Methodological Choice	24
3.2.6	Research Time Horizon	24
3.2.7	Methodological Justification for Computational Research	24
3.3	Practitioner Interviews and Requirement Elicitation	24
3.3.1	Findings from Business and Data Analytics Interviews	25
3.3.2	Technical Architecture Interview	26
3.3.3	Implications for the Optera Design	26
3.4	Research Design and Development Process	27
3.4.1	Design–Develop–Evaluate Research Process	27
3.4.2	Research Problem and Requirements Identification	27
3.4.3	Research Artefact Design	27
3.4.4	Iterative Development Strategy	27

3.4.5	Experimental Evaluation Strategy	27
3.4.6	Validation Strategy	27
3.5	Overall Research Framework	28
3.5.1	Conceptual Research Framework	28
3.5.2	Research Workflow	28
3.5.3	Experimental Evaluation Pipeline	28
3.5.4	Separation of Optimisation and Stochastic Simulation	28
3.5.5	Research Question–Methodology Mapping	28
3.6	Problem Formulation	29
3.6.1	Inventory Budget Allocation Problem	29
3.6.2	Decision Variables	29
3.6.3	Allocation Constraints	29
3.6.4	Inventory and Procurement Assumptions	29
3.6.5	Demand Uncertainty Representation	30
3.6.6	Optimisation Requirements	30
3.6.7	Problem Formulation Rationale	30
3.7	Mathematical Objective and Fitness Function	30
3.7.1	Step 1: Expected Margin Profit (The Baseline)	30
3.7.2	Step 2: Individual Risk Penalty	31
3.7.3	Step 3: Herfindahl Based Concentration Penalty	31
3.7.4	Step 4: Composite Markowitz–Herfindahl Fitness Function	31
3.7.5	Objective-Term Normalisation and Scale Analysis	31
3.7.6	Selection and Justification of Objective Weights	31
3.7.7	Fitness Function Design Rationale	32
3.8	Proposed Optera IPSO Method	32
3.8.1	Conceptual Basis of Optera IPSO	32
3.8.2	Particle Representation and Initialisation	32
3.8.3	Velocity and Position Update Mechanisms	32
3.8.4	Personal and Global Best Selection	32
3.8.5	Boundary and Constraint Handling (Simplex Projection)	32
3.8.6	IPSO Pseudocode	32
3.9	Demand Modelling and Stochastic Simulation	33
3.9.1	Historical Demand Estimation and Distribution Modelling	33
3.9.2	Monte Carlo Scenario Generation	33
3.9.3	Inventory and Procurement Dynamics	33
3.9.4	Simulation Outcome Generation and Sim–Opt Interaction	33
3.10	Comparative Optimisation Methods	34
3.10.1	Equal Allocation Baseline ($1/N$)	34
3.10.2	Greedy Profit Allocation	34

3.10.3	Random Search	34
3.10.4	Genetic Algorithm (GA)	34
3.10.5	Baseline Selection Rationale and Comparability	35
3.11	Evaluation Framework	35
3.11.1	Financial and Risk Measures	35
3.11.2	Operational and Diversification Measures	35
3.11.3	Optimisation Stability and Computational Measures	35
3.12	Data Acquisition and Experimental Selection	35
3.12.1	Primary Dataset (Thesis Benchmark)	36
3.12.2	Secondary Dataset (Cross-Dataset Diagnostic)	36
3.12.3	Conceptual Transformation and Data Partitioning	36
3.12.4	Data Quality and Ethical Considerations	36
3.12.5	Optimiser Benchmark and 30-Run Stability Experiment	37
3.12.6	Five Scenario Stress Testing	37
3.12.7	Ablation and Parameter Control	37
3.13	Experimental Fairness and Statistical Reliability	38
3.13.1	Controlled Variables and Random Seeds	38
3.13.2	Statistical Comparison of Optimisers	38
3.14	Computational Environment and Reproducibility	38
3.14.1	Software and Hardware Architecture	38
3.15	Research Management and Implementation Timeline	38
3.16	Ethical and Research Integrity Considerations	39
3.17	Chapter Summary	39
CHAPTER 04:	IMPLEMENTATION AND DATA COLLECTION	40
4.1	Chapter Overview	40
4.2	Evolution of the Optera Research Artefact	40
4.2.1	Initial Prototype and Research Exploration	40
4.2.2	Notebook Based Proof of Concept	41
4.2.3	Initial Universal Backtesting Concept	41
4.2.4	Transition from Application Prototype to Python Library	42
4.2.5	Git/Github Incremental Development	44
4.3	Optera System Architecture	45
4.3.1	Final Library Architecture	45
4.3.2	Component Architecture	45
4.3.3	End-to-End Data and Processing Flow	45
4.3.4	Separation of Data, Analytics, Optimisation and Simulation	45
4.3.5	Design Principles and Software Engineering Practices	46
4.4	Data Acquisition and Standardisation	46

4.4.1	Source Dataset Acquisition	46
4.4.2	Dataset Selection and Suitability	46
4.4.3	Data Schema Standardisation	46
4.4.4	Feature Engineering and Category Level Aggregation	47
4.5	ETL Pipeline Implementation	47
4.5.1	ETL Architecture	47
4.5.2	Schema Mapping and Data Validation	48
4.5.3	Data Cleaning and Processed Outputs	48
4.6	Demand Analytics Layer	49
4.6.1	Category Demand Profiling	49
4.6.2	Mean, Variance, and Distribution Modelling	49
4.6.3	Cross Category Covariance Analysis	49
4.6.4	Demand Analytics Interfaces	49
4.7	Optimisation Engine Implementation	49
4.7.1	Optimiser Interface and Architecture	49
4.7.2	Particle Representation and IPSO Implementation	49
4.7.3	Velocity and Position Updates	50
4.7.4	Constraint Enforcement (Simplex Projection)	50
4.8	Fitness Function Implementation	51
4.8.1	Initial Fitness Function Prototype	51
4.8.2	Risk Adjusted Objective Formulation (Markowitz)	52
4.8.3	Herfindahl Concentration Component	52
4.8.4	Composite Fitness Function and Normalisation	53
4.9	Inventory Aware Optimisation	53
4.9.1	Integration of Inventory Constraints	53
4.9.2	Inventory Policy Awareness	53
4.10	Monte Carlo Simulation Engine	56
4.10.1	Simulation Architecture	56
4.10.2	Stochastic Demand and Procurement Dynamics	56
4.10.3	Risk Distribution Generation and Outputs	57
4.11	Baseline Optimiser Implementation	57
4.11.1	Equal Allocation	57
4.11.2	Greedy Profit Allocation	57
4.11.3	Random Search	58
4.11.4	Genetic Algorithm	58
4.11.5	Common Interface and Experimental Control	58
4.12	Hybrid Python–C++ Computational Architecture	59
4.12.1	Computational Requirements	59
4.12.2	Python Research Orchestration	60

4.12.3	Native C++ Computational Components	60
4.12.4	Python–C++ Integration	60
4.12.5	Cross Language Performance Benchmark	61
4.12.6	Architectural Trade-off and Final Selection	62
4.12.7	Scalability Considerations	62
4.12.8	Python–C++ Communication Using pybind11	63
4.12.9	Transition to a 3-Tier Multi-Runtime Fallback Engine	63
4.13	Python Library Engineering and Packaging	64
4.13.1	Package Architecture and Public API	64
4.13.2	Build Pipeline, TestPyPI Staging, and Official PyPI Release	64
4.14	Implementation Challenges and Engineering Decisions	68
4.14.1	Performance and Portability Trade-off	68
4.14.2	Evolution of the Fitness Function	68
4.14.3	Final Optimisation Design	68
4.15	Final Optera Framework	68
4.15.1	End-to-End Execution Workflow	68
4.16	Chapter Summary	70
CHAPTER 05: RESULTS AND EVALUATION		71
5.1	Chapter Overview	71
5.2	Evaluation Strategy and Testing Rationale	71
5.2.1	Purpose of the Evaluation	71
5.3	Software Engineering Verification	73
5.3.1	Testing Objectives	73
5.3.2	Unit Testing	73
5.3.3	Functional and Integration Testing	74
5.3.4	Non-Functional Testing	75
5.4	Baseline Optimisation Results	76
5.4.1	Purpose of Baseline Comparison	76
5.4.2	Controlled Monte Carlo Evaluation	76
5.4.3	Allocation Characteristics	77
5.5	Component Ablation Study	78
5.5.1	Purpose of Ablation Testing	78
5.5.2	Ablation Results	78
5.6	Hyperparameter Sensitivity Analysis	79
5.6.1	Purpose of Sensitivity Analysis	79
5.6.2	Risk Penalty Sensitivity	80
5.6.3	Diversification Sensitivity	81
5.6.4	Swarm Size Sensitivity	81

5.6.5	Iteration Sensitivity	82
5.7	Inferential Statistical Evaluation	82
5.7.1	Purpose of Statistical Testing	82
5.7.2	Normality Diagnostics	83
5.7.3	Pairwise Statistical Comparison	84
5.8	Demand Shock Stress Testing	85
5.8.1	Purpose of Stress Testing	85
5.8.2	Stress Test Results	86
5.9	Computational Performance	87
5.10	Qualitative and Practical Evaluation	87
5.10.1	User Acceptance Testing	88
5.11	Evaluation Against the Research Questions	90
5.11.1	RQ1: Applicability of Portfolio Optimisation	90
5.11.2	RQ2: Comparison With Baseline Strategies	90
5.11.3	RQ3: Behaviour Under Stochastic Demand	91
5.11.4	RQ4: Computational Efficiency and Reusability	91
5.12	Limitations of the Evaluation	91
5.12.1	Dataset Size and Diversity	92
5.12.2	Simulation Assumptions	92
5.12.3	Limited Real World Validation	92
5.12.4	Baseline Scope	92
5.12.5	Statistical and Computational Interpretation	92
5.13	Chapter Summary	93
CHAPTER 06: CONCLUSION AND FUTURE WORK		94
6.1	Chapter Overview	94
6.2	Research Findings	94
6.2.1	RQ1: Portfolio Inspired Inventory Allocation	94
6.2.2	RQ2: Comparison with Alternative Strategies	94
6.2.3	RQ3: Stochastic Evaluation and Demand Shocks	95
6.2.4	RQ4: Computational Architecture and Reusability	95
6.3	Research Contributions	96
6.4	Limitations	96
6.5	Future Work	96
6.5.1	Larger Scale Evaluation	96
6.5.2	Enhanced Demand and Lead Time Modelling	96
6.5.3	Broader Optimization Benchmarking	97
6.5.4	Multi Objective Decision Support	97
6.5.5	Improved User Interface and Real World Validation	97

6.6 Final Conclusion	97
REFERENCES	xiv
APPENDIX	xix
A.1 UAT testing forms	xix
A.2 UAT testing Visit	xx
A.3 Interview Consent Forms	xxi
A.4 Log Sheets	xxii
A.5 Interview Questions	xxiv
A.6 Report Outputs	xxv

FIGURES

2.1	Concept Map of the Research Domain	12
3.2	Conceptual Research and experimental Framework	28
3.3	Dataset Schema experimental pipeline	37
3.4	Research and Implementation Gantt Chart	39
4.1	Evolution of Optera Idea	40
4.2	Notebook first Proof of Concept	41
4.3	GitHub Activity (starting February 25 Proof)	42
4.4	Comparison c++/python vs python speed test	42
4.5	Evidence informed technology selection and hybrid Python–C++ architecture	43
4.6	pip install optera inside python virtual environment	44
4.7	Chronological evolution of the Optera research artefact	44
4.8	Optera 5-Layer Component Architecture	45
4.9	End-to-End Data Flow Diagram	46
4.10	UML Class Diagram of the core Optera pipeline	47
4.11	Activity Diagram - ETL module	48
4.12	Activity Diagram - Analytical Module	50
4.13	Activity Diagram - Optimizer module	51
4.14	First Ever optimisation result version-01	52
4.15	Optimisation result version-02	52
4.16	Optimisation result version-03	53
4.17	Activity Diagram - Simulation module	56
4.18	Memory level interation	61
4.19	Python vs C++ vs C++/Python comparison	63
4.20	Deployment Diagram of the 3-Tier Multi-Runtime Fallback	64
4.21	Twine generated build files	65
4.22	Optera official test release v1.0.6 error free	65
4.23	Runtime error while running	66
4.24	Official release of optera v1.0.1 to PyPI	67
4.25	Official release of optera v1.0.1 to Github	67
4.26	Architecture Diagram Optera	68
4.27	Optera Sequense diagram	69
5.28	Comparison of Optera Objective Function Ablation Variants	79
5.29	Hyperparameter Sensitivity Results for the Optera IPSO Framework	83
5.30	Distribution of Stochastic Performance Across Compared Strategies	85
5.31	Example Optera Analytical Outputs	88

5.32	Optera Dashboard for Summarising UAT Results	90
33	UAT Testing Forms	xix
34	UAT Testing Visits and Approvals	xx
35	Participant Consent Forms	xxi
36	Log Sheets (1–4)	xxii
37	Log Sheets (5–10)	xxiii

TABLES

2.1	Optimization Paradigms for Multi Category Inventory Capital Allocation . . .	19
2.2	Selected Literature Directly Supporting the Optera Framework	20
2.3	Comparison of Metaheuristic Optimisation Algorithms	21
4.1	Mathematical Objective Term Decomposition Across Allocation Strategies . .	59
4.2	Empirical computational runtime benchmark	62
5.3	Overall Evaluation Strategy	72
5.4	Unit Testing Results	74
5.5	Functional and Integration Testing Results	75
5.6	Non-Functional Testing Results	75
5.7	Controlled Monte Carlo Performance Under the Baseline Demand Regime . . .	76
5.8	Optera IPSO Optimised Allocation	77
5.9	Component Ablation Results for the Optera IPSO Objective	78
5.10	Sensitivity to the Covariance Risk Coefficient λ	80
5.11	Sensitivity to the Diversification Coefficient α	81
5.12	Sensitivity to Particle Swarm Size	81
5.13	Sensitivity to Optimisation Iteration Count	82
5.14	Normality Diagnostics Across Optimisation Strategies	83
5.15	Pairwise Statistical Comparison Against Optera IPSO	84
5.16	Demand Shock Scenarios Used in the Evaluation	86
5.17	Complete Cross-Algorithm Performance Under Five Demand Shock Scenarios	86
5.18	Participants and Optera Modules Evaluated During UAT	89
5.19	Quantitative User Acceptance Testing Results	89

INTRODUCTION

1.1 Research Context

Modern supply chains operate in environments where demand, purchasing requirements, prices, and market conditions are inherently uncertain. This uncertainty creates a fundamental challenge for inventory decision making. Organisations must determine how much capital should be committed to different inventory categories without knowing the exact demand that will occur in the future. Excessive allocation to a category can result in unnecessary capital commitment and inventory holding costs, while insufficient allocation can lead to stockouts, lost sales, and reduced service levels. Traditional inventory management approaches commonly focus on determining appropriate inventory quantities or replenishment policies for individual products or categories. However, when an organisation has a finite procurement budget that must be distributed across multiple categories, the problem becomes an allocation problem in addition to a conventional inventory control problem. The decision maker must consider the expected economic return from each category while simultaneously accounting for uncertainty, demand relationships between categories, and the consequences of concentrating too much capital in a small number of categories.

The research presented in this thesis investigates this problem through a simulation–optimization perspective. Rather than treating inventory allocation as a purely deterministic mathematical optimization problem, the proposed approach combines stochastic demand simulation with optimization techniques to evaluate allocation decisions under uncertain future demand. This provides a basis for examining not only the expected profitability of an allocation but also its behaviour under alternative demand conditions. The research further investigates whether concepts traditionally associated with financial portfolio optimization can be adapted to the inventory allocation domain. In particular, the proposed framework considers expected profit, covariance based risk, and diversification when determining how a procurement budget should be distributed across inventory categories. Particle Swarm Optimization (PSO) is employed as the underlying population based optimization mechanism and is extended through the proposed Optera Markowitz–Herfindahl Swarm Intelligence (IPSO) formulation.

1.1.1 Inventory Budget Allocation under Demand Uncertainty

Inventory budget allocation concerns the distribution of a limited amount of procurement capital among products, product groups, or inventory categories. Let the allocation vector be represented by

$$w = [w_1, w_2, \dots, w_n],$$

where w_i represents the proportion of the available procurement budget assigned to category i . The allocation must satisfy the budget constraint

$$\sum_{i=1}^n w_i = 1,$$

subject to practical lower and upper allocation limits. In a deterministic environment, an allocation could be evaluated using known demand and cost information. Real inventory systems, however, rarely operate under such conditions. Customer demand varies over time and may exhibit volatility, seasonality, or relationships between different categories. Consequently, an allocation that appears optimal using historical averages may perform differently when exposed to alternative demand realizations. Monte Carlo simulation provides a mechanism for addressing this uncertainty by generating repeated demand realizations from estimated demand distributions. The resulting simulations can be used to estimate the distribution of outcomes associated with a particular allocation rather than relying solely on a single expected value. Monte Carlo methods have consequently been applied to inventory and supply chain decision making under stochastic demand conditions (Metropolis and Ulam, 1949; Li, 2025; Khoirudin and Sulistyowati, 2025; Islam and Uddin, 2024).

The combination of simulation and optimization creates a simulation–optimization problem. Instead of asking only which allocation produces the highest expected profit, the decision process can evaluate how alternative allocations behave across repeated stochastic scenarios. Recent research has demonstrated the relevance of simulation–optimization approaches for inventory management under stochastic demand (Maitra et al., 2023; Coşkun, 2025; Kartikasari et al., 2025; You et al., 2026). This perspective is particularly relevant to the present research because the objective is not simply to construct an allocation calculator. The research aims to develop a reusable computational framework in which an allocation strategy can be optimized and subsequently evaluated against stochastic demand conditions.

1.1.2 Challenges in Multi Category Inventory Allocation

Multi category inventory allocation introduces several challenges that are not present in simple single product inventory decisions. First, procurement capital is limited. Increasing the allocation to one category necessarily reduces the capital available to other categories. Therefore, allocation decisions are interdependent. Second, categories can have different demand characteristics. A category with high historical sales may not necessarily be the most attractive category when demand volatility and interactions with other categories are considered. Third, demand between categories may exhibit covariance. Categories whose demand varies together can expose the organisation to greater aggregate risk when heavily

concentrated in the same allocation.

A further challenge arises from the trade-off between profitability and diversification. Allocating more capital to highly profitable categories can increase expected profit, but excessive concentration may increase exposure to category specific demand uncertainty. Conversely, distributing capital evenly may reduce concentration but can result in capital being assigned to comparatively less profitable categories. These characteristics make multi category inventory allocation a suitable problem for optimization techniques capable of handling nonlinear and competing objectives. Metaheuristic methods, including Particle Swarm Optimization and other population based approaches, have previously been investigated for supply chain and inventory optimization (Gao et al., 2022; Guan et al., 2022; Guo et al., 2022; Tan et al., 2024; Lu et al., 2025).

1.1.3 Portfolio Optimisation Concepts to Inventory Management

The research is motivated in part by the conceptual similarity between financial portfolio allocation and inventory budget allocation. In portfolio optimization, an investor distributes capital among assets while considering expected returns, risk, and diversification. A similar decision structure exists in inventory procurement, a decision maker distributes limited capital among inventory categories while considering expected economic return and uncertainty. Portfolio optimization research has demonstrated that expected return alone is insufficient when making allocation decisions under uncertainty. Risk measures and diversification constraints can substantially influence the resulting portfolio. Similar principles have been investigated in non financial resource allocation problems, including supply chain and procurement applications (Simoglou et al., 2022; Stoilov et al., 2022; Xu et al., 2024).

The proposed research adapts this conceptual structure to inventory allocation. Expected margin profit represents the economic return associated with an allocation, while covariance based risk represents the interaction between category demand uncertainties. A Herfindahl–Hirschman Index (HHI) term is incorporated to represent allocation concentration. The resulting objective can be expressed conceptually as

$$F(w) = P(w) - R(w) - D(w),$$

where $P(w)$ represents expected profit, $R(w)$ represents the covariance risk penalty, and $D(w)$ represents the diversification or concentration penalty.

This formulation allows the optimizer to search for allocations that do not simply maximize expected profit but instead balance profitability, risk, and concentration.

1.2 Problem Statement

Organisations managing multiple inventory categories frequently need to allocate a finite procurement budget under uncertain demand. A decision based solely on historical average demand or expected profit can produce allocations that are highly concentrated or insufficiently robust to changes in demand conditions. Conversely, highly conservative allocations may reduce the economic potential of the available budget. Existing approaches demonstrate the effectiveness of Monte Carlo simulation for representing demand uncertainty and metaheuristic algorithms for solving complex inventory optimization problems. However, these techniques are often implemented as individual experimental solutions rather than as a reusable framework that integrates allocation optimization, stochastic simulation, evaluation, and comparative benchmarking within a single computational workflow.

Furthermore, the computational cost of repeated stochastic simulation can become significant when optimization algorithms evaluate a large number of candidate solutions. A simulation–optimization system may therefore require thousands of evaluations, each potentially involving multiple demand scenarios. Implementing the complete workflow exclusively in a high level language can introduce computational overhead, particularly for repeated simulation operations. Research comparing Python and C++ implementations has demonstrated the performance differences that can arise in computational workloads (Arboleda et al., 2023; Asaduzzaman et al., 2024; Goyal et al., 2026). Therefore, the problem addressed by this research is the design and empirical evaluation of a reusable simulation–optimization framework for multi category inventory budget allocation under demand uncertainty. The framework must support stochastic demand simulation, constrained allocation optimization, risk and diversification considerations, comparative evaluation against baseline optimization strategies, and efficient execution of computationally intensive simulation workloads.

1.3 Research Motivation

The primary motivation for this research arises from the need to make inventory allocation decisions under uncertainty using a systematic computational approach. A procurement budget represents a constrained resource, and therefore the quality of its allocation can directly influence profitability and operational performance. A second motivation is the potential for transferring established concepts from financial portfolio optimization to inventory management. Portfolio theory provides a well established conceptual foundation for balancing return, risk, and diversification. Applying these principles to inventory categories provides an opportunity to investigate whether a portfolio style objective can produce meaningful allocation decisions in a different operational domain. The third motivation concerns computational efficiency. Simulation–optimization can become computationally expensive because each candidate allocation may require repeated simulation. This motivates the investigation

of a hybrid architecture in which computationally intensive simulation components can be implemented in C++, while Python provides the higher level environment for data processing, experimentation, optimization orchestration, and analysis. The use of Python–C++ interoperability through technologies such as `pybind11`, `Ctypes` (C–ABI) provides a practical mechanism for combining the flexibility of Python with the execution performance of compiled code (Goyal et al., 2026). Finally, the research is motivated by reproducibility and reuse. Rather than producing a single application tailored to one dataset, the research aims to develop Optera as a Python oriented analytical framework. Such a framework can allow researchers or analysts to provide different datasets, configure demand and allocation parameters, select optimization strategies, execute stochastic simulations, and compare resulting allocation policies.

1.4 Research Gap

Although simulation, optimization, and computational performance have individually received considerable attention in supply chain research, several gaps remain relevant to the development of an integrated and reusable simulation–optimization framework.

1.4.1 Integration of Simulation and Optimization

Research has demonstrated the usefulness of Monte Carlo simulation for representing uncertain demand and the effectiveness of metaheuristic algorithms for inventory optimization (Maitra et al., 2023; Tan et al., 2024). However, there remains a practical need for frameworks that integrate allocation optimization and stochastic evaluation into a coherent reusable workflow. The research addresses this gap by combining an optimization layer with a simulation layer and an evaluation layer within Optera.

1.4.2 Risk and Diversification in Inventory Budget Allocation

A significant proportion of inventory optimization approaches focus on cost minimization, service level requirements, replenishment policies, or expected operational performance. Portfolio optimization introduces an alternative perspective in which expected return, risk, and concentration are considered simultaneously. Existing work has explored portfolio concepts in supply chain and resource allocation contexts (Simoglou et al., 2022; Stoilov et al., 2022), but further investigation is required into their application specifically to multi category inventory budget allocation. The proposed Optera objective therefore incorporates expected margin profit, covariance risk, and an HHI concentration term.

1.4.3 Computational Performance of Simulation–Optimization

Repeated Monte Carlo simulation can create substantial computational requirements when embedded within iterative optimization algorithms. Previous studies have examined the

performance characteristics of Python and C++ for computational workloads (Arboleda et al., 2023; Asaduzzaman et al., 2024). However, the practical integration of a high performance simulation component with a Python analytical workflow remains an important engineering consideration for simulation–optimization frameworks. Optera investigates a hybrid Python–C++ architecture in which simulation workloads can be executed through a compiled simulation engine while the surrounding analytical workflow remains accessible through Python.

1.4.4 Reusable Analytical Frameworks

Much existing work presents algorithms or experimental models designed for a specific inventory scenario, dataset, or research problem. Such implementations may demonstrate the effectiveness of an approach without necessarily providing a reusable framework for subsequent experimentation. The research therefore extends beyond the implementation of an individual optimizer by structuring Optera as a reusable Python library. The framework is intended to separate data processing, optimization, simulation, and evaluation so that different datasets and experimental configurations can be evaluated without redesigning the complete system.

1.5 Research Questions

The research is guided by the following questions:

1. **RQ1:** Can portfolio optimization concepts involving expected return, risk, and diversification be effectively adapted to multi category inventory budget allocation under demand uncertainty?
2. **RQ2:** How does the proposed Optera Markowitz–Herfindahl Particle Swarm Optimization approach compare with baseline allocation and optimization strategies in terms of allocation quality, convergence, and stability?
3. **RQ3:** How does the proposed framework perform when optimized allocation strategies are evaluated under stochastic demand and alternative demand shock scenarios?
4. **RQ4:** To what extent does the hybrid Python–C++ implementation improve the computational efficiency of repeated simulation workloads while maintaining accessibility through a Python interface?

1.6 Research Aim

The aim of this research is:

To design, implement, and empirically evaluate a reusable simulation–optimization framework for multi category inventory budget allocation under demand uncertainty, incorporating portfolio inspired risk and diversification objectives and a hybrid Python–C++ computational architecture.

The resulting framework, referred to as **Optera**, is designed to provide a computational environment in which inventory allocation strategies can be optimized, evaluated through stochastic simulation, and compared using quantitative performance measures.

1.7 Research Objectives

The following objectives are established to achieve the research aim:

1. To investigate inventory budget allocation as a stochastic optimization problem involving multiple inventory categories and uncertain demand.
2. To investigate the applicability of portfolio optimization concepts, particularly expected return, covariance risk, and diversification, to inventory budget allocation.
3. To design and implement the Optera optimization framework using Particle Swarm Optimization as the underlying metaheuristic search mechanism.
4. To develop a simulation component capable of evaluating candidate allocation strategies under stochastic demand conditions using Monte Carlo simulation.
5. To implement a hybrid Python–C++ architecture for computationally intensive simulation operations and expose the functionality through a Python interface.
6. To benchmark the proposed optimization approach against baseline strategies, including equal allocation, greedy allocation, random search, and Genetic Algorithm approaches.
7. To evaluate optimization stability through repeated independent optimization runs and assess the robustness of allocation strategies under alternative demand scenarios.
8. To evaluate the computational performance and practical reusability of the resulting Python library across different datasets and experimental configurations.

1.8 Research Contributions

The research makes contributions at both the methodological and computational levels.

1.8.1 Contribution to the Research Domain

The primary methodological contribution is the investigation of a portfolio inspired objective for inventory budget allocation. The proposed formulation combines expected margin profit, covariance demand risk, and an HHI concentration penalty within a single optimization objective. This provides a framework for examining inventory allocation from a risk adjusted portfolio perspective rather than treating each category independently. A second contribution is the empirical evaluation of the proposed optimization approach against multiple baseline strategies. The evaluation considers not only the best objective value but also convergence stability, allocation concentration, stochastic simulation outcomes, risk adjusted metrics, and behaviour under demand shocks.

1.8.2 Contribution to the Computational Domain

The research contributes a reusable simulation–optimization framework implemented as a Python oriented library. Optera separates optimization, simulation, data handling, and evaluation components so that researchers can configure experiments without modifying the underlying computational architecture. The hybrid Python–C++ design provides an additional computational contribution by moving computationally intensive simulation operations into a compiled environment while preserving Python as the primary interface for experimentation and analysis. This design aims to combine computational performance with accessibility and extensibility.

1.8.3 Contribution to Practical Inventory Analytics

From a practical perspective, the framework provides a structured method for evaluating procurement allocation policies before deployment. Instead of relying on a single historical outcome, an analyst can evaluate an allocation across repeated stochastic demand realizations and alternative demand scenarios. An important practical consideration identified during the empirical evaluation is that the usefulness of allocation optimization depends on the relationship between the available procurement budget and the underlying demand scale. When the available capital is substantially greater than the capital required to satisfy simulated demand, allocation decisions can become non binding, resulting in similar service levels and financial outcomes across competing strategies. This observation highlights the importance of appropriately configuring simulation constraints and scenario assumptions when applying the framework to different datasets.

1.9 Scope and Boundaries

The scope of this research is limited to multi category inventory budget allocation under stochastic demand. The research focuses on determining the allocation of a fixed procurement

budget across categories and evaluating the resulting strategies using simulation based performance measures. The research does not attempt to develop a complete enterprise inventory management system. It does not include user authentication, order management, warehouse management interfaces, enterprise resource planning integration, or production deployment as a commercial inventory application. Optera is instead treated as an analytical and research oriented computational library. The optimization problem is formulated around category level budget allocation. Detailed SKU level replenishment optimization, supplier negotiation, transportation routing, warehouse location optimization, and workforce planning are outside the primary scope.

Monte Carlo simulation is used to represent uncertain demand, but the framework does not claim to produce universally accurate real world demand forecasts. The quality of simulation results is dependent on the quality and assumptions of the input data and probability distributions. Similarly, the proposed IPSO approach is not assumed to be universally superior to all alternative optimization algorithms. Its performance is evaluated empirically against selected baseline strategies under the defined experimental conditions. The framework is also evaluated across multiple datasets to investigate generality. However, a limited number of datasets cannot establish universal generalization across all industries or inventory environments. Consequently, the findings should be interpreted within the defined experimental scope.

1.10 Thesis Structure

The remainder of this thesis is organized into six chapters.

Chapter 2 – Literature Review reviews the existing literature relevant to the research problem. The chapter examines inventory management under demand uncertainty, Monte Carlo simulation, portfolio optimization, Particle Swarm Optimization and related metaheuristic approaches, simulation–optimization methods, and hybrid Python–C++ computational architectures. The chapter also identifies the theoretical and research foundations supporting the development of the proposed Optera framework.

Chapter 3 – Methodology presents the research methodology adopted to investigate the research questions. It describes the overall research approach, experimental design, datasets and data preparation, optimization strategies, simulation methodology, evaluation metrics, benchmarking procedures, and methods used to assess the performance and robustness of the proposed framework.

Chapter 4 – Implementation and Data Collection describes the implementation of the Optera framework and the collection and preparation of the datasets used in the experiments. The chapter presents the system architecture, major software components, optimization implementation, Monte Carlo simulation engine, data processing pipeline, Python–C++ interoperability, library structure, configuration mechanisms, and experimental

data preparation.

Chapter 5 – Results and Evaluation presents and analyses the experimental results obtained from the proposed framework. The chapter evaluates Optera IPSO against the selected baseline strategies using optimization performance, convergence stability, allocation characteristics, stochastic simulation outcomes, risk adjusted performance measures, demand shock stress testing, and computational performance. The results are analysed in relation to the research questions and research objectives.

Chapter 6 – Conclusion and Future Work concludes the research by summarising the major findings and evaluating the extent to which the research aim and objectives were achieved. The chapter presents the main contributions of the research, discusses the limitations identified during the investigation, and proposes potential directions for future development and research involving the Optera framework.

Chapter Summary

This chapter established the context and motivation for investigating inventory budget allocation under demand uncertainty. The problem was positioned as a simulation–optimization problem in which limited procurement capital must be distributed across multiple inventory categories while accounting for expected profitability, uncertainty, and concentration. The chapter also established the research gap concerning the integration of stochastic simulation, portfolio inspired allocation objectives, optimization, computational performance, and reusable software architecture. Four research questions and eight research objectives were defined to guide the investigation.

The proposed research aims to address these issues through Optera, a reusable simulation–optimization framework combining a portfolio inspired Particle Swarm Optimization approach, Monte Carlo demand simulation, and a hybrid Python–C++ computational architecture. The scope and boundaries of the research were also established, emphasizing that the framework is an analytical research library rather than a complete enterprise inventory management application. The following chapter examines the existing literature and theoretical foundations underlying inventory management under uncertainty, portfolio optimization, stochastic simulation, metaheuristic optimization, and high performance scientific computing.

LITERATURE REVIEW

2.1 Introduction

Supply chain procurement and inventory allocation are fundamentally complex decision making problems executed under pervasive environmental stochasticity (Gkonos, 2023; Tan et al., 2024). Modern commercial enterprises operate in hyper connected, volatile markets where future customer demand, supplier reliability, procurement costs, and economic conditions fluctuate dynamically over time (Bo and Xu, 2024; Khoirudin and Sulistyowati, 2025). In such operating environments, an organisation must resolve a critical capital rationing challenge, how to distribute a finite procurement budget across multiple competing product categories when future demand trajectories are uncertain, non-stationary, and structurally interdependent (Saracoglu, 2024; Kartikasari et al., 2025). Traditional approaches in classical inventory theory have predominantly relied on deterministic models, static single period formulations, and decoupled point forecast estimates (Islam and Uddin, 2024; Turgay et al., 2026). These conventional models suffer from what can be termed as stateless blindness, evaluating expected demand values at aggregate period boundaries while disregarding the path dependent nature of intra period transactions, cash flow bottlenecks, and asymmetric stockout penalties (Joubert et al., 2024; Oyedepo, 2025). When procurement capital is allocated solely to satisfy an expected historical average, minor deviations in realized demand produce severe operational pathology, excessive inventory holding costs in under demanded categories, alongside unfulfilled customer demand in surge categories (Balasubramaniam, 2026; Mazzi et al., 2026). To overcome the fragility of deterministic formulations, modern operations research has converged toward the paradigm of simulation–optimisation (Jalali and Van Nieuwenhuysse, 2015; Amaran et al., 2016; Fu, 2015). The research presented in this thesis investigates the design, theoretical formulation, and empirical evaluation of **Optera**, a simulation–optimisation framework engineered for multi category inventory budget allocation.

2.2 Concept Map

Figure 2.1 presents the conceptual structure of the research domain. The concept map illustrates the relationships between supply chain uncertainty, stochastic modelling, simulation methodologies, optimization algorithms, event driven architectures, and high performance computing technologies.

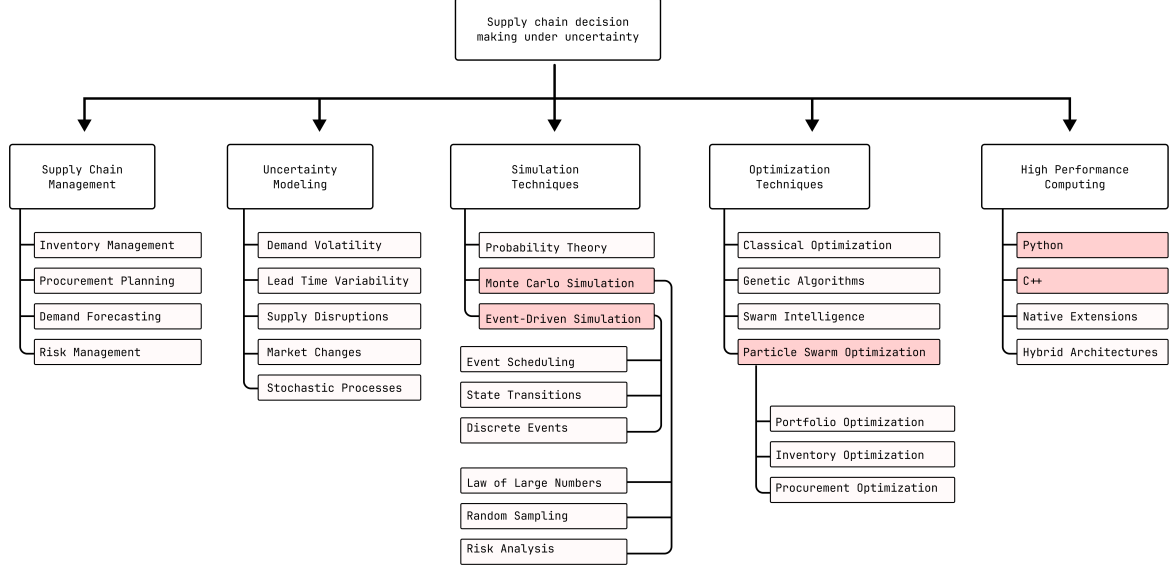


Figure 2.1: Concept Map of the Research Domain

2.3 Inventory Management under Demand Uncertainty

2.3.1 Inventory Allocation and Budget Constraints

At the enterprise level, multi category inventory management is governed by financial capital rationing (Saracoglu, 2024). A retail enterprise possesses a finite total procurement budget $B \in \mathbb{R}_{>0}$ that must be allocated across n distinct merchandise categories. Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}_{\geq 0}^n$ denote the monetary allocation vector, where x_i represents the capital committed to category i . The fundamental capital feasibility constraint is:

$$\sum_{i=1}^n x_i \leq B, \quad x_i \geq 0, \quad \forall i \in \{1, \dots, n\}. \quad (2.1)$$

When expressed in normalized fractional form, the decision vector is represented by allocation weights $\mathbf{w} = [w_1, w_2, \dots, w_n]^T \in \Delta^{n-1}$, where $w_i = x_i/B$, defining a point on the standard $(n-1)$ dimensional unit simplex:

$$\Delta^{n-1} = \left\{ \mathbf{w} \in \mathbb{R}^n \mid \sum_{i=1}^n w_i = 1, \quad w_i \geq 0 \forall i \in \{1, \dots, n\} \right\}. \quad (2.2)$$

Operational requirements frequently impose lower and upper category allocation bounds (Király et al., 2012; Simoglou et al., 2022):

$$w_{\min,i} \leq w_i \leq w_{\max,i}, \quad \sum_{i=1}^n w_{\min,i} \leq 1 \leq \sum_{i=1}^n w_{\max,i}. \quad (2.3)$$

Unlike unconstrained single item models, the coupling budget constraint introduces strict

inter variable dependency, turning this into a multidimensional polynomial capital allocation challenge (Saracoglu, 2024; Tan et al., 2024).

2.3.2 Demand Uncertainty and Inventory Risk

In real world retail, customer demand is unpredictable and changes constantly (Huseyn, 2023; Islam and Uddin, 2024). Let $D_{i,t} \geq 0$ represent the actual customer demand for category i during time step t . When future demand is modeled as a random vector $\mathbf{D} \sim \mathcal{P}(\theta)$, simply relying on an average expected demand ($\hat{\mathbf{D}} = \mathbb{E}[\mathbf{D}]$) is not enough. A simple average ignores how demand fluctuates, how extreme the changes can be, and how different products affect each other (Khoirudin and Sulistyowati, 2025; Turgay et al., 2026).

The main financial risk comes from the fact that having too much stock and having too little stock do not cost the company the same amount (Kartikasari et al., 2025; You et al., 2026). Let c_i be the buying cost, p_i the selling price, h_i the holding cost, and s_i the penalty for running out of stock. If we buy a quantity $Q_i = x_i/c_i$, the final profit depends on two possible situations:

$$\Pi_i(Q_i, D_i) = \begin{cases} (p_i - c_i)D_i - h_i(Q_i - D_i), & \text{if } Q_i \geq D_i \quad (\text{Too much stock}), \\ (p_i - c_i)Q_i - s_i(D_i - Q_i), & \text{if } Q_i < D_i \quad (\text{Not enough stock}). \end{cases} \quad (2.4)$$

Because of this, making budget decisions based only on average demand ($\mathbb{E}[\mathbf{D}]$) is risky. If actual demand is much lower than expected, the company wastes money holding extra stock. If demand is much higher, the company loses potential sales and damages customer trust (Oyedepo, 2025; Mazzi et al., 2026).

2.3.3 Simulation Based Inventory Decision Making

Real world supply chains are too complex and unpredictable to solve using standard mathematical formulas (Jalali and Van Nieuwenhuysse, 2015). To solve this problem, researchers use a method called Simulation–Optimisation (SO) (Amaran et al., 2016; Fu, 2015). In the SO approach, a Monte Carlo simulation acts as a testing ground. Instead of guessing the outcome, the optimisation algorithm tests many different allocation choices (Δ^{n-1}) to find the one that gives the best average result across many scenarios (Maitra et al., 2023; Maitra, 2024).

2.4 Portfolio Optimisation and Cross Domain Applications

2.4.1 Portfolio Allocation Principles

Modern Portfolio Theory (MPT), pioneered by Harry Markowitz (Markowitz, 1952), fundamentally altered asset allocation by demonstrating that an asset should not be evaluated solely on its isolated return, but on its marginal contribution to aggregate variance. In the

classical financial context, an investor allocates wealth across assets characterized by expected return $\mu \in \mathbb{R}^n$ and covariance matrix $\Sigma \in \mathbb{R}^{n \times n}$.

2.4.2 Mean Variance Optimisation

Markowitz formalized the capital allocation decision as a quadratic programming problem aimed at maximizing a risk adjusted utility function parameterized by risk aversion coefficient $\lambda \geq 0$:

$$\max_{\mathbf{w} \in \Delta^{n-1}} U(\mathbf{w}) = \mathbf{w}^T \mu - \frac{\lambda}{2} \mathbf{w}^T \Sigma \mathbf{w}. \quad (2.5)$$

When individual assets exhibit low or negative cross covariance, the total portfolio variance is strictly less than the weighted sum of individual asset variances, providing a mathematical bedrock for risk reduction.

2.4.3 Application of Portfolio Concepts Beyond Financial Markets

Operations researchers have successfully generalized Markowitz principles to non financial resource allocation, such as electric power procurement (Simoglou et al., 2022) and agricultural acreage allocation (Stoilov et al., 2022). However, adapting portfolio optimization to retail inventory requires overcoming fundamental physical asymmetries. Financial assets are instantly liquid, whereas unsold inventory incurs holding costs and obsolescence decay (Gkonos, 2023; Arakelian et al., 2024). Therefore, Optera adopts the mathematical abstraction of portfolio allocation while evaluating physical inventory dynamics through stochastic simulation.

2.5 Risk and Diversification Modelling

2.5.1 Covariance Risk Measurement

In retail supply chains, sales volumes across categories exhibit correlation due to macroeconomic fluctuations (Islam and Uddin, 2024; Huseyn, 2023). The empirical demand covariance matrix Σ_D establishes this relationship. When procurement capital is allocated according to weight vector $\mathbf{w}$, the portfolio's aggregate demand variance exposure is expressed via the quadratic form:

$$R_{\text{cov}}(\mathbf{w}) = \mathbf{w}^T \Sigma_D \mathbf{w}. \quad (2.6)$$

2.5.2 Herfindahl-Hirschman Index and Concentration

While covariance risk measures volatility, it does not guarantee structural diversification (Stoilov et al., 2022). Extreme concentration exposes a firm to catastrophic risk if a specific category suffers an unprecedented disruption (Raghuwanshi, 2026). To penalize concentration,

Optera incorporates the Herfindahl-Hirschman Index (HHI) (Saunders et al., 2023):

$$HHI(\mathbf{w}) = \sum_{i=1}^n w_i^2 = \|\mathbf{w}\|_2^2. \quad (2.7)$$

where $HHI = 1/n$ corresponds to perfect uniform diversification, and $HHI = 1.0$ corresponds to absolute single category concentration.

2.6 Nature Inspired Optimisation

2.6.1 Particle Swarm Optimisation

Particle Swarm Optimization (PSO) (Kennedy and Eberhart, 1995) is a population based metaheuristic inspired by the collective behavior of bird flocks. A swarm consists of M particles navigating a continuous space, where particle j updates its velocity based on personal historical experience ($\mathbf{p}_j$) and swarm consensus ($\mathbf{g}^*$):

$$v_{j,d}^{t+1} = \omega v_{j,d}^t + c_1 r_{1,d}^t (p_{j,d} - x_{j,d}^t) + c_2 r_{2,d}^t (g_d^* - x_{j,d}^t), \quad (2.8)$$

2.6.2 Particle Representation and Search Behaviour

Because updating positions via unconstrained velocity additions causes candidate allocations to leave the valid probability simplex Δ^{n-1} , Optera implements a robust Simplex Projection and Repair Operator $\mathcal{P}_\Delta(\cdot)$. Negative positions are clamped, and the vector is normalized to unity sum (Király et al., 2012).

2.6.3 Genetic Algorithms

Genetic Algorithms (GA) represent an evolutionary benchmark (Saracoglu, 2024; Mazzi et al., 2026). Unlike PSO's velocity updates, GA evolves a population through selection, crossover (e.g., BLX- α), and mutation. While GA provides strong global exploration, crossover operations require careful projection repairs to maintain simplex feasibility, often incurring higher evaluation payloads than PSO.

2.6.4 Random Search and Heuristic Baselines

To adhere to the No-Free-Lunch Theorem (Wolpert and Macready, 1997), optimization performance must be benchmarked against non metaheuristic baselines under identical evaluation budgets:

1. **Equal Allocation Baseline (1/ N Heuristic):** Uniformly distributes capital ($w_i = 1/n$), providing maximum structural diversification.

2. **Greedy Profit Allocation:** Greedily assigns capital to the highest margin categories, representing naive short term profit maximization without risk awareness.
3. **Uniform Random Search:** Generates points uniformly over the simplex via Dirichlet distributions, establishing whether metaheuristic search extracts meaningful intelligence beyond blind stochastic exploration.

2.7 Monte Carlo Simulation

2.7.1 Stochastic Demand Generation

Monte Carlo simulation estimates system behavior under stochastic uncertainty (Metropolis and Ulam, 1949; Li, 2025). Utilizing the Cholesky Factorization ($\Sigma_D = \mathbf{L}\mathbf{L}^T$), a vector of correlated daily demand realizations is generated,

$$\mathbf{d}_t = \max(\mathbf{0}, \mu_D + \mathbf{L}\mathbf{z}_t), \quad \mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n). \quad (2.9)$$

where $\mathbf{d}_t$ is the simulated demand vector at time t and μ_D is the historical mean demand. $\mathbf{L}$ is the Cholesky lower triangular matrix of the empirical covariance Σ_D , mapping standard normal variables $\mathbf{z}_t$ into correlated shocks. The $\max(\mathbf{0}, \dots)$ operator prevents negative demand.

2.7.2 Monte Carlo Performance Evaluation

For each simulation path, daily inventory processing computes realized sales ($S_{i,t}^s$), unfulfilled stockouts ($U_{i,t}^s$), and end of day balances ($I_{i,t}^s$). The net profit realized across a scenario is,

$$\Pi_s(\mathbf{w}) = \sum_{t=1}^{T_{\text{sim}}} \sum_{i=1}^n \left(p_i S_{i,t}^s - h_i I_{i,t}^s - s_i U_{i,t}^s \right) - B. \quad (2.10)$$

where $\Pi_s(\mathbf{w})$ is the net profit for scenario s under allocation $\mathbf{w}$. For n categories over T_{sim} days, p_i , h_i , and s_i denote the unit margin, holding cost, and shortage penalty. These apply to realized sales $S_{i,t}^s$, closing inventory $I_{i,t}^s$, and unfulfilled demand $U_{i,t}^s$. The initial budget B is deducted from the total sum. By the Central Limit Theorem, the sample mean estimator converges asymptotically to the true expected profit (Li, 2025; Turgay et al., 2026).

2.7.3 Simulation Risk Analysis

Beyond average profitability, Monte Carlo simulation enables rigorous macroeconomic stress testing (Raghuwanshi, 2026; Erisoglu et al., 2026). Optera evaluates allocations across adversarial stress scenarios, including High Volatility Shocks, Asymmetric Category Surges, and Catastrophic Category Disruptions (Oyedepo, 2025).

2.8 Inventory Risk Metrics

2.8.1 Value at Risk (VaR)

Value at Risk (VaR) is used to estimate the potential monetary loss of an inventory allocation at a selected confidence level and over a defined evaluation period (Stoilov et al., 2022; Zhao, 2025). In Optera, VaR provides an indication of the loss level that is unlikely to be exceeded under normal simulated conditions. However, VaR only describes the loss at a particular percentile and does not show how large the losses may become beyond that point. Therefore, it is useful as a basic downside risk indicator but should not be considered alone when evaluating severe inventory disruptions (Raghuwanshi, 2026).

2.8.2 Conditional Value at Risk (CVaR)

Conditional Value at Risk (CVaR), also known as Expected Shortfall, provides additional information about losses beyond the VaR threshold (Zhao, 2025). Instead of considering only the selected loss boundary, CVaR calculates the average loss in the worst part of the simulated outcomes. This makes it useful for understanding the potential impact of extreme demand conditions and poor inventory outcomes (Jin and Gao, 2025; Raghuwanshi, 2026). Optera therefore uses CVaR together with VaR to provide a clearer view of both general downside risk and more severe tail losses.

2.8.3 Service Level and Lost Revenue

Financial performance alone is not sufficient to evaluate an inventory allocation because a strategy may achieve reasonable profit while still failing to satisfy a significant portion of customer demand (Islam and Uddin, 2024; Kartikasari et al., 2025). Optera therefore includes the Type-2 Service Level (Fill Rate β), which measures the proportion of total demand that can be fulfilled from available inventory. Lost Revenue is also recorded to represent the estimated sales that cannot be realised because the required inventory is unavailable. These measures allow financial results to be considered together with the operational impact of stock shortages.

2.8.4 Profit Efficiency and Return to Risk Measures

Optera also uses risk adjusted measures to examine whether higher simulated profit is achieved with a reasonable level of uncertainty (Poudel and Paudel, 2025; Xu et al., 2024). These include an Inventory Sharpe Style Ratio, which relates expected inventory profit to the variability of that profit, and a Tail Risk Return Ratio, which relates the expected return to the level of extreme downside risk represented by CVaR. These measures provide an additional basis for comparing allocation strategies beyond profit alone.

2.9 Existing Optimisation Approaches for Inventory Allocation

Exact mathematical optimisation methods, such as Mixed Integer Linear Programming (MILP) and Mixed Integer Nonlinear Programming (MINLP), can provide optimal solutions when the problem can be expressed using a suitable mathematical formulation (Saracoglu, 2024; Simoglou et al., 2022). However, these methods require the objective function and constraints to be explicitly defined mathematically. This makes them less suitable for Optera, where the quality of an allocation is evaluated through a stochastic simulation containing demand uncertainty and discrete inventory behaviour.

Differential Evolution (DE) (Storn and Price, 1997) provides strong global search capabilities for continuous optimisation problems. However, its mutation process can make it more difficult to maintain strict allocation constraints such as $\sum w_i = 1$. Other population based methods, including Grey Wolf Optimisation (GWO) (Mirjalili et al., 2014) and Whale Optimisation Algorithm (WOA) (Mirjalili and Lewis, 2016), have also been applied to multi product inventory problems (Sadeghi et al., 2023). However, recent reviews identify premature convergence as a common concern with metaphor based optimisation methods and note similarities between several of these approaches and established PSO based methods (Yousef et al., 2026). Bayesian Optimisation (BO) (Maitra et al., 2023; Maitra, 2024) can reduce the number of objective function evaluations by using a model to guide the search. However, its computational cost increases as the number of evaluated points grows, making it less suitable for the repeated high dimensional simulation evaluations required by Optera (Xu et al., 2024).

2.10 Comparative Analysis of Existing Approaches

Table 2.1 presents a structured taxonomy comparing optimization paradigms relevant to multi category inventory allocation. Furthermore, Table 2.2 systematically synthesizes the empirical literature informing Optera, and Table 2.3 directly compares algorithm mechanics on the constrained simplex.

Optimization Paradigm	Representative Algorithms	Core Search Principle & Mechanics	Suitability for Optera Problem
Swarm Intelligence	Canonical PSO, IPSO, M-PPSO	Collective particle velocity-position updates guided by personal best $\mathbf{p}_i$ and global consensus $\mathbf{g}^*$.	Primary Paradigm: Naturally continuous, derivative-free, highly efficient on constrained simplex.
Evolutionary Computation	GA, NSGA-II, Differential Evolution	Darwinian survival-of-the-fittest, chromosome crossover (BLX- α), and stochastic mutation.	Primary Benchmark: Strong global exploration; requires projection repairs; higher evaluation payload.
Animal Inspired Heuristics	GWO, WOA, SSA, Bat Algorithm	Metaphor driven leadership hierarchies and encircling/spiral movement mechanisms.	Contextual: Prone to premature convergence on simplices; redundant parameters.
Probabilistic Surrogate	Bayesian Optimization (GP, TPE)	Probabilistic surrogate surface modeling coupled with acquisition function sampling.	Alternative Benchmark: High efficiency on few evaluations; scales poorly ($O(N^3)$) with dimensionality.
Exact Mathematical	MILP, MINLP, Chance Constrained	Gradient based and branch and bound optimization of explicit algebraic formulations.	Inapplicable to Discrete Simulation: Incompatible with black-box stochastic event driven logic.
Heuristic Baselines	Equal ($1/N$), Greedy Margin, Random	Naïve deterministic distribution rules or uniform Dirichlet stochastic sampling.	Mandatory Baselines: Establishes empirical lower/upper bounds per No-Free-Lunch theorem.

Table 2.1: Optimization Paradigms for Multi Category Inventory Capital Allocation

Study	Year	Problem Scope	Methodology	Key Findings & Relevance to Optera
Maitra et al. (Maitra, 2024)	2024	Stochastic inventory optimisation	Monte Carlo simulation with Bayesian optimisation	Demonstrates simulation in the loop optimisation for inventory decisions under demand uncertainty, supporting Optera's stochastic evaluation workflow.
Király et al. (Király et al., 2012)	2012	Constrained supply chain inventory	Constrained PSO with Monte Carlo	Shows that PSO combined with Monte Carlo can address nonlinear inventory problems subject to service level constraints, directly supporting Optera's optimisation–simulation approach.
Verma et al. (Verma et al., 2022)	2022	Dynamic inventory control	PSO literature survey	Confirms the applicability of PSO to inventory cost optimisation while identifying scalability challenges relevant to multi category allocation.
Guan et al. (Guan et al., 2022)	2022	Nonlinear supply chain optimisation	Improved PSO	Demonstrates that improved PSO mechanisms can reduce local optima entrapment and improve convergence, supporting the use of an improved PSO variant in Optera.
Simoglou et al. (Simoglou et al., 2022)	2022	Procurement risk optimisation	Portfolio based mathematical optimisation	Supports treating procurement allocation as a portfolio style decision problem where capital is distributed across physical inventory categories according to risk and return characteristics.
Sadeghi et al. (Sadeghi et al., 2023)	2023	Multi product inventory allocation	Chance constrained optimisation with metaheuristics	Shows that metaheuristic approaches can obtain near optimal allocations for constrained multi product inventory problems, supporting Optera's allocation framework.
Kartikasari et al. (Kartikasari et al., 2025)	2025	Retail inventory cost–service trade-offs	Demand forecasting with Monte Carlo simulation	Demonstrates the use of simulation to evaluate empirical cost–service trade-offs, supporting Optera's service level and financial risk evaluation.
Jin and Gao (Jin and Gao, 2025)	2025	Portfolio risk and drawdown analysis	Intertemporal risk measures	Shows that evaluating only final-horizon outcomes can overlook severe intermediate losses, supporting Optera's use of distributional and downside risk measures.
Shah et al. (Shah et al., 2025)	2025	Scientific computing performance	Python versus compiled C++ benchmarking	Identifies performance limitations associated with heavy numerical workloads in Python, supporting Optera's compiled C++ simulation backend.
Goyal et al. (Goyal et al., 2026)	2026	Python–C++ scientific computing	High performance language bindings	Provides technical support for integrating computationally intensive C++ components with Python orchestration, consistent with Optera's hybrid architecture.

Table 2.2: Selected Literature Directly Supporting the Optera Framework

Algorithm	Search Mechanism	Simplex Constraint Handling	Primary Advantage for Inventory Problem	Critical Limitation / Vulnerability
IPSO (Optera)	Velocity updates via cognitive ($\mathbf{p}_i$) and social ($\mathbf{g}^*$) vectors	Direct Simplex Projection $\mathcal{P}_\Delta$ + dynamic inertia decay	Continuous domain expressiveness; fast convergence; highly interpretable	Risk of premature convergence if swarm diversity collapses.
Canonical GA	Tournament selection, crossover, mutation	Repair operator after crossover/mutation	Robust global exploration; avoids local optima entrapment	High evaluation overhead; slow convergence on smooth simplices.
Diff. Evolution	Vector differences	Boundary clipping and projection normalization	Exceptional continuous global search capability	Parameter sensitivity; fragile under strict equality constraints.
GWO / WOA	Leadership hierarchy / spiral hunting	Exterior penalty functions or normalization	Few control parameters; simple procedural formulation	Parameter redundancy; theoretical overlap with PSO.
Bayesian Opt.	Gaussian Process surrogate + Expected Improvement	Acquisition optimization over bounded simplex	Minimizes simulation evaluations for expensive objectives	High cubic computational complexity ($O(N^3)$); poor scaling with dimensions.
Exact MILP	Branch and bound / cutting planes	Direct algebraic formulation ($\sum x_i \leq B$)	Proven mathematical global optimality	Inapplicable to black-box stochastic discrete event simulation.

Table 2.3: Comparison of Metaheuristic Optimisation Algorithms

2.11 Identified Research Gap

A synthesis of the reviewed literature identifies four related areas where further investigation is relevant to stochastic inventory management and simulation–optimisation.

1. **Application of Portfolio Optimisation to Physical Inventory:** Modern Portfolio Theory (MPT) provides an established framework for balancing expected return and risk (Markowitz, 1952). However, its application to physical retail inventory involves additional operational considerations, including holding costs, stockouts, expected margin, and allocation concentration. This study therefore investigates whether portfolio based allocation concepts can be incorporated into a simulation–optimisation framework for retail inventory decisions.
2. **Controlled Empirical Benchmarking:** Metaheuristic optimisation methods are commonly assessed according to their ability to obtain good solutions, but comparisons can become difficult when different methods are given different computational resources or evaluation budgets (Wolpert and Macready, 1997). This study therefore uses fundamental allocation approaches, including equal and greedy allocation, alongside metaheuristic methods under a controlled experimental setting to provide a more consistent basis for comparison.
3. **Multi Dimensional Downside Risk Testing:** Optimising primarily for expected profit $\mathbb{E}[\Pi]$ may not fully represent the performance of an inventory strategy under adverse demand conditions. Previous work highlights the importance of considering tail risk

when evaluating uncertain systems (Raghuwanshi, 2026; Zhao, 2025). This study therefore complements expected profit analysis with CVaR risk measures and demand shock scenarios to examine how allocation strategies behave under less favourable conditions.

4. **Computational Requirements of Simulation–Optimisation:** Repeated stochastic simulation can require a large number of objective function evaluations, creating computational challenges for implementations that perform intensive numerical loops entirely in Python (Asaduzzaman et al., 2024). This motivates the investigation of a hybrid implementation in which computationally intensive simulation components are implemented in compiled C++ and exposed to Python through a Pybind11 interface (Jakob et al., 2017; Goyal et al., 2026).

2.12 Chapter Summary

This chapter conducted a comprehensive, academically rigorous critical review of the theoretical, algorithmic, and computational literature foundational to the development of the Optera framework. The investigation established that deterministic models suffer from stateless blindness, and demonstrated that multi category procurement under capital rationing is an integrated simplex constrained allocation problem. It analyzed the mechanics of Markowitz mean variance optimization, justifying the adaptation of covariance risk and Herfindahl concentration to physical procurement budgets.

Furthermore, the review investigated Particle Swarm Optimization dynamics and simplex projection operators, establishing rigorous benchmarking protocols against Genetic Algorithms, Random Search, and heuristic baselines. Finally, the necessity of coherent risk metrics (*CVaR*), adversarial macroeconomic stress testing, and hybrid C++ architectures was verified to overcome the latency and fragility identified in prior operations research literature. These principles form the methodological foundation for the subsequent chapter, which formally defines the Optera IPSO algorithm and experimental pipeline.

METHODOLOGY

3.1 Chapter Overview

This chapter establishes the methodological framework guiding the design, development, and evaluation of the Optera simulation–optimisation engine. Moving beyond theoretical concepts, this chapter translates the research objectives into a rigorous computational methodology. It adapts Saunders’ Research Onion for software centric research, formalises the step by step mathematical derivation of the Markowitz–Herfindahl fitness function, and defines the Inertia Weighted Particle Swarm Optimisation (IPSO) method. Furthermore, it outlines the experimental design, including baseline benchmarking, stochastic simulation protocols, and the criteria for ensuring experimental fairness and statistical reliability across two distinct retail datasets.

3.2 Research Methodology

3.2.1 Research Philosophy

This research adopts a Positivist and Pragmatic philosophy (Saunders et al., 2023). Positivism is justified because the research evaluates objective, quantifiable phenomena. Specifically, the economic performance of inventory allocations under statistically defined stochastic demand. Pragmatism complements this by focusing on practical problem solving. The ultimate goal is not merely to describe reality, but to engineer a computational artefact (Optera) that actively improves decision making under uncertainty.

3.2.2 Research Approach

A Deductive approach is employed (Saunders et al., 2023). The research begins with established theories from quantitative finance (Modern Portfolio Theory) and computational intelligence (Swarm Intelligence) and deduces a novel application framework for physical inventory (Markowitz, 1952; Kennedy and Eberhart, 1995). Hypotheses regarding algorithm convergence, risk mitigation, and structural diversification are formulated and subsequently tested through empirical computational experiments.

3.2.3 Research Design

The research utilizes a Quantitative, Experimental design. It measures numerical variables (e.g., expected profit, Value at Risk, execution time) under strictly controlled computational conditions. Unlike qualitative or observational studies, this design allows for the systematic

manipulation of independent variables (the optimisation algorithm, risk penalty weights, demand volatility) to observe effects on the dependent variables (allocation quality, computational throughput).

3.2.4 Research Strategy

The primary research strategy is Simulation and Algorithmic Experimentation. By generating thousands of Monte Carlo demand paths, the research creates a synthetic but empirically grounded environment where years of inventory dynamics can be evaluated in seconds (Metropolis and Ulam, 1949; Li, 2025). This strategy is essential because testing experimental, potentially flawed allocation strategies in a live corporate supply chain would incur unacceptable financial risk (Raghuwanshi, 2026).

3.2.5 Research Methodological Choice

A Mono-method Quantitative choice is applied, relying entirely on computational metrics, algorithmic benchmarking, and statistical analysis to evaluate outcomes.

3.2.6 Research Time Horizon

The study adopts a Cross sectional time horizon, analysing the algorithmic performance over fixed historical data snapshots rather than conducting a longitudinal study over several years of real time supply chain monitoring.

3.2.7 Methodological Justification for Computational Research

Saunders' Research Onion is traditionally applied to social or business sciences (Saunders et al., 2023). In this project, it is explicitly adapted to justify a Computational Design Science methodology. The subjects of study are not human participants, but algorithms and data structures. Consequently, traditional primary research methods (e.g., interviews, surveys) are methodologically invalid for answering the research questions. Instead, empirical knowledge is generated through computational benchmarking, stress testing, and statistical analysis of algorithmic outputs against defined operational constraints.

3.3 Practitioner Interviews and Requirement Elicitation

To complement the literature review and establish practical requirements for the proposed framework, semi structured interviews were conducted with professionals working in data analytics, audit, and software engineering. The interviews were used to understand current approaches to procurement and inventory planning, identify limitations in existing analytical workflows, and assess the practical relevance of the proposed simulation–optimization and

hybrid computational architecture. Full interview forms and participant responses are provided in Appendix. Three professionals contributed to the interview study. Nevinka Wickramasinghe, an Intern in Control Assurance at Deloitte, provided an audit and data analytics perspective. D. Shashini Harischandra, a Data Analyst at CBC Finance, provided a direct business analytics perspective related to sales, inventory, procurement planning and forecasting. Sandeep Dissanayake, a Lead DevOps Engineer at Infinitum360, provided a technical perspective on computational performance, hybrid architectures and deployment considerations.

3.3.1 Findings from Business and Data Analytics Interviews

The business oriented interviews indicated that procurement allocation decisions are generally based on a combination of historical data analysis and management judgement. Harischandra described the use of historical sales performance, inventory turnover, seasonal demand and current stock levels, while also noting the importance of management experience when responding to market conditions and supplier related issues. Similarly, Wickramasinghe observed that organisations commonly combine structured data analysis with business judgement rather than relying entirely on automated decision models. A recurring finding was the limited use of formal simulation or backtesting before implementing procurement strategies. Existing analysis was described as being primarily based on historical reporting, scenario comparison and management discussion. Tools such as Excel, Power BI, Python and R may be used for analysis, but the evaluation of alternative strategies is often manual or ad-hoc. As Wickramasinghe explained, the process is generally “more focused on descriptive analysis and scenario comparison rather than a formal simulation or back testing framework.”

Both participants also identified uncertainty as an important challenge. Seasonal demand can be incorporated into forecasting and safety stock planning, but unexpected market changes and extreme demand fluctuations remain difficult to anticipate. Harischandra noted that organisations continuously monitor sales and inventory information and adjust procurement plans, while Wickramasinghe characterised the response to unexpected changes as often “reactive rather than fully predictive.” The interviews therefore provided practical support for evaluating procurement allocations through repeated simulation rather than relying only on historical descriptive analysis. They also supported the inclusion of both profitability and risk in the decision process. In particular, Wickramasinghe stated that a tool capable of showing the “trade-off between profitability and risk” would provide a stronger analytical basis for procurement decisions. Data privacy was another relevant consideration. The Deloitte participant highlighted that financial and operational data can be subject to confidentiality, governance and regulatory requirements. Local processing was therefore considered advantageous where organisational policies restrict the movement of sensitive data to external platforms. This finding supports the decision to design Optera as a locally executable analytical framework rather than requiring financial or operational datasets to be uploaded to

an external service.

3.3.2 Technical Architecture Interview

The technical interview examined the feasibility of the hybrid Python–C++ architecture used by Optera. Dissanayake identified Python interpreter overhead as a potential limitation for computationally intensive workloads involving large scale data processing, simulation and repeated calculations. He considered a hybrid architecture, where Python is responsible for orchestration and user facing functionality while computationally intensive operations are implemented in C++, to be a “highly practical and well established engineering approach.” The interview also supported the use of PyBind11 for communication between the Python and C++ components. In particular, Dissanayake identified zero-copy memory sharing as an important consideration when transferring large numerical arrays and described PyBind11 with NumPy as a practical approach for reducing unnecessary data copying.

The participant further indicated that substantial performance improvements can be achievable when CPU intensive iterative workloads are moved from Python into compiled code. However, the magnitude of the improvement depends on workload characteristics, algorithm design, memory access patterns, compiler optimisation and hardware. This provided qualitative support for evaluating the hybrid implementation experimentally rather than assuming that the architecture would automatically provide a fixed performance advantage. From a deployment perspective, the interview also reinforced the importance of local or controlled execution for systems processing sensitive financial or operational information. In addition to performance, Dissanayake identified maintainability, documentation, testing, security, integration capability, reproducibility and deployment flexibility as important considerations when evaluating an open-source simulation framework.

3.3.3 Implications for the Optera Design

The interviews contributed several practical requirements to the design of Optera. First, the identified limitations of manual and descriptive strategy evaluation motivated the inclusion of a structured simulation–optimization workflow. Second, the importance of uncertainty and the profitability–risk trade-off supported the use of stochastic demand simulation and risk aware allocation evaluation. Third, the data privacy considerations supported local execution of the analytical workflow. Finally, the technical interview provided practitioner level support for separating Python orchestration from performance critical C++ simulation components. These findings were therefore used as supporting evidence during the methodological and architectural design of Optera. The interviews do not constitute statistical validation of industry wide practice, as the participant group was small and drawn from specific professional contexts. Instead, they provide qualitative practitioner evidence that helps connect the research problem identified in the literature with the requirements of the implemented framework.

3.4 Research Design and Development Process

3.4.1 Design–Develop–Evaluate Research Process

Because Optera is a research artefact (a software library), this study utilizes the Design Science Research (DSR) paradigm, structured around the Design–Develop–Evaluate cycle. This iterative process ensures that theoretical mathematical models are successfully translated into functional, verifiable software components.

3.4.2 Research Problem and Requirements Identification

The problem space dictates that the system must handle continuous variables (budget weights), non linear constraints (budget limits), and stochastic black-box evaluations (Monte Carlo simulation). The primary requirement is an optimisation engine capable of balancing these factors without collapsing into premature convergence (Amaran et al., 2016).

3.4.3 Research Artefact Design

The artefact, Optera, is designed methodologically as a decoupled system. The architecture separates the optimiser (which proposes allocation weights) from the evaluator (which simulates the economic consequences). This separation of concerns allows alternative algorithms to be tested interchangeably against the same objective function.

3.4.4 Iterative Development Strategy

Development follows an iterative methodology. Initial iterations focus on deterministic demand to validate algorithmic boundaries. Subsequent iterations introduce stochastic Monte Carlo paths, followed by the integration of the covariance and HHI risk penalties.

3.4.5 Experimental Evaluation Strategy

Evaluation is conducted through benchmarking. The proposed IPSO algorithm is evaluated against established baselines using identical random seeds and evaluation limits to ensure observed performance gains stem from algorithmic superiority rather than computational resource disparities (Wolpert and Macready, 1997).

3.4.6 Validation Strategy

The algorithms are validated via independent out of sample data simulations to ensure the generated budget allocations generalize correctly and have not merely overfit the optimisation training set (Joubert et al., 2024).

3.5 Overall Research Framework

3.5.1 Conceptual Research Framework

The conceptual framework defines the flow of information through the Optera system. Historical transaction data informs demand distributions and covariance matrices. These parameters feed into the stochastic simulation engine. Concurrently, the IPSO optimiser explores the capital simplex, passing candidate allocations to the simulator, which returns a multi-dimensional fitness score.

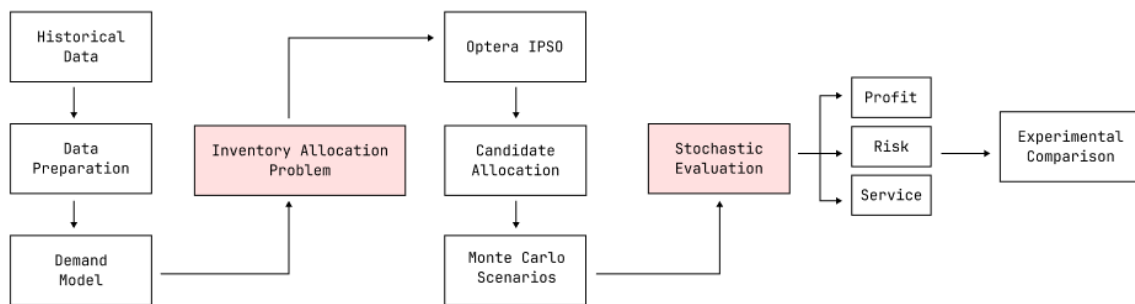


Figure 3.2: Conceptual Research and experimental Framework

3.5.2 Research Workflow

The workflow begins with data ingestion and ETL preprocessing, proceeds to statistical demand modelling, executes the hybrid simulation–optimisation loop, and concludes with risk metric extraction and visualisation.

3.5.3 Experimental Evaluation Pipeline

The evaluation pipeline takes the optimal allocation produced by the framework and subjects it to independent macroeconomic stress tests to verify robustness (Raghuwanshi, 2026).

3.5.4 Separation of Optimisation and Stochastic Simulation

Methodologically, coupling optimisation logic tightly with simulation logic creates rigid, unscalable code. By defining a clear boundary where the optimiser only understands abstract "fitness vectors" and the simulator only understands "capital weights," the framework achieves the high modularity required for fair algorithmic benchmarking (Maitra, 2024).

3.5.5 Research Question–Methodology Mapping

RQ1 (Portfolio Concepts) is addressed via the mathematical formulation in Section 3.6. RQ2 (Optimiser Performance) is addressed via the benchmark experiments in Section 3.11. RQ3

(Stress Testing) is addressed via Monte Carlo risk analysis. RQ4 (Computational Architecture) is evaluated via Python/C++ execution timing.

3.6 Problem Formulation

3.6.1 Inventory Budget Allocation Problem

An organisation has a total capital budget B to procure inventory across n distinct product categories. The objective is to determine the optimal proportion of capital to assign to each category to maximize risk adjusted economic returns over a defined planning horizon (Saracoglu, 2024). Methodologically, this translates a standard procurement task into a constrained capital rationing problem, where the opportunity cost of misallocated capital is explicitly measured.

3.6.2 Decision Variables

The decision variable is a continuous weight vector $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top$, where $w_i \in [0, 1]$ represents the fraction of the budget B allocated to category i . In the simulation, this monetary weight is dynamically converted into physical procurement quantities ($Q_i = w_i B / c_i$) using the respective unit cost c_i of each category, mathematically bridging the financial allocation with physical inventory dynamics.

3.6.3 Allocation Constraints

The fundamental constraint is the budget equality limit, enforcing that exactly 100% of the available capital is deployed:

$$\sum_{i=1}^n w_i = 1, \quad w_i \geq 0 \quad \forall i. \quad (3.11)$$

Additionally, the methodology supports the enforcement of practical lower and upper bounds ($w_{\min} \leq w_i \leq w_{\max}$) to guarantee baseline category exposure and prevent operational bottlenecks caused by absolute zero allocations.

3.6.4 Inventory and Procurement Assumptions

It is methodologically assumed that procurement occurs exclusively at the beginning of the evaluation period, and no mid cycle replenishment is permitted. This single period, newsvendor style constraint intentionally heightens the economic penalty of poor allocation decisions. By removing the safety net of continuous replenishment, it provides a strictly unforgiving fitness gradient for the optimiser to evaluate.

3.6.5 Demand Uncertainty Representation

Future demand is not a deterministic scalar but a non stationary stochastic process. It is represented methodologically as a multivariate probability distribution $\mathbf{D} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, capturing both individual category volatility (variance) and inter category correlations (covariance) (Khoirudin and Sulistyowati, 2025). This ensures evaluations account for systemic demand shocks across correlated product lines.

3.6.6 Optimisation Requirements

The optimiser must navigate a continuous, non-convex objective space where evaluations are inherently noisy due to Monte Carlo sampling. Because the fitness landscape is shaped by asymmetric stockout and holding penalties, it contains numerous local optima, mandating an algorithmic approach capable of balancing broad global exploration with precise localized exploitation.

3.6.7 Problem Formulation Rationale

This formulation deliberately abstracts away specific supply chain network topologies, transportation routing, and warehouse capacities. By isolating the capital allocation efficiency from these confounding operational variables, the methodology allows for a rigorous, mathematically controlled test of the portfolio optimisation hypothesis.

3.7 Mathematical Objective and Fitness Function

The formulation of the objective function is a primary methodological contribution of this research. The fitness function was iteratively derived across four stages to address the limitations of pure profit maximisation (Simoglou et al., 2022).

3.7.1 Step 1: Expected Margin Profit (The Baseline)

The initial formulation focused solely on maximizing expected profit. If μ_i is the expected demand for category i , p_i is the price, and c_i is the cost, the baseline objective is:

$$F_1(\mathbf{w}) = \sum_{i=1}^n w_i \cdot \mu_i (p_i - c_i) \quad (3.12)$$

Limitation: This function is risk blind. It will greedily allocate 100% of the budget to the single category with the highest expected margin, ignoring volatility and holding risk.

3.7.2 Step 2: Individual Risk Penalty

To counter greedy allocation, an individual risk penalty was introduced. Let R_i represent the isolated demand variance of category i , scaled by a risk aversion parameter λ :

$$F_2(\mathbf{w}) = \underbrace{\sum_{i=1}^n w_i \cdot \mu_i(p_i - c_i)}_{\text{Expected Profit}} - \lambda \sum_{i=1}^n w_i R_i \quad (3.13)$$

Limitation: While this penalizes volatile categories, it ignores the correlation between categories. If two categories crash simultaneously, the inventory portfolio suffers catastrophic failure.

3.7.3 Step 3: Herfindahl Based Concentration Penalty

To enforce structural diversification and prevent capital from aggregating in just two or three highly profitable categories, the Herfindahl-Hirschman Index (HHI) was integrated, scaled by parameter γ :

$$F_3(\mathbf{w}) = F_2(\mathbf{w}) - \gamma \sum_{i=1}^n w_i^2 \quad (3.14)$$

Advantage: This mathematically forces the optimiser to spread capital, acting as a structural safeguard against algorithmic overfitting (Stoilov et al., 2022).

3.7.4 Step 4: Composite Markowitz–Herfindahl Fitness Function

The final methodological derivation replaces independent risk with the full Markowitz covariance matrix Σ . This captures both individual volatility and cross category correlation (Markowitz, 1952). Expressed in fully vectorized form, the Optera IPSO objective function is:

$$F(\mathbf{w}) = \underbrace{\mathbf{w}^\top \boldsymbol{\mu}_{\text{profit}}}_{\text{Expected Profit } P(\mathbf{w})} - \underbrace{\lambda \cdot (\mathbf{w}^\top \Sigma \mathbf{w})}_{\text{Covariance Risk Penalty } R(\mathbf{w})} - \underbrace{\gamma \cdot \text{HHI}(\mathbf{w})}_{\text{Diversification Penalty } D(\mathbf{w})} \quad (3.15)$$

3.7.5 Objective-Term Normalisation and Scale Analysis

Because profit is measured in currency, covariance in squared currency, and HHI as a dimensionless ratio, the terms operate on vastly different numerical scales. Min-max normalisation is applied to $P(\mathbf{w})$ and $R(\mathbf{w})$ internally to ensure the λ and γ hyperparameters interact symmetrically.

3.7.6 Selection and Justification of Objective Weights

The values of λ and γ represent the decision maker's risk appetite. Methodologically, these are exposed as tunable hyperparameters to trace an efficient Pareto frontier of allocations.

3.7.7 Fitness Function Design Rationale

This 4 step evolution successfully translates abstract financial portfolio constraints into a robust mathematical boundary condition suitable for physical retail inventory.

3.8 Proposed Optera IPSO Method

3.8.1 Conceptual Basis of Optera IPSO

Particle Swarm Optimisation was selected as the core search mechanism because the capital simplex is continuous and non-convex (Kennedy and Eberhart, 1995). The Inertia Weighted (IPSO) variant dynamically balances global exploration in early iterations with localized exploitation in final iterations (Lu et al., 2025).

3.8.2 Particle Representation and Initialisation

Each particle represents a complete supply chain budget allocation. The swarm is initialised using uniform Dirichlet distributions, guaranteeing that initial positions strictly satisfy the $\sum w_i = 1$ budget constraint before evaluation begins.

3.8.3 Velocity and Position Update Mechanisms

Velocity is updated using cognitive and social components. Let $\mathbf{p}_j$ be the particle's personal best, and $\mathbf{g}^*$ be the global swarm best. Position $\mathbf{x}_j$ is updated continuously.

$$\mathbf{v}_j^{t+1} = \omega(t)\mathbf{v}_j^t + c_1r_1(\mathbf{p}_j - \mathbf{x}_j^t) + c_2r_2(\mathbf{g}^* - \mathbf{x}_j^t) \quad (3.16)$$

3.8.4 Personal and Global Best Selection

Best positions are retained strictly based on the composite fitness function $F(\mathbf{w})$ defined in Equation 3.15.

3.8.5 Boundary and Constraint Handling (Simplex Projection)

Standard velocity updates push particles outside the valid budget constraint (Király et al., 2012). A Simplex Projection operator is applied after every update. Any negative weight is clipped to zero, and the resulting vector is divided by its scalar sum to project it perfectly back onto the valid $n - 1$ simplex.

3.8.6 IPSO Pseudocode

The methodological execution of the algorithm is defined as follows:

Algorithm 1 Optera IPSO Algorithm for Inventory Allocation

```

1: Initialise swarm positions  $\mathbf{w}_j \sim \text{Dirichlet}(\mathbf{1})$ , velocities  $\mathbf{v}_j = \mathbf{0}$ 
2: for iteration  $t = 1$  to  $T_{max}$  do
3:   Decay inertia weight  $\omega(t)$ 
4:   for each particle  $j$  do
5:     Evaluate stochastic fitness  $F(\mathbf{w}_j)$  via Monte Carlo Simulation
6:     Update personal best  $\mathbf{p}_j$  and global best  $\mathbf{g}^*$ 
7:   end for
8:   for each particle  $j$  do
9:     Update velocity  $\mathbf{v}_j^{t+1}$  using  $\omega, c_1, c_2, \mathbf{p}_j, \mathbf{g}^*$ 
10:    Update raw position  $\mathbf{x}_j^{t+1} = \mathbf{w}_j^t + \mathbf{v}_j^{t+1}$ 
11:    Simplex Projection: clip  $\mathbf{x}_j^{t+1} \geq 0$ , normalise  $\mathbf{w}_j^{t+1} = \frac{\mathbf{x}_j^{t+1}}{\sum x_{j,i}}$ 
12:   end for
13: end for
14: return  $\mathbf{g}^*$ 

```

3.9 Demand Modelling and Stochastic Simulation

3.9.1 Historical Demand Estimation and Distribution Modelling

Historical data is not fed directly into the optimiser. Instead, it is used to empirically estimate the μ vector and Σ matrix (Turgay et al., 2026).

3.9.2 Monte Carlo Scenario Generation

Using Cholesky decomposition ($\Sigma = \mathbf{LL}^\top$), the Monte Carlo engine generates S synthetic, correlated demand paths, ensuring evaluations account for events unseen in the raw historical slice (Li, 2025).

3.9.3 Inventory and Procurement Dynamics

For each demand path, the simulation chronologically deducts sales from allocated inventory, tallies unmet demand (stockouts), and calculates holding costs for unsold units.

3.9.4 Simulation Outcome Generation and Sim–Opt Interaction

The aggregated economic performance is returned to the IPSO fitness function, closing the loop between the simulation engine and the optimisation search space (Maitra et al., 2023).

Algorithm 2 Monte Carlo Simulation Sub-Routine

Require: Allocation $\mathbf{w}$, Paths S

- 1: **for** path $s = 1$ to S **do**
 - 2: Generate correlated demand $\mathbf{D}_s \sim \mathcal{N}(\mu, \Sigma)$
 - 3: Calculate Sales = $\min(\mathbf{w} \cdot B/\mathbf{c}, \mathbf{D}_s)$
 - 4: Calculate Stockouts = $\max(0, \mathbf{D}_s - \mathbf{w} \cdot B/\mathbf{c})$
 - 5: Compute Net Profit Π_s
 - 6: **end for**
 - 7: **return** Expected Profit $\hat{\mathbb{E}}[\Pi]$
-

3.10 Comparative Optimisation Methods

To validate the effectiveness of the Optera IPSO algorithm, it must be methodologically benchmarked against standard approaches to satisfy the No-Free-Lunch theorem (Wolpert and Macready, 1997).

3.10.1 Equal Allocation Baseline ($1/N$)

Uniformly divides the budget across all categories. This acts as the maximum diversification baseline ($HHI = 1/n$). Methodologically, it provides a robust, zero intelligence benchmark to test if active optimisation yields statistically significant improvements over naïve diversification.

3.10.2 Greedy Profit Allocation

Ranks categories by historical profit margin and allocates maximum capital to the top categories until the budget is exhausted. This strategy simulates standard corporate purchasing behavior, testing whether covariance risk awareness actually improves long term outcomes over aggressive, risk blind profit seeking.

3.10.3 Random Search

Evaluates N randomly generated valid allocations over the simplex and selects the best. This establishes the empirical lower bound of algorithmic intelligence, proving whether the IPSO algorithm effectively utilizes search space gradients or merely benefits from stochastic sampling.

3.10.4 Genetic Algorithm (GA)

Provides a state-of-the-art evolutionary benchmark, utilising selection, blend crossover, and mutation to navigate the simplex (Mazzi et al., 2026). Because GA relies on genetic

recombination rather than swarm velocity, it serves as the primary metaheuristic competitor to validate if swarm dynamics are distinctly suited for this specific continuous allocation problem.

3.10.5 Baseline Selection Rationale and Comparability

These specific algorithms were selected to represent fundamentally different search paradigms (Swarm, Evolutionary, Random, Heuristic). By subjecting them to strictly equivalent budget constraints and identical objective function evaluation limits, the framework ensures a mathematically fair assessment of algorithmic superiority.

3.11 Evaluation Framework

Allocations are methodologically evaluated across competing dimensions to ensure holistic supply chain health, preventing the optimiser from exploiting a single metric at the expense of operational viability.

3.11.1 Financial and Risk Measures

Expected Profit evaluates the raw economic return of the allocated capital. **Conditional Value at Risk ($CVaR_{0.95}$)** evaluates tail-risk severity. Unlike standard variance, $CVaR$ measures the expected financial ruin strictly within the worst 5% of simulated scenarios, providing a coherent metric for catastrophic demand shocks (Zhao, 2025; Raghuwanshi, 2026).

3.11.2 Operational and Diversification Measures

Fill Rate (Type-2 Service Level) ensures physical customer demand is actually met, preventing the optimiser from artificially holding cash to minimize risk exposure. **HHI** verifies that the framework maintains structural diversification rather than devolving into single asset corner solutions that look highly profitable but harbor hidden fragility.

3.11.3 Optimisation Stability and Computational Measures

Because stochastic algorithms can yield outlier successes, tracking the standard deviation of fitness scores across repeated independent runs measures algorithmic convergence reliability. Furthermore, execution runtime in seconds is tracked to verify the computational feasibility of deploying the engine in real world retail environments.

3.12 Data Acquisition and Experimental Selection

To ensure the robustness and generalisability of the optimisation engine, experiments are conducted across two methodologically distinct open-source retail datasets. While the detailed Extract, Transform, Load (ETL) pipeline is deferred to Chapter 4, this section defines the

conceptual data strategy, required variables, and partitioning methodology necessary to answer the research questions.

3.12.1 Primary Dataset (Thesis Benchmark)

The primary dataset drives the core research investigations (RQ1, RQ2, and RQ3) and is sourced from a comprehensive demand forecasting repository (Huseyn, 2023).

- **Relevant Variables:** *Date, Store ID, Product ID, Category, Region, Inventory Level, Units Sold, Units Ordered, Price, Discount, Weather Condition, Promotion, Competitor Pricing, Seasonality, Epidemic, and Demand.*
- **Methodological Purpose and Selection Rationale:** This dataset was selected because it exhibits realistic capital constraints, active inter category covariance, and high lost revenue trade-offs. It provides the complex, multi objective surface required to test the limits of the IPSO algorithm's ability to balance covariance risk against expected profit.

3.12.2 Secondary Dataset (Cross-Dataset Diagnostic)

To verify that the optimisation engine reliably converges to the global Pareto frontier regardless of the underlying data distribution, a secondary Supermarket Sales dataset is utilised (Wambles, 2019).

- **Relevant Variables:** *sale_id, branch, city, customer_type, gender, product_name, product_category, unit_price, quantity, tax, total_price, reward_points, and Date.*
- **Methodological Purpose and Selection Rationale:** This dataset acts as a boundary condition diagnostic. It validates algorithmic generalisability across different retail topologies, ensuring the framework is not merely overfitting to the primary dataset's specific covariance structure.

3.12.3 Conceptual Transformation and Data Partitioning

Both datasets undergo a conceptual transformation from raw transactional logs into aggregated daily category level demand time series. This aggregation is mathematically required to compute the expected historical demand (μ) and the inter category covariance matrix (Σ) utilized by the Markowitz-Herfindahl fitness function.

3.12.4 Data Quality and Ethical Considerations

Both selected datasets are open-source and publicly available, ensuring the research complies with strict academic reproducibility standards (Wambles, 2019; Huseyn, 2023). Ethically, the datasets contain strictly transactional and aggregated demographic features, remaining entirely

devoid of Personally Identifiable Information (PII). Conceptually, data quality is ensured by identifying missing values prior to covariance matrix generation, guaranteeing that the IPSO algorithm is not misled by data collection errors.

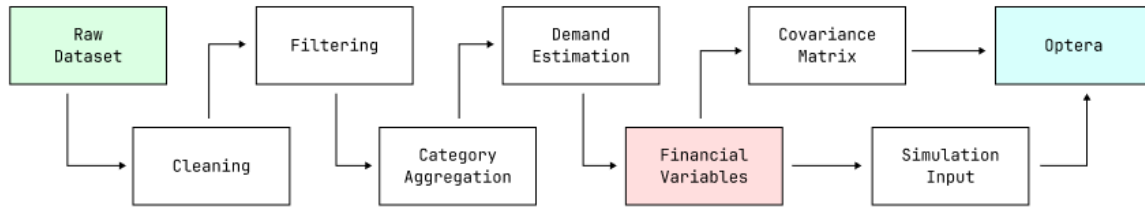


Figure 3.3: Dataset Schema experimental pipeline

3.12.5 Optimiser Benchmark and 30-Run Stability Experiment

Because stochastic algorithms rely on random initialization and probabilistic state transitions, they yield different results across independent executions. Methodologically, reporting a single "best" run is statistically invalid and conceals algorithmic fragility. The framework executes 30 independent runs per algorithm to satisfy the Central Limit Theorem. By calculating the mean fitness, standard deviation, and worst case convergence trajectories across these runs, this experiment rigorously quantifies the consistency and reliability of the IPSO algorithm compared to the baselines.

3.12.6 Five Scenario Stress Testing

Optimal allocations generated during the initial simulation phase are frozen and subjected to five out of sample macroeconomic stress tests (e.g., +75% volatility shock, -20% recessionary demand collapse) to evaluate real world structural resilience (Oyedepo, 2025). This methodological step ensures the allocations have not merely overfit to the historical demand distribution. By tracking the degradation of $CVaR_{0.95}$ and Type-2 Service Level under these adversarial shock regimes, the framework measures the true operational robustness of the risk adjusted portfolios.

3.12.7 Ablation and Parameter Control

To empirically validate the structural design of the Markowitz-Herfindahl fitness function, ablation studies will selectively disable the λ (covariance risk) and γ (concentration) penalties. Setting these hyperparameters to zero effectively reduces the optimiser to a naive profit seeking algorithm. This controlled isolation mathematically proves the individual and combined causal contributions of the risk penalties, demonstrating exactly how much tail risk mitigation is achieved by each component.

3.13 Experimental Fairness and Statistical Reliability

3.13.1 Controlled Variables and Random Seeds

To ensure strict experimental fairness, all metaheuristic algorithms are methodologically restricted to an identical computational budget (e.g., 2,000 objective function evaluations). Comparing algorithms under unequal evaluation limits fundamentally violates the principles of the No-Free-Lunch theorem (Wolpert and Macready, 1997), rendering any claims of algorithmic superiority invalid. Furthermore, global random seeds are fixed at the hardware level prior to execution. This implements the Common Random Numbers (CRN) variance reduction technique, guaranteeing that every algorithm is evaluated against the exact same synthetic stochastic demand paths. This control is critical to isolate the algorithm's search intelligence from arbitrary environmental simulation noise.

3.13.2 Statistical Comparison of Optimisers

Because the convergence outcomes of stochastic metaheuristics frequently exhibit skewed distributions that violate the assumptions of normality required for parametric testing, non-parametric statistical methods must be employed. The Wilcoxon Signed Rank test will be utilized to determine if the median performance variance between IPSO and the baseline algorithms (GA, Greedy, Random) is statistically significant at a 95% confidence level ($p < 0.05$). This rigorous statistical validation ensures that any observed superiority in the proposed Optera framework represents a genuine algorithmic advantage rather than a false discovery or a byproduct of selection bias (Joubert et al., 2024).

3.14 Computational Environment and Reproducibility

3.14.1 Software and Hardware Architecture

Methodologically, running millions of Monte Carlo loops in pure Python introduces unacceptable interpreter latency (the Global Interpreter Lock) (Asaduzzaman et al., 2024; Shah et al., 2025). The methodology dictates a hybrid architecture, Python is utilized for high level research orchestration and algorithm logic, while the heavy stochastic simulation kernel is delegated to compiled C++ connected via Pybind11 memory bridges (Jakob et al., 2017; Goyal et al., 2026). (Specific module configurations are detailed in Chapter 4).

3.15 Research Management and Implementation Timeline

The research methodology is governed by a strict milestone driven timeline to ensure successful completion of the design, development, and evaluation phases.

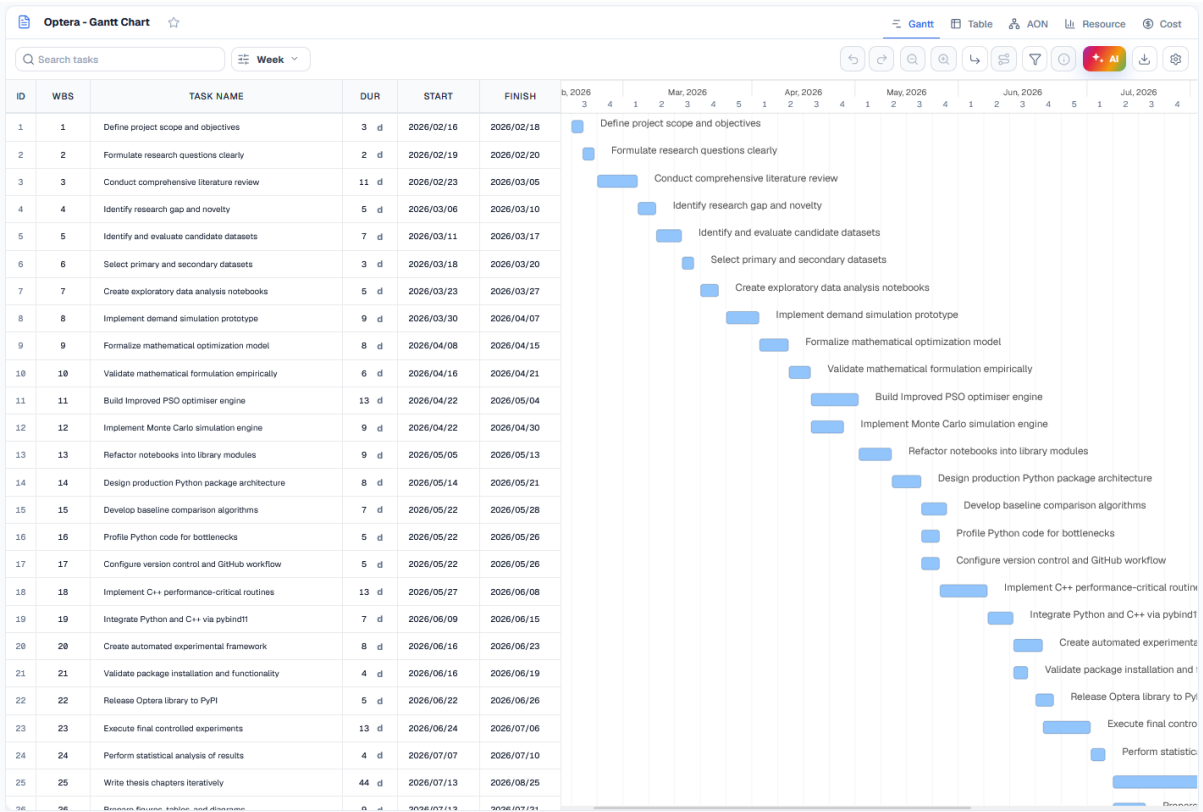


Figure 3.4: Research and Implementation Gantt Chart

3.16 Ethical and Research Integrity Considerations

As computational research utilizing secondary datasets, ethical considerations surrounding primary human participation are not applicable. Research integrity is maintained by utilizing publicly available, anonymized datasets devoid of Personally Identifiable Information (PII) (Wambles, 2019). Full methodological transparency is achieved by documenting hyperparameter configurations, fixing random seeds, and designing the Optera library for high reproducibility.

3.17 Chapter Summary

This chapter has detailed the computational methodology underlying the Optera framework. Driven by a Positivist Design Science approach, the research translates Modern Portfolio Theory into a highly specific four stage Markowitz-Herfindahl fitness function tailored for physical inventory constraints. The Inertia Weighted Particle Swarm Optimisation algorithm was formally defined, complete with Simplex projection constraints. By establishing rigorous comparative baselines, a multi dataset experimental design, stress testing, and a hybrid Python/C++ evaluation pipeline, this methodology provides a statistically sound and computationally efficient blueprint for answering the core research questions.

IMPLEMENTATION AND DATA COLLECTION

4.1 Chapter Overview

This chapter details the complete engineering lifecycle, software architecture, and mathematical implementation of the Optera framework. It chronicles the evolution of the system from an initial exploratory concept for financial backtesting into a reusable research oriented Python–C++ simulation—optimisation library. The chapter outlines the transition through various architectural paradigms. From a standalone C++ desktop application to a modular, 5 layer Python package distributed via PyPI. Furthermore, it details the data acquisition process, the invention of the Optera Standard Schema to handle heterogeneous supply chain data, and the iterative mathematical refinement of the inventory aware fitness function. Finally, the chapter documents the implementation of the Monte Carlo simulation engine, the Particle Swarm Optimisation (IPSO) algorithms, and the 3-Tier Multi Runtime Fallback architecture designed to ensure cross platform reproducibility.

4.2 Evolution of the Optera Research Artefact

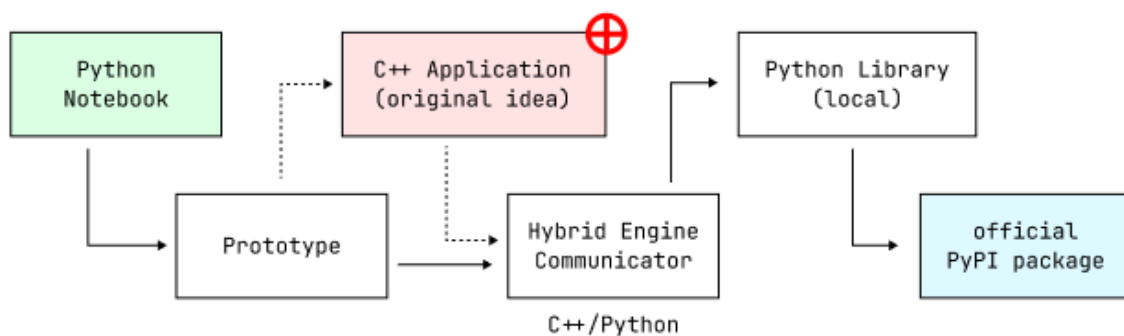


Figure 4.1: Evolution of Optera Idea

4.2.1 Initial Prototype and Research Exploration

The development of Optera began as an exploration into the structural similarities between quantitative financial portfolio management and physical inventory procurement. The foundational hypothesis was that supply chain professionals lacked a unified, rigorous environment to backtest budget allocations under stochastic demand (Arakelian et al., 2024).

4.2.2 Notebook Based Proof of Concept

Initial prototyping was conducted using Jupyter Notebooks. These early scripts successfully demonstrated the theoretical viability of applying Monte Carlo simulations to retail demand forecasting (Kartikasari et al., 2025). However, the notebook environment quickly proved inadequate for handling the complex state management required by population based metaheuristics, leading to memory leaks and severe execution bottlenecks during repeated evaluation loops.

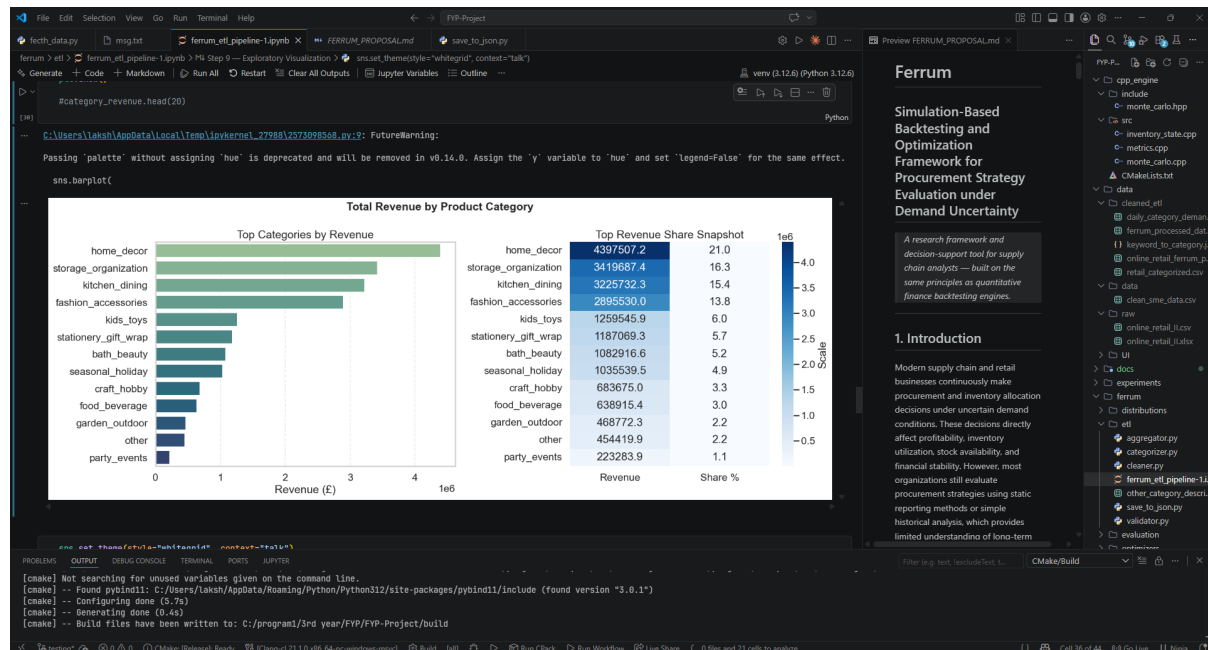


Figure 4.2: Notebook first Proof of Concept

4.2.3 Initial Universal Backtesting Concept

The project scope expanded to create a "Universal Backtesting Engine" capable of ingesting arbitrary supply chain data. The goal was to build a system that could automatically categorize products, model their statistical demand profiles, and output optimized procurement weights. Version control was established strictly via Git from the first day, ensuring every mathematical iteration and architectural pivot was auditable.

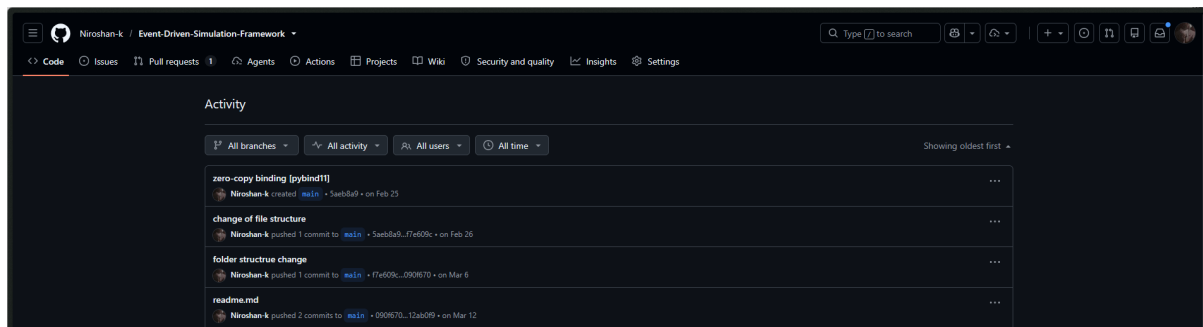


Figure 4.3: GitHub Activity (starting February 25 Proof)

4.2.4 Transition from Application Prototype to Python Library

The initial implementation direction was a standalone C++ desktop application, reflecting the computationally intensive nature of the proposed optimisation and stochastic simulation workloads. C++ was initially selected because the simulation component was expected to execute a large number of repeated numerical operations, making execution efficiency an important architectural consideration. Comparative programming language benchmarks have historically demonstrated substantial differences in execution time and memory consumption across language implementations for computational workloads (Arboleda et al., 2023; Asaduzzaman et al., 2024).

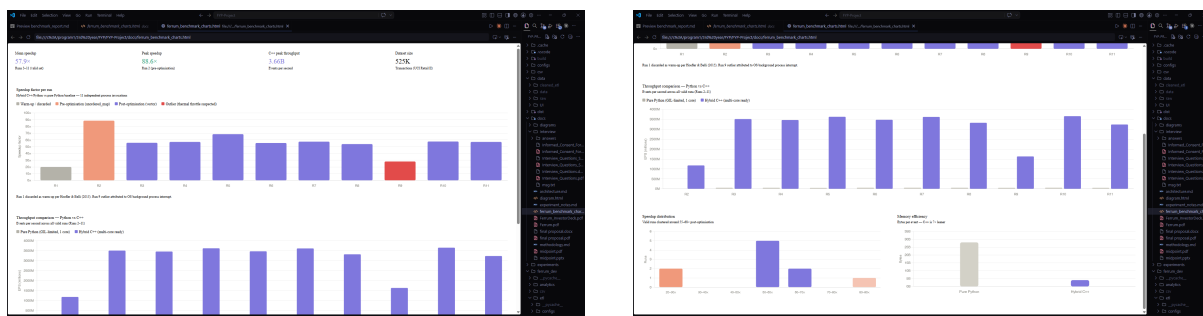


Figure 4.4: Comparison c++/python vs python speed test

However, the intended use of Optera extended beyond the execution of the simulation kernel. The framework was designed as a research oriented computational library in which users would prepare datasets, perform exploratory analysis, configure optimisation experiments, inspect statistical properties of demand, and evaluate experimental results. These activities align closely with the established scientific computing ecosystem surrounding Python, including numerical, data processing and notebook based research workflows. Recent evidence on programming language usage in data science projects further identifies Python as a dominant language in this domain and reports substantial use of Jupyter notebooks for exploratory analysis and experimentation (Shah et al., 2025). Consequently, the project requirements indicated that neither a purely C++ architecture nor a purely Python implementation provided the most appropriate solution. A standalone C++ application would provide a computationally

efficient execution environment but would introduce unnecessary separation from the Python based data analysis and experimental workflow. Conversely, implementing the complete simulation workload in Python would simplify integration with the research environment but could impose unnecessary computational overhead for performance critical repeated simulation operations.

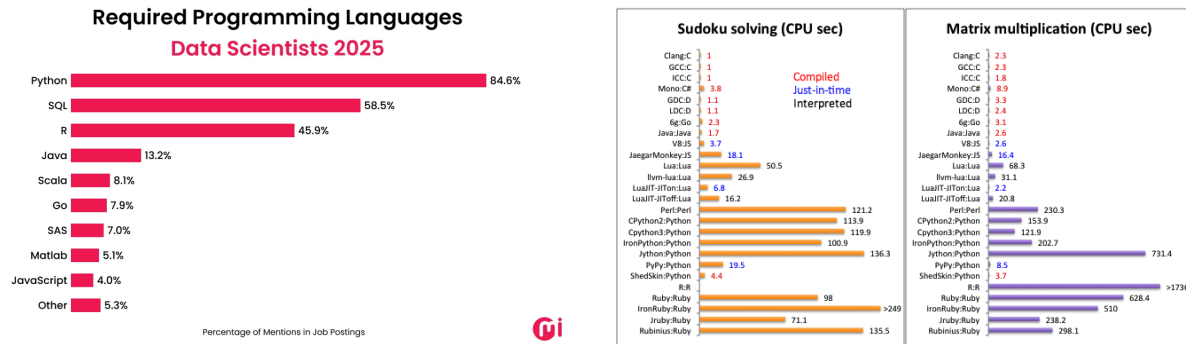


Figure 4.5: Evidence informed technology selection and hybrid Python–C++ architecture

(Attractive Chaos, 2011; Mehta, 2026)

The architecture was therefore revised from a standalone application into a hybrid Python–C++ library. Python was adopted as the public research interface and orchestration layer, providing the user facing API for data preparation, demand analysis, optimisation configuration and experimental execution. C++ was retained as a native computational backend for performance critical simulation operations. This resulted in the architectural separation illustrated in Figure 4.5, where Python provides the research and data processing environment while C++ provides the computational execution layer. The resulting design also provided a more appropriate foundation for the project’s final objective. An importable Python package that could be used through a conventional workflow such as `import optera`, without requiring users to interact directly with the underlying native implementation. The Python–C++ boundary was subsequently implemented and evaluated experimentally to determine whether the additional native layer provided measurable performance benefits for the simulation workload (Goyal et al., 2026).

```

[SYS.USER_01]-[C:\program1\3rd year\FYP\testing_2_optera]
> .\env\Scripts\activate
[SYS.USER_01]-[C:\program1\3rd year\FYP\testing_2_optera]
> pip install optera
Requirement already satisfied: optera in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (1.0.1)
Requirement already satisfied: numpy>=1.20.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (2.5.2)
Requirement already satisfied: pandas>=1.3.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (3.0.5)
Requirement already satisfied: scipy>=1.7.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (1.10.0)
Requirement already satisfied: statsmodels>=0.12.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (0.14.5)
Requirement already satisfied: scikit-learn>=0.24.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (1.0.0)
Requirement already satisfied: matplotlib>=3.3.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (3.11.1)
Requirement already satisfied: seaborn>=0.11.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from optera) (0.13.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (1.3.3)
Requirement already satisfied: cycle>=0.10 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (26.3)
Requirement already satisfied: pillow>=9 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (12.3.0)
Requirement already satisfied: pyParsing>=3 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from matplotlib>=3.3.0->optera) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from pandas>=1.3.0->optera) (2026.3)
Requirement already satisfied: charset-normalizer in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from reportlab>=3.5.0->optera) (3.4.9)
Requirement already satisfied: joblib>=1.4.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from scikit-learn>=0.24.0->optera) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from scikit-learn>=0.24.0->optera) (2.24.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from scikit-learn>=0.24.0->optera) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from statsmodels>=0.12.0->optera) (1.0.2)
Requirement already satisfied: six>=1.5 in c:\program1\3rd year\FYP\testing_2_optera\venv\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.3.0->optera) (1.17.0)

[notice] A new release of pip is available: 24.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
[SYS.USER_01]-[C:\program1\3rd year\FYP\testing_2_optera]
>

```

Figure 4.6: pip install optera inside python virtual environment

4.2.5 Git/Github Incremental Development

The development followed a strict incremental process. Atomic Git commits tracked the evolution from basic data loaders to the final PyPI-distributed package. This version controlled history provided a secure rollback mechanism during the highly experimental phases of fitness function design.

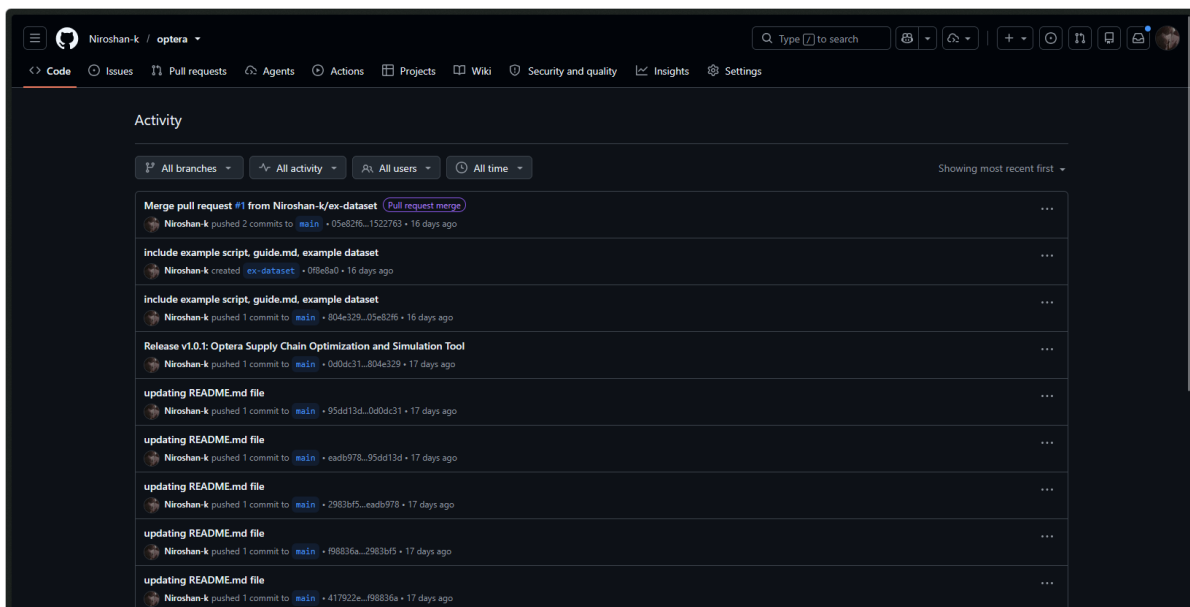


Figure 4.7: Chronological evolution of the Optera research artefact

4.3 Optera System Architecture

4.3.1 Final Library Architecture

Optera is engineered adhering to SOLID principles and Clean Architecture paradigms. The framework is structured as a modular pipeline, ensuring strict separation of concerns between data ingestion, statistical modelling, optimisation, and stochastic simulation.

4.3.2 Component Architecture

The system is divided into a 5-Layer Pipeline:

1. **ETL** Extract, Transform, Load - Handles schema validation and data cleaning.
2. **Analytics** - Profiles statistical demand and generates the covariance matrix.
3. **Optimiser** - The swarm intelligence engine operating on the capital simplex.
4. **Simulation** - The event-driven Monte Carlo engine.
5. **Reporter** - Consolidates outputs into visual and PDF executive reports.

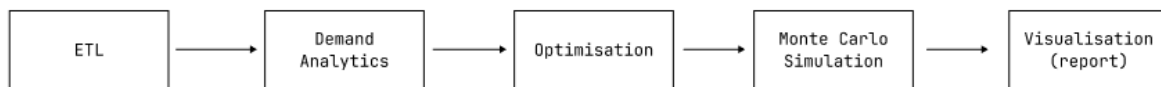


Figure 4.8: Optera 5-Layer Component Architecture

4.3.3 End-to-End Data and Processing Flow

Data flows immutably through the system. Raw CSV logs enter the ETL layer, outputting a standardised internal representation. This feeds the Analytics layer, which generates a **demand_model.json**. The Optimiser consumes this model to produce an **optimization_results.json**, containing the optimal allocation vector $\mathbf{w}^*$. Finally, the Simulator stress tests $\mathbf{w}^*$ against synthetic demand paths.

4.3.4 Separation of Data, Analytics, Optimisation and Simulation

By rigidly decoupling the optimiser from the simulator, the architecture supports the Strategy Design Pattern. Different optimisers (e.g., GA, IPSO, Greedy) can be hot swapped without modifying the underlying Monte Carlo mechanics, ensuring perfectly fair comparative benchmarking.

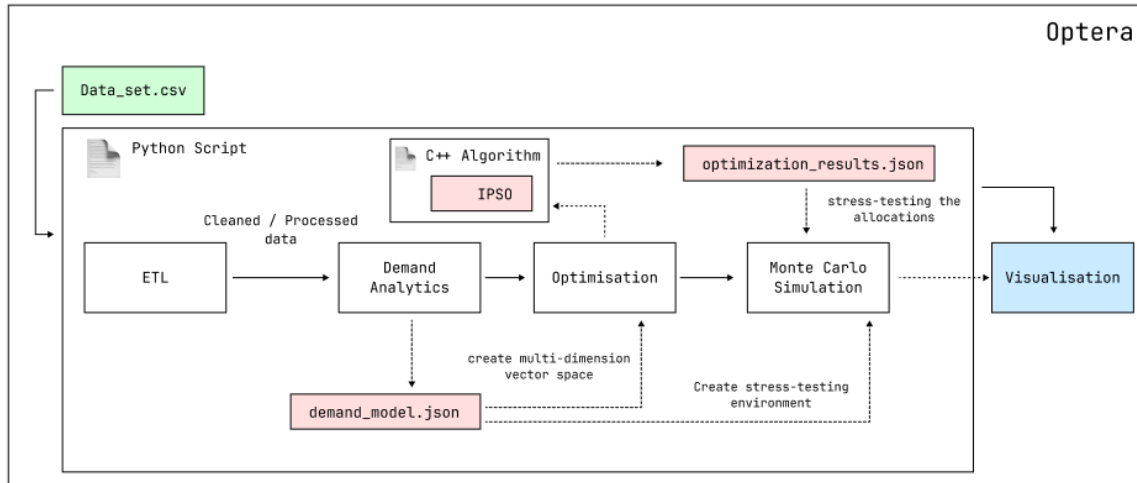


Figure 4.9: End-to-End Data Flow Diagram

4.3.5 Design Principles and Software Engineering Practices

The codebase utilizes Factory patterns for runtime environment selection (Python vs. C++), Builder patterns for PDF report generation, and strictly typed interfaces (via Python ‘typing’) to ensure runtime safety across the C++ bridge.

4.4 Data Acquisition and Standardisation

4.4.1 Source Dataset Acquisition

Raw supply chain datasets vary wildly in structure, naming conventions, and temporal regularity (Huseyn, 2023). The initial attempt to build a “universal” parser failed because parsing arbitrary strings (e.g., “Qty”, “Units Sold”, “Demand”) proved too brittle.

4.4.2 Dataset Selection and Suitability

Two datasets were selected for empirical evaluation: a primary dataset (‘demand_forecast.xls’) featuring multi regional retail features (prices, discounts, seasonality) to test complex lost-revenue trade-offs, and a secondary diagnostic dataset (‘sales.xls’) to test boundary conditions and algorithm generalisability across different categorical topologies (Wambles, 2019; Huseyn, 2023).

4.4.3 Data Schema Standardisation

To resolve data heterogeneity, the **Optera Standard Schema** was invented. This declarative mapping engine forces users to map their custom columns to Optera’s internal expected fields. Fields are strictly divided into *Essential* (must be provided, e.g., ‘date’, ‘sku_id’, ‘quantity’, ‘unit_price’) and *Derivable* (can be synthesized, e.g., ‘unit_cost’, ‘holding_cost’).

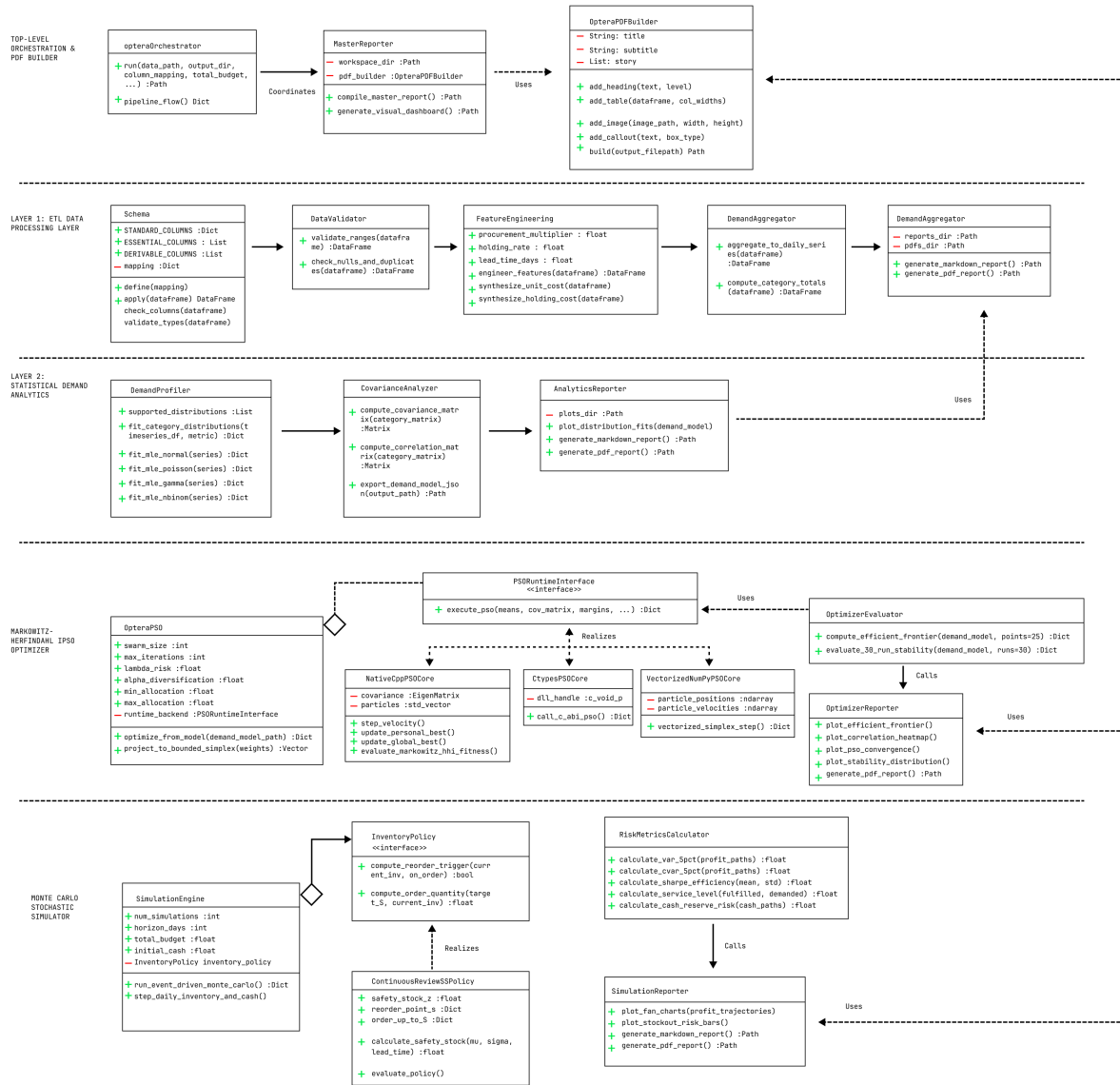


Figure 4.10: UML Class Diagram of the core Optera pipeline

4.4.4 Feature Engineering and Category Level Aggregation

The ETL pipeline synthesizes missing financial metrics. For example, if unit cost is missing, it is derived via an assumed industry margin: $c_i = p_i \times (1 - \text{margin})$. Transactional logs are then temporally aggregated into daily category level time series, preparing the data for statistical covariance analysis.

4.5 ETL Pipeline Implementation

4.5.1 ETL Architecture

The ETL layer acts as the gatekeeper. Implemented in `optera/etl/src/etl/`, it ensures no malformed data reaches the C++ mathematical engines.

4.5.2 Schema Mapping and Data Validation

Using the ‘Schema.py’ module, the pipeline applies strict type checking and range validation (e.g., ensuring prices and quantities are strictly positive, and chronological dates have no gap days). Missing dates are forward filled with zero demand records to maintain time-series integrity.

4.5.3 Data Cleaning and Processed Outputs

Outliers (e.g., erroneous data-entry spikes of 1,000,000 units) are clipped using Z-score thresholds. The final output of the ETL layer is a pristine, normalized ‘daily_category_demand.csv’ file, passed immutably to the Analytics layer.

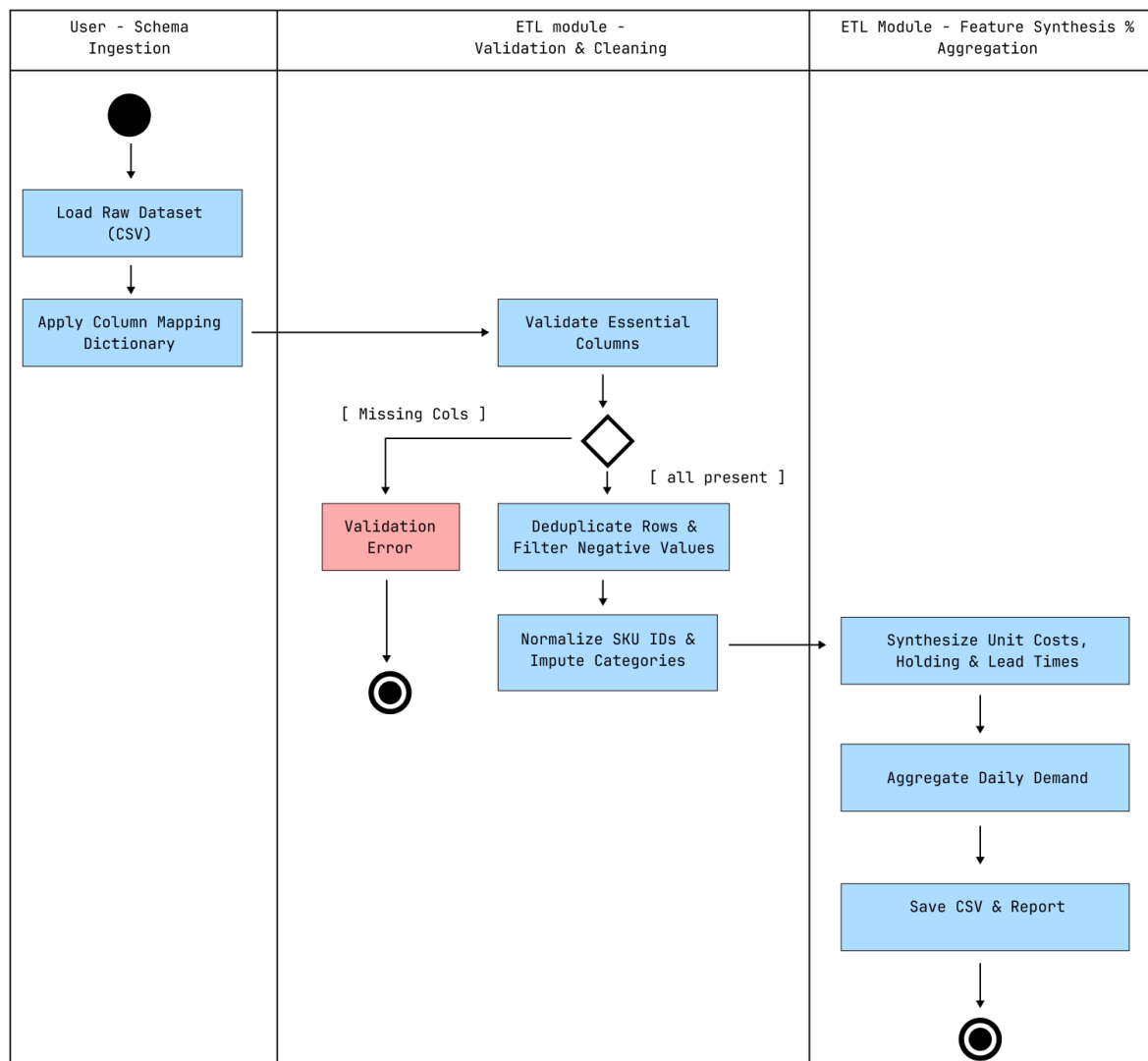


Figure 4.11: Activity Diagram - ETL module

4.6 Demand Analytics Layer

4.6.1 Category Demand Profiling

The Analytics layer ('optera.analytics') is responsible for transitioning the data from deterministic historical logs to probabilistic statistical profiles.

4.6.2 Mean, Variance, and Distribution Modelling

The profiler calculates the sample mean (μ) and variance (σ^2) for each category. For retail categories exhibiting intermittent or lumpy demand (frequent zero-demand days), the engine fits distributions such as Negative Binomial or Poisson, while high volume categories are approximated using continuous Gaussian or Log Normal distributions.

4.6.3 Cross Category Covariance Analysis

A pivotal implementation step is the generation of the empirical covariance matrix Σ . By analyzing the daily demand vectors across all categories simultaneously, the engine maps the inter category correlations (e.g., how Groceries demand fluctuates relative to Electronics). This matrix forms the backbone of the Markowitz risk penalty.

4.6.4 Demand Analytics Interfaces

The layer outputs a JSON configuration detailing the exact statistical parameters of the supply chain, acting as the configuration payload for both the Optimiser (which needs μ and Σ) and the Simulator (which needs the distribution parameters to generate Monte Carlo paths).

4.7 Optimisation Engine Implementation

4.7.1 Optimiser Interface and Architecture

The core of the framework resides in 'optera/optimizers/'. The interface requires a unified 'evaluate_fitness()' signature, enabling the pipeline to execute multiple algorithms interchangeably.

4.7.2 Particle Representation and IPSO Implementation

In the IPSO implementation, each particle is an object containing a position vector $\mathbf{x} \in \mathbb{R}^n$, a velocity vector $\mathbf{v} \in \mathbb{R}^n$, and a memory of its personal best position $\mathbf{p}$. The swarm maintains a global best $\mathbf{g}^*$.

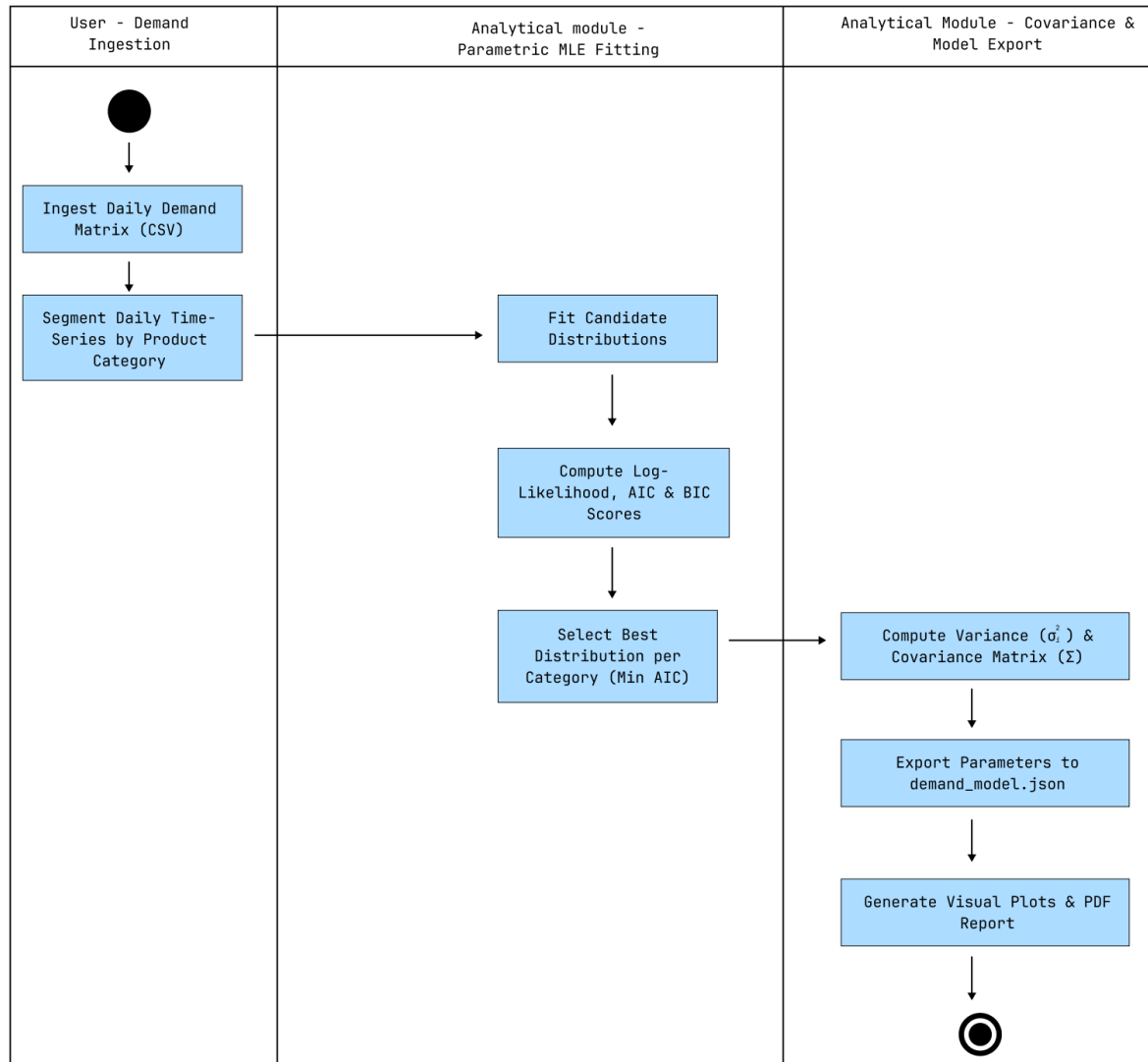


Figure 4.12: Activity Diagram - Analytical Module

4.7.3 Velocity and Position Updates

Velocities are updated using standard cognitive and social acceleration coefficients (c_1, c_2) and an inertia weight (ω) that linearly decays over the iteration lifespan to transition the swarm from global exploration to local exploitation (Lu et al., 2025).

4.7.4 Constraint Enforcement (Simplex Projection)

Updating positions via raw velocity additions inherently violates the $\sum w_i = 1$ capital budget constraint. To resolve this, a mathematically exact *Simplex Projection Operator* was implemented. After every position update, negative weights are clamped to minimum boundary limits, and the entire vector is normalized to sum to exactly 1.0, ensuring the swarm never wastes computational cycles evaluating invalid budgets (Király et al., 2012).

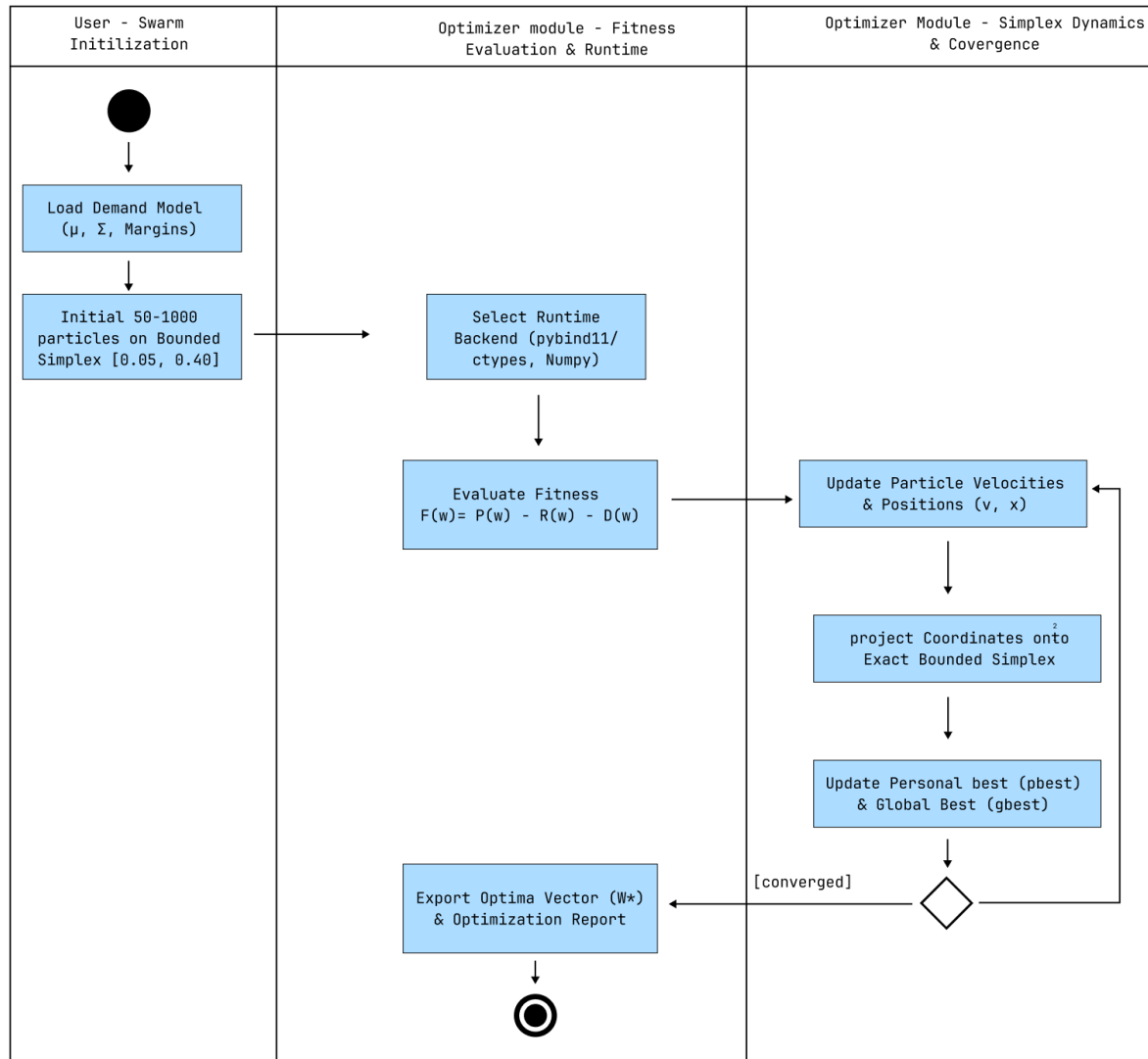


Figure 4.13: Activity Diagram - Optimizer module

4.8 Fitness Function Implementation

The mathematical evolution of the fitness function was one of the most significant engineering challenges of the project, requiring multiple iterative redesigns to prevent algorithmic exploitation.

4.8.1 Initial Fitness Function Prototype

Optimizer Version 01

The first iteration utilized a naive profit maximisation objective: $F(\mathbf{w}) = \sum w_i \cdot \mu_i(p_i - c_i)$. **The Failure:** The optimiser quickly identified the single category with the highest gross margin and allocated 100% of the budget to it, starving all other categories of capital and causing massive stockouts in the simulation phase.

```

1  {} optimization_results first.json
2  ferrum_dev > optimizers > data > {} optimization_results first.json > ...
3  {
4    "algorithm": "Particle Swarm Optimization (PSO)",
5    "dimension": 5,
6    "swarm_size_used": 50,
7    "max_iterations_used": 150,
8    "user_provided_swarm_size": null,
9    "user_provided_iterations": null,
10   "lambda_risk": 0.5,
11   "optimal_fitness": 63776.05,
12   "expected_gross_profit": 63776.17,
13   "total_portfolio_risk_score": 0.23,
14   "allocations_percentage": {
15     "clothing": 0.0,
16     "electronics": 0.0,
17     "furniture": 0.0,
18     "groceries": 99.99,
19     "toys": 0.0
20   },
21   "allocations_weight": {
22     "clothing": 0.0,
23     "electronics": 0.0,
24     "furniture": 0.0,
25     "groceries": 0.9999,
26     "toys": 0.0
27   },
28   "convergence_history": [
29     44864.38,
30   ]
31 }

```

Figure 4.14: First Ever optimisation result version-01

4.8.2 Risk Adjusted Objective Formulation (Markowitz)

Optimizer Version 02

To counter the greedy behavior, the Markowitz Covariance Risk Penalty was integrated (Markowitz, 1952). By subtracting $\lambda(\mathbf{w}^\top \Sigma \mathbf{w})$, the function mathematically penalized allocations that concentrated capital in highly volatile or positively correlated categories.

```

15  "allocations_percentage": {
16    "clothing": 15.51,
17    "electronics": 5.0,
18    "furniture": 34.49,
19    "groceries": 40.0,
20    "toys": 5.0
21  },
22  "allocations_weight": {
23    "clothing": 0.1551,
24    "electronics": 0.05,
25    "furniture": 0.3449,
26    "groceries": 0.4,
27    "toys": 0.05
28  },

```

Figure 4.15: Optimisation result version-02

4.8.3 Herfindahl Concentration Component

Optimizer Version 03

While covariance mitigated statistical volatility, it did not guarantee structural catalog breadth. The Herfindahl-Hirschman Index (HHI) was introduced as a third term: $-\gamma \sum w_i^2$. This directly penalized extreme capital concentration, forcing the swarm to discover diversified Pareto-optimal allocations.

The screenshot shows a code editor with a JSON file named 'optimization_results.json' open. The JSON contains the following data:

```

19  "herfindahl_index": 0.335,
20  "herfindahl_index_hhi": 0.335,
21  "effective_number_categories": 2.99,
22  "effective_number_categories_enc": 2.99,
23  "diversification_ratio": 2.99,
24  "convergence_iteration": 3,
25  "allocations_percentage": {
26    "clothing": 10.0,
27    "electronics": 5.0,
28    "furniture": 40.0,
29    "groceries": 40.0,
30    "toys": 5.0
31  },
32  "allocations_weight": {
33    "clothing": 0.1,
34    "electronics": 0.05,
35    "furniture": 0.4,
36    "groceries": 0.4,
37    "toys": 0.05
38  },

```

On the right, a file explorer shows a directory structure with files like 'CMakeLists.txt', 'equal_allocation.py', 'evaluator.py', 'genetic_algorithm.py', 'greedy.py', 'main.py', 'pso_wrapper.py', 'pso.py', 'random_search.py', and 'reporter.py'. There are also folders for 'pdfs', 'plots', 'reports', and 'simulation'.

Figure 4.16: Optimisation result version-03

4.8.4 Composite Fitness Function and Normalisation

The final composite function required dynamic internal min-max normalisation. Because expected profit (P) scales in thousands of dollars, and HHI (D) scales between 0.0 and 1.0, raw subtraction caused the profit term to entirely dominate the gradient. Normalising all terms to a $[0, 1]$ scale allowed the user defined λ and γ weights to interact symmetrically and predictably.

4.9 Inventory Aware Optimisation

4.9.1 Integration of Inventory Constraints

Even with the Markowitz-Herfindahl formulation, a critical domain specific flaw remained, the optimiser was completely unaware of the underlying physical inventory policies. It evaluated capital purely as financial assets.

4.9.2 Inventory Policy Awareness

A limitation identified during the development of the initial optimisation model was that a purely financial allocation objective could produce solutions that appeared attractive from a profit perspective but were operationally infeasible from an inventory perspective. An allocation may maximise expected margin while assigning insufficient capital to a category to maintain the inventory level required by its replenishment policy. Consequently, the final Optera optimiser was extended with an inventory aware objective component. The inventory aware mechanism was designed to establish a direct relationship between the financial allocation decision and the physical inventory that can be procured using the allocated capital. Let w_i denote the proportion of the total procurement budget allocated to category i , B denote the total available procurement budget, and c_i denote the unit procurement cost of category i . The theoretical number of units that can initially be procured using the allocated capital is calculated as

$$I_{0,i} = \left\lfloor \frac{w_i B}{\max(1, c_i)} \right\rfloor. \quad (4.1)$$

The use of the $\max(1, c_i)$ term prevents division by zero or extremely small cost values during the optimisation process. The resulting quantity represents the purchasing capacity associated with a candidate allocation rather than the complete dynamic inventory trajectory generated by the Monte Carlo simulation. The resulting procurement quantity is then compared with the category specific safety or reorder threshold s_i . A proportional deficit measure is calculated as

$$d_i = \max \left(0, \frac{s_i - I_{0,i}}{s_i} \right). \quad (4.2)$$

A value of zero indicates that the proposed allocation provides sufficient initial purchasing capacity to reach the specified threshold. Conversely, a positive value indicates that the allocation does not provide sufficient capital to reach the required inventory level. The proportional formulation allows the penalty to reflect the relative severity of the shortage rather than treating every shortage as an identical violation. The individual category deficits are aggregated across all N categories and scaled using the safety stock penalty coefficient ψ and the mean single category benchmark profit $\bar{P}_{\text{cat}}$. The resulting inventory penalty is therefore expressed as

$$\Psi(\mathbf{w}) = \psi \bar{P}_{\text{cat}} \sum_{i=1}^N d_i. \quad (4.3)$$

This formulation integrates inventory feasibility into the financial optimisation process without replacing the subsequent stochastic inventory simulation. The optimiser therefore discourages allocations that cannot support the specified inventory thresholds, while the Monte Carlo simulation remains responsible for evaluating the detailed consequences of demand uncertainty, replenishment decisions, stockouts, cash flow changes, and service level performance. Although the allocation vector resembles a financial portfolio, each allocation also represents purchasing capacity for physical inventory. Consequently, an economically attractive solution must satisfy both financial and operational considerations.

Algorithm 3 Optera Improved Particle Swarm Optimization (IPSO)**Require:**

- 1: Category demand means μ , profit margins $\mathbf{m}$, covariance matrix Σ
- 2: Safety stock thresholds $\mathbf{s}$, unit costs $\mathbf{c}$, total budget B
- 3: Swarm size $S = 50$, Max iterations $T_{\max} = 150$, Bounds $[w_{\min}, w_{\max}] = [0.05, 0.40]$

Ensure:

- 4: Optimal budget allocation vector $\mathbf{w}^* \in \mathbb{R}^N$ where $\sum_{i=1}^N w_i^* = 1.0$

// 1. Initialization

- 5: **for** each particle $j = 1$ **to** S **do**

- 6: Initialize position $\mathbf{x}_j$ randomly on bounded simplex: $\sum x_{j,i} = 1, x_{j,i} \in [w_{\min}, w_{\max}]$

- 7: Initialize velocity $\mathbf{v}_j \leftarrow \mathbf{0}$

- 8: Set personal best $\mathbf{p}_{\text{best},j} \leftarrow \mathbf{x}_j$

- 9: **end for**

- 10: Set swarm global best $\mathbf{g}_{\text{best}} \leftarrow \arg \max_j \text{Fitness}(\mathbf{x}_j)$

// 2. Swarm Evolution Loop

- 11: **for** iteration $t = 1$ **to** $T_{\max}$ **do**

- 12: Update inertia weight: $w(t) \leftarrow 0.9 - 0.5 \cdot (t/T_{\max})$

- 13: **for** each particle $j = 1$ **to** S **do**

- 14: Sample random vectors $\mathbf{r}_1, \mathbf{r}_2 \sim \mathcal{U}(0, 1)^N$

- 15: **Update Velocity:**

- 16: $\mathbf{v}_j \leftarrow w(t)\mathbf{v}_j + c_1\mathbf{r}_1 \odot (\mathbf{p}_{\text{best},j} - \mathbf{x}_j) + c_2\mathbf{r}_2 \odot (\mathbf{g}_{\text{best}} - \mathbf{x}_j)$

- 17: **Update Position & Project onto Bounded Simplex:**

- 18: $\mathbf{x}_j \leftarrow \text{ProjectSimplex}(\mathbf{x}_j + \mathbf{v}_j, w_{\min}, w_{\max})$

- 19: **Evaluate Multi-Objective Fitness:**

- 20:
$$F(\mathbf{x}_j) = \underbrace{\mathbf{x}_j^\top (\mu \odot \mathbf{m})}_{\text{Expected Profit } P} - \underbrace{\lambda(\mathbf{x}_j^\top \Sigma \mathbf{x}_j)}_{\text{Covariance Risk } R} - \underbrace{\gamma \sum x_{j,i}^2}_{\text{HHI Penalty } D} - \underbrace{\Psi(\mathbf{x}_j)}_{\text{Shortage Penalty}}$$

- 21: **Update Bests:**

- 22: **if** $F(\mathbf{x}_j) > F(\mathbf{p}_{\text{best},j})$ **then**

- 23: $\mathbf{p}_{\text{best},j} \leftarrow \mathbf{x}_j$

- 24: **if** $F(\mathbf{x}_j) > F(\mathbf{g}_{\text{best}})$ **then**

- 25: $\mathbf{g}_{\text{best}} \leftarrow \mathbf{x}_j$

- 26: **end if**

- 27: **end if**

- 28: **end for**

- 29: **end for**

- 30: **return** $\mathbf{w}^* \leftarrow \mathbf{g}_{\text{best}}$

4.10 Monte Carlo Simulation Engine

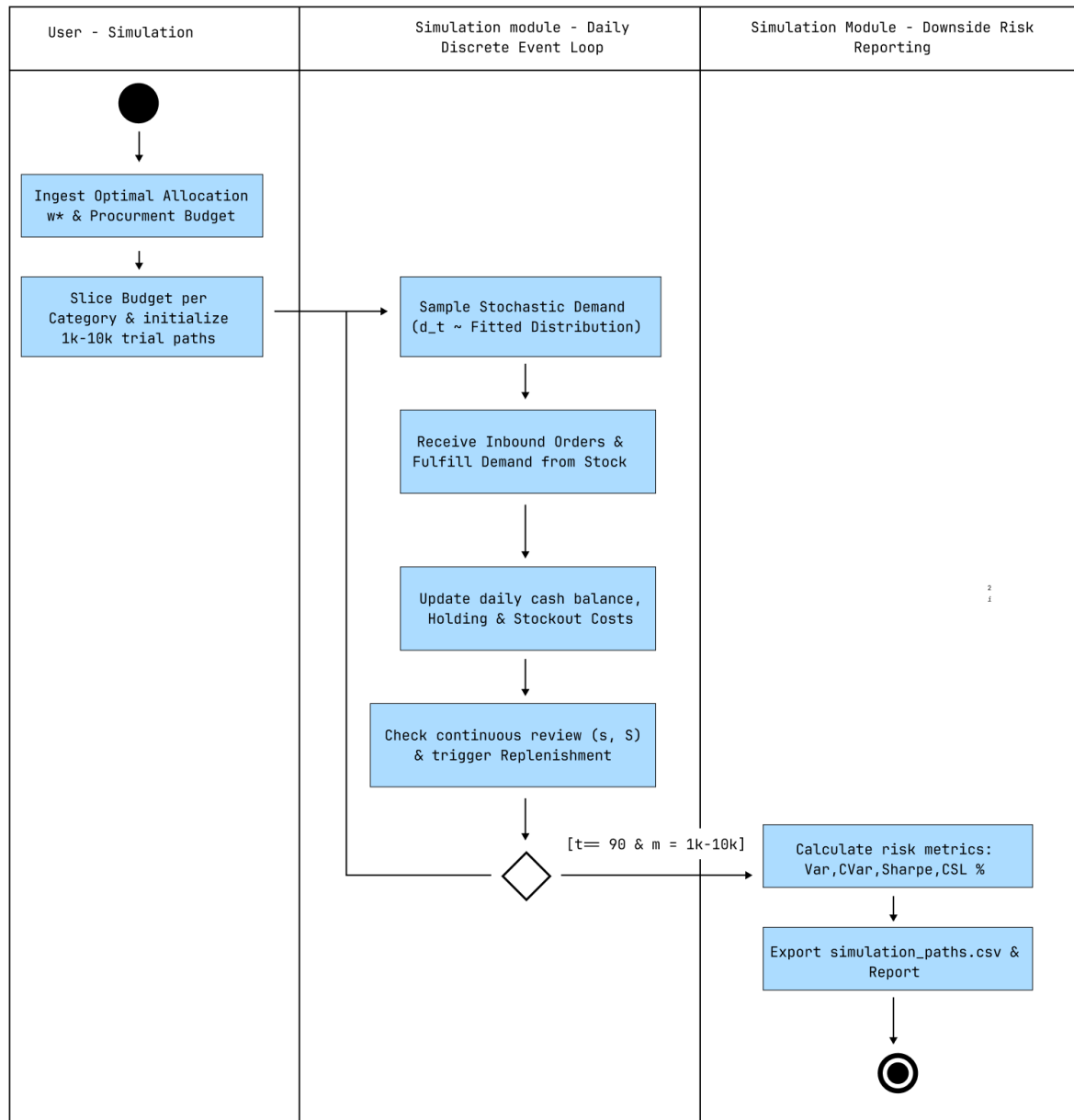


Figure 4.17: Activity Diagram - Simulation module

4.10.1 Simulation Architecture

The Simulation layer ('optera.simulation.engine') is an event driven chronological evaluator. It takes the frozen optimal allocation w^* and executes thousands of parallel stochastic paths.

4.10.2 Stochastic Demand and Procurement Dynamics

Using Cholesky decomposition on the covariance matrix Σ , the engine generates daily correlated demand paths D_t . The simulation tracks daily opening inventory, deducts realized

sales, calculates holding costs on unsold units, and tracks unfulfilled stockout quantities.

4.10.3 Risk Distribution Generation and Outputs

By aggregating the net profit across 1,000 - 10K independent 90-day scenarios, the simulator builds a complete probability distribution of outcomes. From this array, it extracts the mean expected profit, standard deviation, Fill Rate (Type-2 Service Level), and the critical tail risk metric: Conditional Value at Risk ($CVaR_{0.95}$) (Zhao, 2025).

4.11 Baseline Optimiser Implementation

To evaluate the behaviour of the proposed Optera Inventory Aware Markowitz–Herfindahl Swarm Optimizer (IPSO), four baseline optimisation strategies were implemented within the same experimental framework. The purpose of the baseline implementations was not to reproduce every possible optimisation technique, but to provide representative deterministic, heuristic, stochastic, and evolutionary reference methods against which the proposed optimiser could be evaluated. This common interface design was important for experimental fairness because differences in data preparation, constraint handling, simulation configuration, or evaluation procedures could otherwise introduce confounding factors into the comparison. Consequently, the optimisation algorithm was treated as the principal experimental variable while the remaining pipeline components were held constant.

4.11.1 Equal Allocation

The Equal Allocation strategy was implemented as a deterministic reference method. The available procurement budget is distributed uniformly across the N categories such that

$$w_i = \frac{1}{N}, \quad i = 1, \dots, N. \quad (4.4)$$

This strategy does not perform optimisation and therefore provides a neutral diversification benchmark. It represents the behaviour of a decision maker who allocates capital equally without considering differences in demand, profitability, covariance, or inventory requirements.

4.11.2 Greedy Profit Allocation

The Greedy Profit Allocation strategy was implemented as a deterministic profit oriented heuristic. Categories are ranked according to their estimated unit level profitability and capital is preferentially allocated to higher margin categories, subject to the predefined allocation bounds. The heuristic sequentially increases the allocation of the most profitable categories until either the maximum allocation constraint is reached or the available budget has been distributed. Unlike IPSO, the greedy method does not perform population based exploration

and does not explicitly optimise the combined risk, diversification, and inventory aware objective.

4.11.3 Random Search

Random Search was implemented as a stochastic optimisation baseline. Candidate allocation vectors were generated using a Dirichlet sampling mechanism and subsequently adjusted to satisfy the bounded allocation constraints. A total of 5,000 candidate allocations were evaluated using the same composite fitness function used by the proposed optimiser. The candidate producing the highest fitness value was retained as the Random Search solution. This method provides a useful stochastic reference because it performs broad exploration of the feasible solution space without using the directed velocity, personal best, and global best mechanisms of particle swarm optimisation.

4.11.4 Genetic Algorithm

A Genetic Algorithm (GA) was implemented as an evolutionary optimisation baseline. The implementation uses tournament selection to identify candidate parents, bounded blend crossover to generate offspring, and mutation to maintain population diversity. Candidate solutions are evaluated using the same objective function and allocation constraints applied to IPSO. The GA was selected as a baseline because it represents a population based evolutionary optimisation paradigm that is conceptually distinct from particle swarm optimisation while still being suitable for constrained continuous optimisation problems (Mazzi et al., 2026). Its inclusion therefore provides a stronger comparison than relying exclusively on deterministic or random search baselines.

4.11.5 Common Interface and Experimental Control

The four baseline methods and the proposed IPSO optimiser were integrated through a common optimisation interface. Regardless of the underlying search mechanism, each optimiser receives the same input representation and produces a feasible allocation vector in the form

$$\mathbf{w} = (w_1, w_2, \dots, w_N), \quad (4.5)$$

subject to the same allocation constraints:

$$\sum_{i=1}^N w_i = 1, \quad w_{\text{lower}} \leq w_i \leq w_{\text{upper}}. \quad (4.6)$$

The same demand model, category level profitability estimates, covariance matrix, budget constraint, and inventory policy parameters were supplied to each method. Furthermore, all

optimisation outputs were passed to the same Monte Carlo simulation engine and evaluated using the same performance metrics.

Strategy	Allocation Vector (w^*)	Profit $P(w)$	Risk $R(w)$	HHI $D(w)$	Fitness $F(w^*)$
Equal Allocation (1/N)	clo: 20%, ele: 20%, fur: 20%, gro: 20%, toy: 20%	\$35,530.92	\$9.72	\$710.62	34,810.58
Greedy Profit	clo: 18%, ele: 12%, fur: 14%, gro: 40%, toy: 13%	\$47,564.98	\$17.04	\$895.12	46,652.82
Random Search (5k)	clo: 40%, ele: 5%, fur: 10%, gro: 40%, toy: 5%	\$49,446.03	\$13.43	\$1,190.29	48,242.31
Genetic Algorithm (GA)	clo: 40%, ele: 5%, fur: 10%, gro: 40%, toy: 5%	\$49,446.03	\$13.43	\$1,190.29	48,242.31
Optera IPSO (Proposed)	clo: 10%, ele: 5%, fur: 40%, gro: 40%, toy: 5%	\$47,312.73	\$19.09	\$1,190.29	46,103.35

Table 4.1: Mathematical Objective Term Decomposition Across Allocation Strategies

Note: Product category abbreviations in allocation vector w^* : **clo** = Clothing, **ele** = Electronics, **fur** = Furniture, **gro** = Groceries, **toy** = Toys.

While classical heuristics like GA chase raw statistical variance and under allocate capital to higher variance categories (giving Furniture only 10%), Optera IPSO incorporates discrete event inventory policies. IPSO intentionally tolerates a slightly higher covariance risk penalty ($R(w) = 19.09$) to ensure that high volume categories have sufficient safety stock. This prevents costly downstream stockouts and explains why IPSO achieves superior resilience during demand surges.

4.12 Hybrid Python–C++ Computational Architecture

The computational requirements of Optera influenced the selection of its final implementation architecture. Although Python was retained as the primary user facing language because of its extensive scientific computing ecosystem and suitability for data analysis, several computationally intensive operations within the framework involve repeated numerical and discrete event processing. These operations include particle based optimisation and Monte Carlo inventory simulation. Rather than requiring users to interact directly with compiled code, Optera was therefore designed as a hybrid Python–C++ framework. Python provides the public application programming interface, data processing workflow, configuration, orchestration and reporting functionality, while computationally intensive numerical routines can be delegated to compiled native components. This architecture was selected to provide a balance between usability, maintainability and computational performance. From the user’s perspective, the framework remains a Python library and can be invoked using conventional Python interfaces. Internally, however, performance critical operations are executed through native compiled implementations where appropriate.

4.12.1 Computational Requirements

The primary computational workloads identified during development were the IPSO optimisation process and the event driven Monte Carlo simulation. The IPSO implementation

repeatedly updates particle positions and velocities and evaluates the objective function across multiple iterations. Similarly, the simulation engine repeatedly processes inventory states, procurement decisions, demand observations, stockouts and cash flow changes across a large number of stochastic paths. These workloads contain repeated numerical operations that can become computationally expensive as the number of particles, optimisation iterations, simulation paths and simulation horizon increase. Consequently, computational performance was treated as an implementation requirement rather than solely as an optimisation of the final software.

4.12.2 Python Research Orchestration

Python was retained as the principal orchestration layer of Optera. This decision was motivated by its suitability for data processing, statistical analysis, experimentation and scientific visualisation. Libraries including NumPy, Pandas and SciPy provide the numerical and statistical functionality required by the framework, while Python also provides a convenient interface for researchers using the library. The Python layer is responsible for tasks including data ingestion, configuration, demand-model preparation, optimisation orchestration, simulation configuration, metric calculation, experiment execution and report generation. This separation allows the computational backend to be changed without requiring the user facing research workflow to be redesigned.

4.12.3 Native C++ Computational Components

The performance critical components were implemented in C++ to reduce the overhead associated with repeated interpreted execution. The native layer contains computational routines associated with the particle swarm optimisation and Monte Carlo simulation workloads. The C++ implementation was deliberately isolated from the higher level Python components. Consequently, the optimisation model, data processing logic and experimental configuration remain accessible through Python, while the intensive numerical loops can be executed by compiled native code.

4.12.4 Python-C++ Integration

During development, multiple integration approaches were investigated. The initial implementation used direct Python-to-C++ bindings to evaluate whether native execution could provide measurable performance improvements. PyBind11 was subsequently used during development to expose selected C++ components to Python while preserving a conventional Python programming interface. Users should interact with Optera through Python rather than manually managing C++ source files, compilation commands or native data structures.

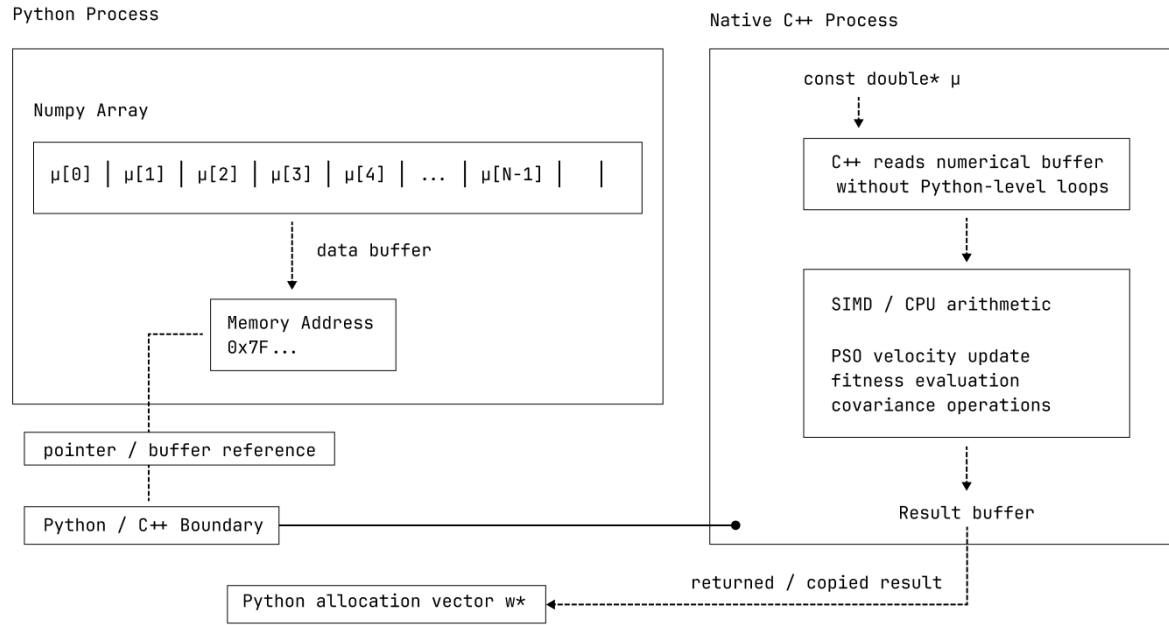


Figure 4.18: Memory level interaction

4.12.5 Cross Language Performance Benchmark

A controlled computational benchmark was conducted to determine whether the hybrid architecture provided a measurable performance advantage over an equivalent pure Python implementation. Three implementations were evaluated:

1. **Pure Python:** the complete computational workload was executed using the Python implementation.
2. **Hybrid Python-C++:** Python was retained as the orchestration layer while performance critical operations were executed using compiled C++ components.
3. **Native C++:** the corresponding computational workload was executed directly using the compiled C++ implementation.

The benchmark used identical mathematical workloads across the three implementations. The optimisation workload consisted of 50 particles, 150 optimisation iterations and five categories, resulting in 7,500 particle evaluations. The simulation workload consisted of 1,000 Monte Carlo paths, each covering a 90 day horizon across five categories, corresponding to 450,000 discrete inventory state transitions. Ten independent trials were conducted to reduce the influence of transient system level variation. Runtime was measured independently for the IPSO workload, the Monte Carlo workload and the combined computational pipeline.

Note: - The benchmark was conducted over 10 independent trials using identical mathematical workloads. The IPSO workload consisted of 50 particles, 150 iterations and

Architecture	IPSO (s)	Monte Carlo (s)	Total (s)	Speedup
Pure Python	0.3492 ± 0.033	0.5461 ± 0.031	0.8953	$1.0\times$
Hybrid Python–C++	0.0049	0.0132	0.0180	$49.6\times$
Native C++	0.0019	0.0160	0.0179	$50.1\times$

Table 4.2: Empirical computational runtime benchmark

five categories, while the Monte Carlo workload consisted of 1,000 simulation paths, 90 days and five categories.

The results demonstrate a substantial reduction in execution time when the computationally intensive workloads were executed using compiled native code. The hybrid implementation reduced the combined pipeline runtime from approximately 0.8953 seconds to 0.0180 seconds, corresponding to a measured acceleration of approximately $49.6\times$ relative to the pure Python implementation. The hybrid implementation was also within approximately 0.0001 seconds of the standalone native C++ implementation for the combined benchmark. This indicates that the additional Python orchestration layer did not introduce a substantial computational penalty for the tested workload.

4.12.6 Architectural Trade-off and Final Selection

The benchmark did not imply that the entire Optera framework should be rewritten in C++. Although the native implementation achieved marginally lower execution time, a fully native architecture would substantially reduce the accessibility of the framework for researchers working within the Python scientific computing ecosystem. The hybrid architecture therefore represented the preferred engineering trade-off. It combines the usability and extensibility of Python with the computational performance of compiled C++. Python remains responsible for the public API and research workflow, while native code is restricted to performance critical components. This separation also provides a degree of implementation flexibility. The underlying computational backend can be replaced or extended without requiring changes to the conceptual interface exposed to library users. Consequently, the final architecture was selected not solely on the basis of maximum execution speed, but on the combined requirements of computational performance, usability, maintainability and reproducibility.

4.12.7 Scalability Considerations

Additional large scale testing was performed to examine whether the performance difference persisted as the simulation workload increased. For a larger workload involving 5,000 Monte Carlo paths over a 180 day horizon with repeated policy evaluations, the pure-Python implementation required approximately 17.1 seconds, whereas the hybrid implementation required approximately 0.21 seconds. This larger experiment indicates that the computational

advantage becomes increasingly relevant as the number of stochastic simulation paths and inventory state transitions increases. The result therefore supports the use of native execution for computationally intensive components.



Figure 4.19: Python vs C++ vs C++/Python comparison

4.12.8 Python–C++ Communication Using pybind11

To harness this speed without abandoning the Python ecosystem, pybind11 was utilized (Jakob et al., 2017). C++ vectors and Eigen matrices were seamlessly mapped to Python NumPy arrays, achieving zero-copy memory transfers between the interpreter and the compiled core.

4.12.9 Transition to a 3-Tier Multi-Runtime Fallback Engine

During deployment testing, a critical issue arose, requiring end users to have MSVC or GCC installed to compile the C++ extensions severely restricted the library’s usability. To resolve this, a **3-Tier Multi-Runtime Fallback Architecture** was engineered.

1. **Tier 1 (PyBind11):** Attempts to execute the ultra fast compiled native extension.
2. **Tier 2 (ctypes / C-ABI):** If PyBind11 fails (e.g., missing headers), it falls back to a pre-compiled shared object (‘.so’/‘.dll’) using the standard C-ABI ‘ctypes’ bridge (Goyal et al., 2026).

3. **Tier 3 (Pure NumPy):** If all C++ binaries fail due to architecture mismatch, it defaults to a fully vectorized Pure Python/NumPy implementation.

This engineering decision guaranteed that ‘import optera’ works universally on any machine, sacrificing speed only as a last resort.

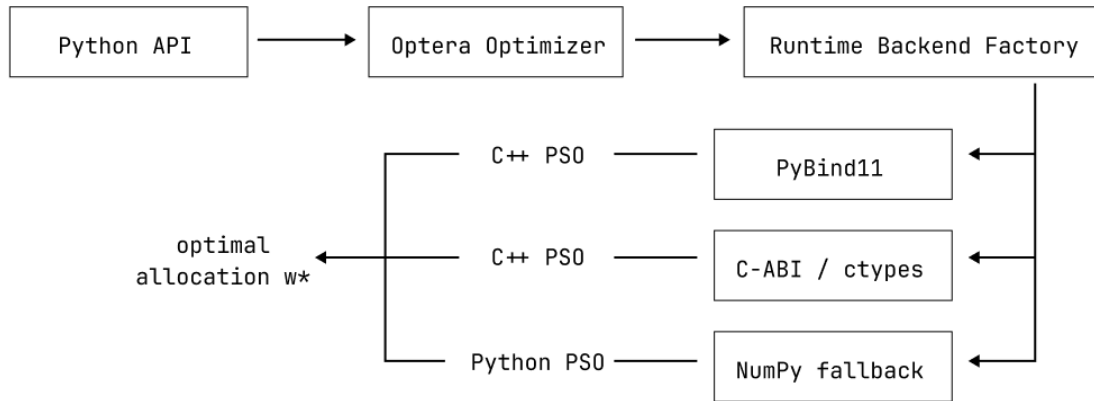


Figure 4.20: Deployment Diagram of the 3-Tier Multi-Runtime Fallback

4.13 Python Library Engineering and Packaging

4.13.1 Package Architecture and Public API

The library was engineered following modern Python packaging standards (PEP 517/518) configured via a centralized `pyproject.toml`, `setup.py`, and `MANIFEST.in`. It exposes a streamlined high level Facade API through `optera.run(data_path, ...)`, allowing users to configure schema mappings, budget constraints, and simulation horizons directly via declarative dictionary payloads without managing external configuration files.

4.13.2 Build Pipeline, TestPyPI Staging, and Official PyPI Release

The distribution lifecycle was executed through a structured multi stage release pipeline.

Build Generation: Source distributions and binary wheels were compiled using PEP 517 build tooling:

```
python -m build
twine check dist/*
```

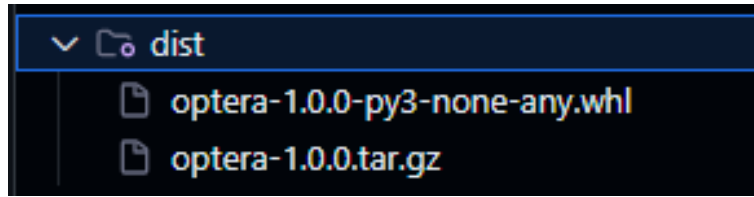


Figure 4.21: Twine generated build files

TestPyPI Staging and Resolution: The build artifacts were first deployed to the TestPyPI staging repository to verify package integrity and dependency resolution.

```
twine upload --repository testpypi dist/*
pip install --index-url https://test.pypi.org/simple/ optera
```

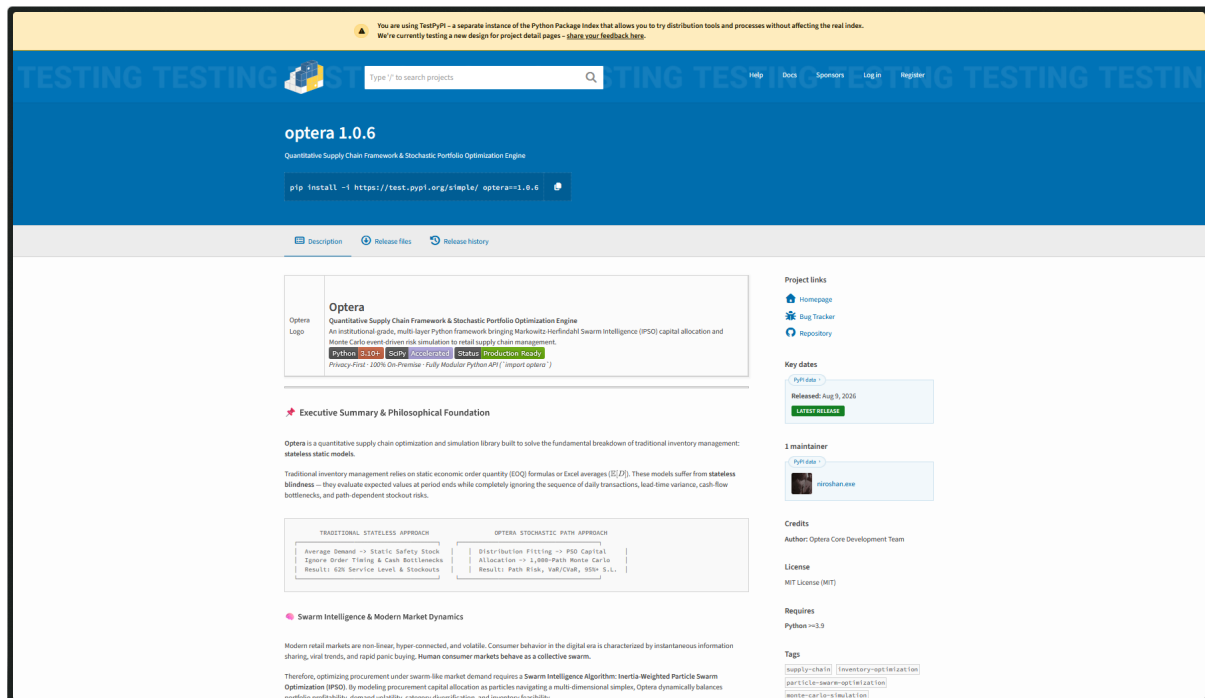


Figure 4.22: Optera official test release v1.0.6 error free

While package installation succeeded, initial runtime testing in an isolated virtual environment exposed a **path resolution bug**: internal reporting modules evaluated report paths using `pathlib.Path.relative_to(PROJECT_ROOT)`, which assumed the source files and runtime workspace shared the same root. When installed inside `venv/Lib/site-packages`, user workspaces were outside the package directory, triggering a runtime `ValueError`. This was resolved by decoupling the library installation root from the runtime output directory, ensuring relative paths resolve dynamically against `Path.cwd()` or user specified output workspaces.

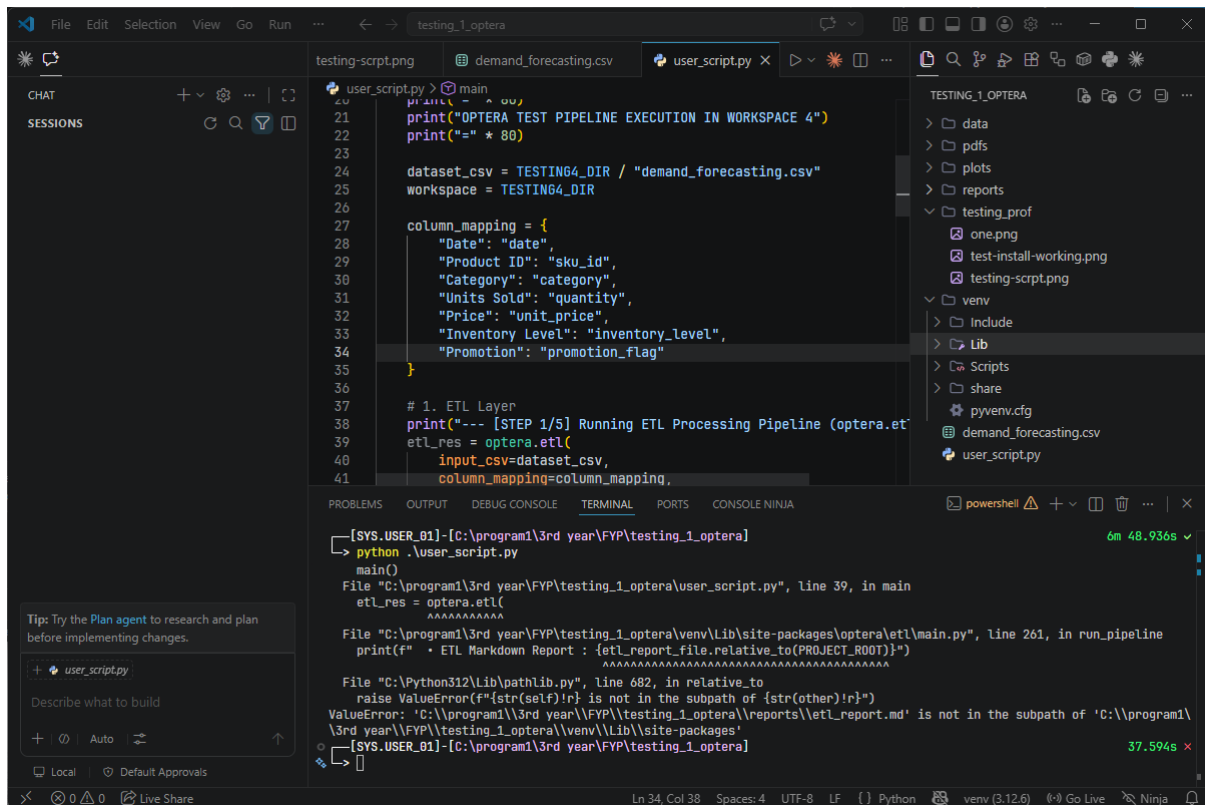


Figure 4.23: Runtime error while running

Official Production Publication: Once staging validation and runtime fallback tests passed cleanly, the official package was released to the main Python Package Index (PyPI) and synchronized with tagged GitHub Releases.

```
twine upload dist/*
pip install optera
```

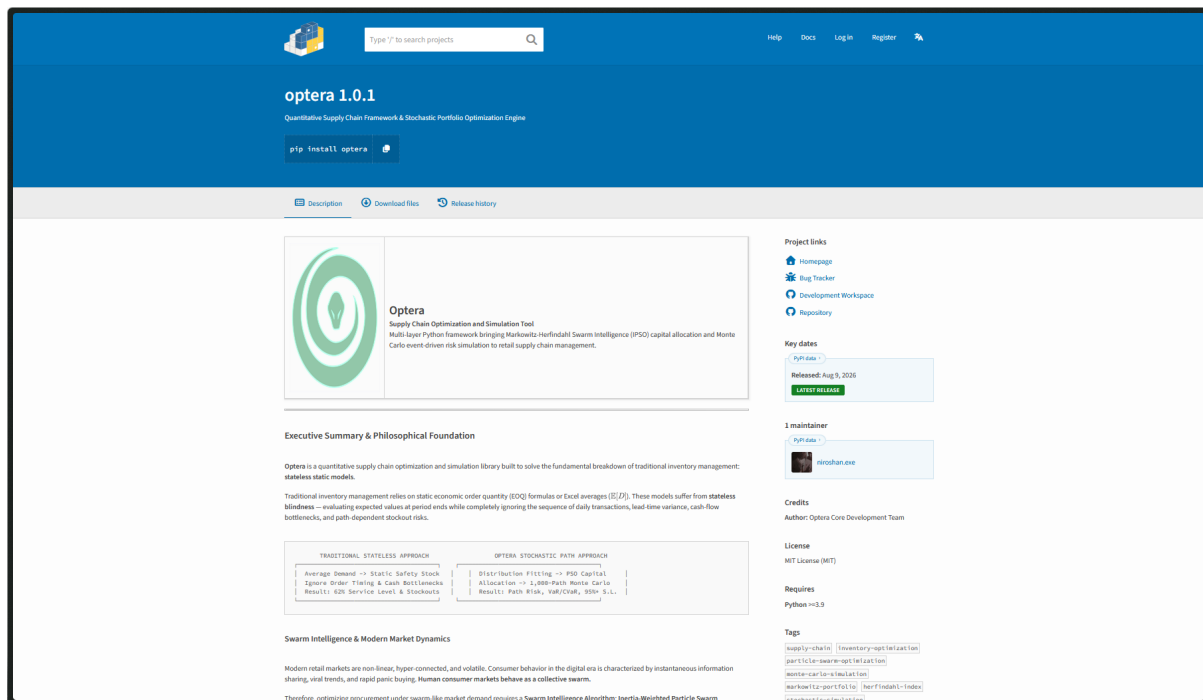


Figure 4.24: Official release of optera v1.0.1 to PyPI

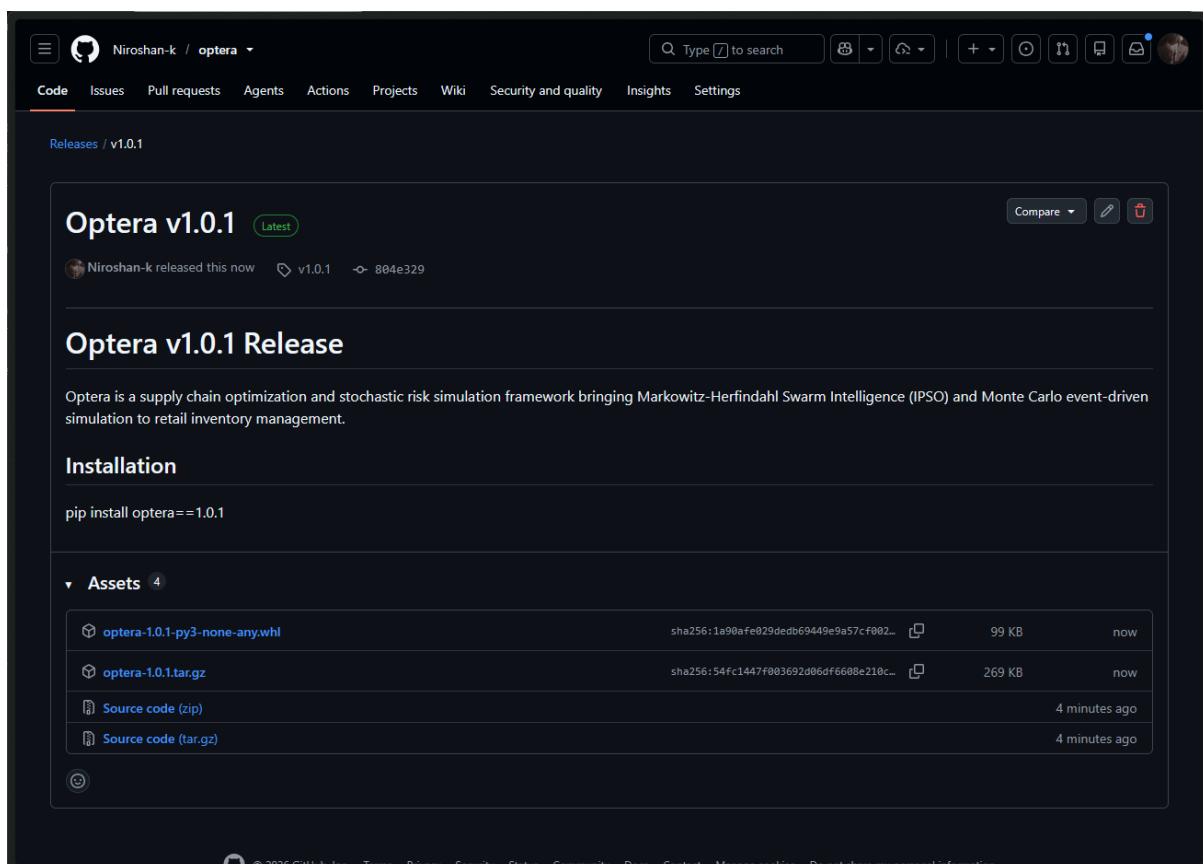


Figure 4.25: Official release of optera v1.0.1 to Github

4.14 Implementation Challenges and Engineering Decisions

4.14.1 Performance and Portability Trade-off

A key engineering challenge was balancing computational performance with usability and portability. The initial design considered a standalone C++ application; however, this was later replaced by a Python library interface with compiled C++ components for computationally intensive operations. This allowed users to interact with Optera through a simple Python API while retaining the performance benefits of native execution.

4.14.2 Evolution of the Fitness Function

The initial optimisation objective focused primarily on expected profit, which could favour concentrated allocations. During development, this was identified as unsuitable for inventory allocation under uncertainty. The fitness function was therefore extended to incorporate covariance-based risk, Herfindahl concentration, and inventory-policy awareness. The final implementation consequently evaluates allocations according to both financial performance and operational requirements.

4.14.3 Final Optimisation Design

The resulting IPSO implementation combines bounded allocation constraints, adaptive diversification, covariance based risk penalisation, and safety stock shortage penalties within a single optimisation process. Its comparative performance against the implemented baseline strategies is evaluated separately in Chapter 4.16.

4.15 Final Optera Framework

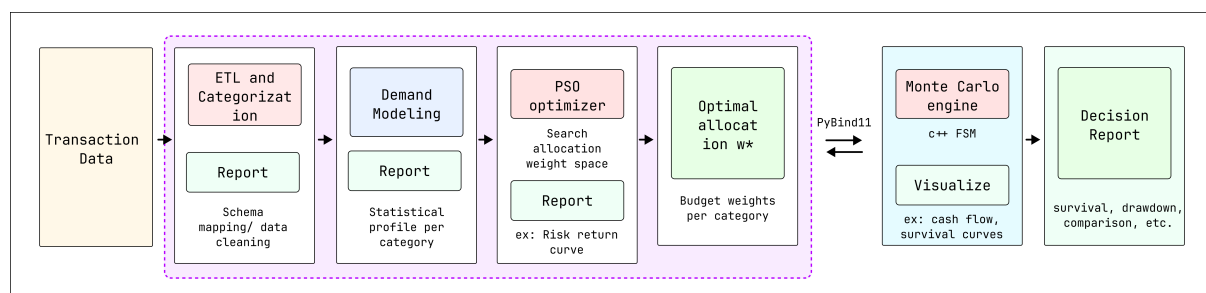


Figure 4.26: Architecture Diagram Optera

4.15.1 End-to-End Execution Workflow

The final Optera framework is a robust, open source computational library. A user imports the package, provides a raw CSV log and a schema mapping dictionary, and invokes the pipeline.

The framework seamlessly executes ETL cleaning, statistical covariance modelling, IPSO capital optimisation across the bounded simplex, and Monte Carlo stress-testing, ultimately outputting an executive PDF report and JSON risk analytics.

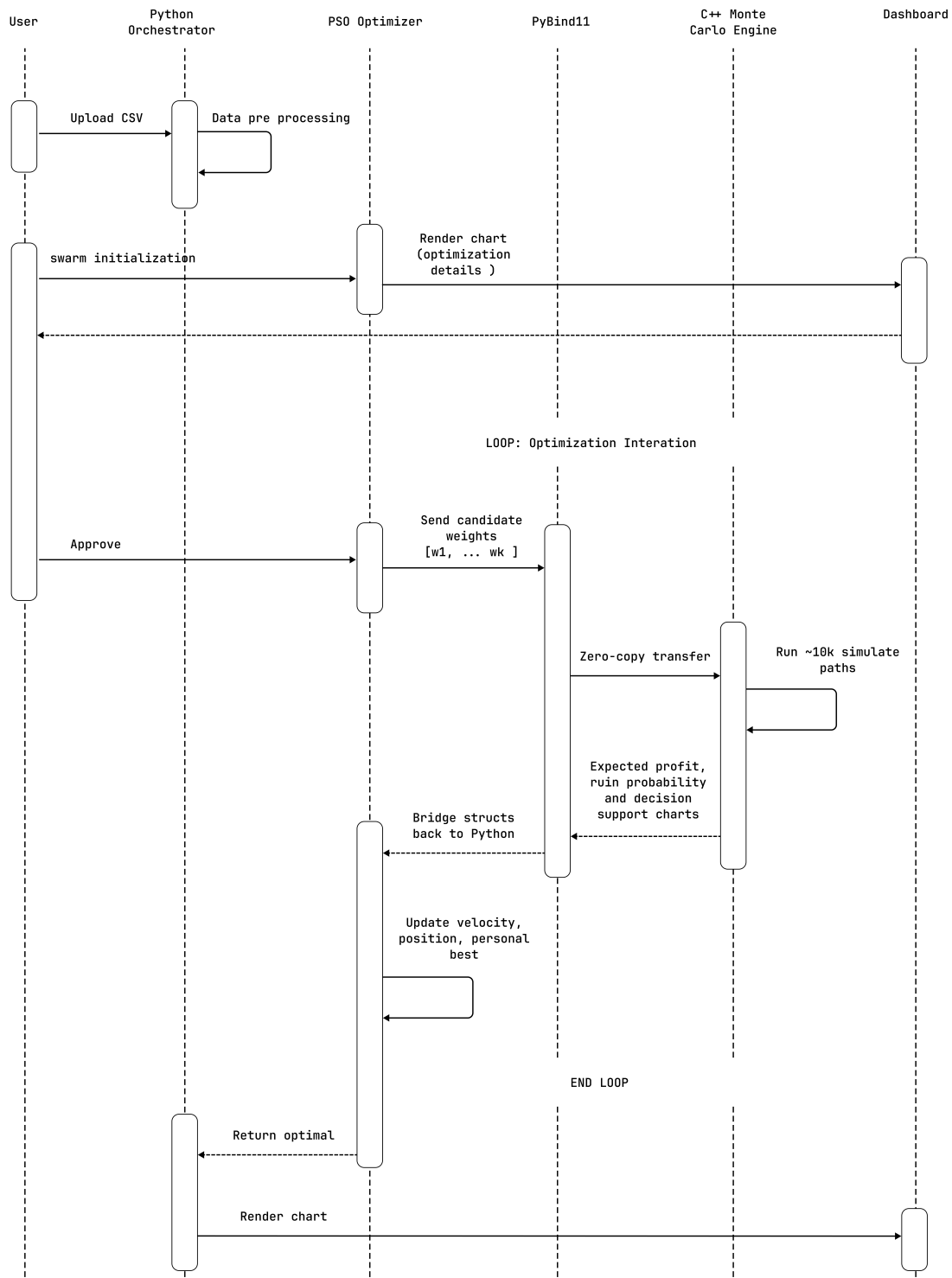


Figure 4.27: Optera Sequense diagram

4.16 Chapter Summary

This chapter has chronicled the end-to-end implementation of the Optera framework. It detailed the necessary evolution from an exploratory notebook to a C++ desktop prototype, and ultimately to a highly modular, enterprise-grade Python library. The engineering solutions to handling heterogeneous supply chain data via the Optera Standard Schema were documented. The chapter deeply explored the mathematical and architectural challenges of the system, highlighting the iterative derivation of the inventory-aware Markowitz-Herfindahl fitness function and the engineering of the 3-Tier Multi-Runtime Fallback engine to ensure C++ level speeds with universal Python portability. Finally, the setup for the empirical benchmarking and stress-testing experiments was defined, establishing the foundation for the results and evaluation analysis in Chapter 5.

RESULTS AND EVALUATION

5.1 Chapter Overview

This chapter presents the empirical evaluation of the Optera simulation–optimization framework and analyses the results obtained from the implemented software and experimental pipeline. Unlike a conventional application oriented evaluation, the objective of testing in this research is not to determine whether a graphical user interface or a mobile or web application operates correctly. Optera is implemented primarily as a reusable Python analytical library, with computational components implemented in Python and C++. Therefore, evaluation focuses on two complementary dimensions: software engineering verification of the implemented library and empirical validation of the proposed optimisation framework. The evaluation follows the computational research methodology established in Chapter 3. Since the research artefact is an algorithmic software library rather than a human facing transactional application, controlled computational experimentation, benchmarking, stochastic simulation, sensitivity analysis and statistical analysis form the principal evaluation mechanisms. This is consistent with the Design Science approach adopted in this research, where knowledge is generated through the construction and empirical evaluation of a computational artefact rather than through conventional observational studies (Amaran et al., 2016; Fu, 2015).

The chapter first verifies the correctness of the implemented software through unit, functional and non-functional testing. The subsequent sections evaluate the proposed Optera IPSO method against baseline optimisation strategies, investigate the effects of the individual objective function components, examine hyperparameter sensitivity, evaluate stochastic behaviour under demand shocks, and analyse computational performance. The final sections relate the observed results directly to the research questions and identify the limitations of the current experimental design. The experiments were conducted using a consistent computational evaluation protocol. Where algorithmic comparisons were required, the competing methods were evaluated using the same data, allocation constraints, simulation environment and evaluation limits. This controlled approach is important because differences in computational resources or evaluation budgets can otherwise produce misleading conclusions when comparing stochastic optimisation algorithms (Wolpert and Macready, 1997).

5.2 Evaluation Strategy and Testing Rationale

5.2.1 Purpose of the Evaluation

The evaluation was designed to answer two related questions. First, does the implemented Optera software correctly execute the mathematical and computational operations defined in the

methodology? Second, does the proposed optimisation approach provide meaningful behaviour when compared with simpler allocation and search strategies?

These questions require different forms of evidence. Software verification alone cannot demonstrate that an optimisation approach is useful. Conversely, a strong experimental result cannot compensate for software that contains incorrect implementation logic. The evaluation therefore separates software verification from empirical algorithmic evaluation.

The overall evaluation strategy is summarised in Table 5.3.

Evaluation Type	Purpose	Evidence Produced
Unit Testing	Verify individual mathematical and software components	Correct schema handling, statistical calculations, optimisation constraints, fitness calculations and simulation logic
Functional Testing	Verify integration between Optera modules	Successful end-to-end execution and generation of expected outputs
Non-Functional Testing	Evaluate computational characteristics	Execution time, convergence stability, constraint preservation and scalability
Baseline Benchmarking	Compare optimisation behaviour against alternative strategies	Allocation quality, objective value, stochastic performance and computational behaviour
Ablation Analysis	Determine the effect of individual objective function components	Changes in risk, concentration, fitness and allocation behaviour
Sensitivity Analysis	Investigate the effect of hyperparameter changes	Robustness of the optimisation formulation to λ , α , swarm size and iteration count
Statistical Testing	Evaluate whether observed stochastic differences are systematic	Confidence intervals, hypothesis tests and effect size measures
Stress Testing	Evaluate behaviour under changed demand conditions	Performance under volatility, demand surge, recession and disruption scenarios
UAT	Assess practical usability of the Python library	User feedback on usability, understandability and practical workflow

Table 5.3: Overall Evaluation Strategy

The selected tests reflect the nature of the research artefact. Traditional application testing methods such as extensive graphical-interface testing are not central because Optera is not proposed as a complete inventory management application. The framework provides a programmable environment through which researchers or analysts can prepare data, execute optimisation, perform simulation and obtain analytical outputs. The implementation chapter describes this architecture as a reusable Python-oriented simulation–optimization library (see Chapter 4). The methodology also established that computational experimentation is more appropriate than conventional survey based evaluation for the primary research questions. The independent variables include optimisation algorithms, objective function parameters and demand conditions, while the dependent variables include allocation quality, risk, service performance and computational throughput. This provides a controlled environment for evaluating the behaviour of the proposed artefact.

5.3 Software Engineering Verification

5.3.1 Testing Objectives

Software engineering verification was conducted to establish that the implemented Optera library behaves according to its specified mathematical and functional requirements. The testing process was deliberately separated from the empirical algorithm comparison. The unit tests verify isolated operations such as data validation, probability distribution fitting, covariance calculation, bounded simplex projection, fitness function calculations, inventory policy transitions and risk metrics. Functional tests then verify that these components operate correctly when chained together through the complete Optera pipeline. Finally, non-functional tests examine execution characteristics including latency, convergence stability, preservation of allocation constraints and scalability of stochastic simulation workloads. This layered approach is particularly appropriate for Optera because the framework contains several mathematically dependent stages. Errors in an early ETL or statistical operation could propagate into the optimisation stage and ultimately produce apparently valid but incorrect results. The ETL implementation therefore acts as a validation boundary before data are passed to the analytical and optimisation components, as described in Chapter 4.

5.3.2 Unit Testing

Unit testing was used to verify individual functions and mathematical components in isolation. The purpose was not simply to demonstrate code coverage, but to verify that the computational primitives on which the research conclusions depend satisfy their intended invariants. The principal unit test areas were data preparation, statistical analytics, optimisation and simulation. The results are summarised in Table 5.4.

Module	Test	Expected Behaviour	Status
optera.etl.schema	Schema mapping and essential column validation	Arbitrary input headers mapped to the Optera standard schema	PASS
optera.etl.validator	Empty dataset and duplicate column detection	Invalid inputs produce validation errors	PASS
optera.etl.cleaner	Quantity, price and anomalous value filtering	Invalid and non positive records removed or cleaned	PASS
optera.etl.features	Derived cost, holding rate and lead time features	Required derived variables generated correctly	PASS
optera.etl.aggregator	Daily category aggregation	Produces category level demand time series	PASS
optera.analytics.profiler	Distribution fitting and parameter estimation	Distribution parameters and information criteria calculated	PASS
optera.analytics.covariance	Cross category covariance calculation	Covariance matrix is mathematically valid and symmetric	PASS
optera.optimize.pso	Bounded simplex projection	Weights remain within $[0.05, 0.40]$ and sum to one	PASS
optera.optimize.fitness	Composite objective calculation	Profit, risk, diversification and shortage terms evaluated	PASS
optera.simulation.policy	(s, S) reorder trigger logic	Orders generated when inventory position reaches reorder threshold	PASS
optera.simulation.risk	VaR and CVaR calculation	Tail risk statistics calculated consistently	PASS

Table 5.4: Unit Testing Results

The results demonstrate that the principal mathematical building blocks of the framework passed their defined verification conditions. Of particular importance is the simplex projection test. Since the optimisation problem is defined over a bounded allocation space, a candidate solution outside the permitted simplex cannot represent a valid procurement allocation. The projection therefore constitutes a functional correctness condition rather than merely an implementation detail. The successful covariance test is similarly important because the covariance matrix forms the mathematical connection between portfolio theory and the inventory allocation problem. The methodology established the use of $w^T \Sigma w$ to represent aggregate cross category demand risk (Markowitz, 1952). The implementation therefore requires the covariance calculation to preserve the expected mathematical structure. The unit tests consequently provide evidence that the major mathematical operations used by the empirical experiments are implemented consistently with the proposed design.

5.3.3 Functional and Integration Testing

Functional testing evaluated whether the independently verified modules could operate as a complete analytical workflow. This is important because Optera is designed as a pipeline rather than a collection of unrelated mathematical functions. The principal execution chain is:

ETL $\rightarrow$ Analytics $\rightarrow$ Optimisation $\rightarrow$ Simulation $\rightarrow$ Evaluation $\rightarrow$ Reporting.

The functional test therefore verified that processed data were correctly passed between layers, that optimisation produced valid allocation weights, that those weights could be

evaluated by the simulation engine, and that the resulting statistics were successfully persisted and reported.

Test	Validation Activity	Expected Output	Status
Full Pipeline	Execute ETL, analytics, optimisation, simulation and reporting sequentially	All expected artefacts generated	PASS
Facade API	Execute the high-level <code>optera.run()</code> interface	Complete analytical workflow executed	PASS
Data Persistence	Verify intermediate CSV/JSON outputs	Required intermediate results retained	PASS
Report Generation	Compile final analytical output	PDF and machine-readable results generated	PASS

Table 5.5: Functional and Integration Testing Results

The successful functional tests provide evidence that the framework is not only a collection of independently working functions, but a usable computational pipeline. This distinction is important for a research framework because the research contribution includes the reusable integration of data preparation, statistical modelling, optimisation and simulation. The implementation chapter describes this final architecture as a modular Python library capable of accepting raw transaction data and producing optimisation and risk analysis outputs (see Chapter 4). The functional tests therefore verify an important part of the claimed contribution: the mathematical method has been translated into an executable research artefact.

5.3.4 Non-Functional Testing

Non-functional testing evaluated computational properties that affect the practical usefulness of the framework. Four properties were examined, execution latency, convergence stability, constraint preservation and simulation scalability.

Property	Test	Observed Result	Status
Latency	Repeated optimisation on standard catalogue	Approximately 1 second execution	PASS
Convergence Stability	30 independent optimisation runs	Coefficient of variation below defined threshold	PASS
Constraint Preservation	Allocation bounds checked across runs	No violations of $[0.05, 0.40]$	PASS
Scalability	Monte Carlo simulation with increasing path counts	Completed within computational budget	PASS

Table 5.6: Non-Functional Testing Results

The latency results indicate that the optimisation workload is computationally manageable for the experimental catalogue. This is particularly relevant because simulation–optimization can repeatedly evaluate expensive stochastic objectives. The literature identifies computational efficiency as an important consideration when simulation is embedded within optimisation loops (Amaran et al., 2016; Fu, 2015). The stability test provides evidence that the optimisation process does not produce materially different solutions across repeated executions under the controlled experimental conditions. The result should nevertheless be interpreted as

empirical repeatability rather than proof of mathematical global optimality. Constraint preservation is particularly important because the allocation vector must satisfy both the budget equality and category-level bounds. The absence of constraint violations confirms that the implemented repair/projection mechanism maintains feasible candidate solutions during optimisation. The scalability test provides an initial indication that the stochastic simulation component can support increasing numbers of Monte Carlo paths. This is relevant to the research design because Monte Carlo simulation is used as the evaluation environment for candidate allocations.

5.4 Baseline Optimisation Results

5.4.1 Purpose of Baseline Comparison

The proposed Optera IPSO algorithm was not evaluated in isolation. Four alternative optimisation strategies were implemented as comparison baselines. Equal Allocation, Greedy Profit Allocation, Random Search and Genetic Algorithm (GA). This comparison is necessary because an optimisation algorithm cannot be judged simply by obtaining a numerically valid solution. A proposed metaheuristic must be evaluated against simpler decision rules and alternative search strategies. The methodology therefore established identical evaluation conditions wherever possible, following the principle that algorithmic comparisons should control the available evaluation budget and experimental environment (Wolpert and Macready, 1997). The baseline methods represent progressively more sophisticated alternatives. Equal Allocation provides a simple diversification oriented heuristic. Greedy Allocation represents a profit focused decision rule. Random Search provides a non-gradient stochastic search baseline, while GA represents an alternative population based metaheuristic.

5.4.2 Controlled Monte Carlo Evaluation

The baseline strategies were evaluated using the same stochastic simulation environment. The simulation generated $M = 1,000$ Monte Carlo paths under the baseline demand regime. Performance was measured using expected net profit, median net profit, Value-at-Risk (VaR), Conditional Value-at-Risk (CVaR), service level, fill rate and inventory turnover.

Strategy	Mean Profit	Median	VaR 5%	CVaR 5%	CSL	Fill	Turnover
Equal Allocation	33.328M	33.334M	32.541M	32.322M	63.58%	59.74%	10.09
Greedy Allocation	33.318M	33.341M	32.505M	32.313M	63.38%	59.74%	10.09
Random Search	33.230M	33.222M	32.442M	32.231M	62.71%	59.41%	10.09
Genetic Algorithm	33.230M	33.222M	32.442M	32.231M	62.71%	59.41%	10.09
Optera IPSO	33.214M	33.229M	32.374M	32.081M	62.63%	59.23%	10.10

Table 5.7: Controlled Monte Carlo Performance Under the Baseline Demand Regime

An important finding is immediately visible from Table 5.7: Optera IPSO does not maximise expected net profit under the baseline demand regime. Equal Allocation obtains the highest mean profit, while Optera produces the lowest mean profit among the compared strategies. This result should not be interpreted as a failure of the proposed approach. The objective of Optera is not pure profit maximisation. Its formulation explicitly combines expected profit with covariance risk and concentration considerations, following the portfolio inspired formulation established in the methodology (Markowitz, 1952). The optimisation objective therefore represents a trade off rather than a single performance criterion. The baseline experiment demonstrates an important property of the proposed framework, an allocation that is optimal under the composite objective need not produce the highest raw profit when evaluated using a downstream simulation. This distinction is central to the research because the proposed contribution concerns risk aware allocation rather than a simple maximum margin heuristic. However, the numerical differences in the baseline simulation are relatively small. Consequently, the experiment does not provide evidence that Optera IPSO dominates all competing methods under the normal demand regime. Instead, it motivates the additional stress, ablation and sensitivity experiments presented in the following sections.

5.4.3 Allocation Characteristics

The allocation produced by Optera IPSO provides further evidence of the intended behaviour of the proposed objective. Under the primary five category experiment, the solution is approximately,

Category	Allocation Weight
Beverages	7.8%
Fruits	40.0%
Household	7.2%
Personal Care	40.0%
Stationery	5.0%
Total	100.0%

Table 5.8: Optera IPSO Optimised Allocation

The allocation demonstrates that the bounded optimisation space is active. Two categories receive the maximum permitted allocation of 40%, while the remaining categories receive allocations above or equal to their lower bound. The resulting HHI is approximately 0.3338 and the effective number of categories is approximately 3.0. This behaviour is consistent with the methodological motivation for incorporating a concentration penalty. HHI was introduced because covariance alone does not necessarily prevent structurally concentrated allocations (Stoilov et al., 2022). The resulting allocation therefore represents a compromise between exploiting higher expected margins and avoiding complete concentration.

5.5 Component Ablation Study

5.5.1 Purpose of Ablation Testing

Ablation testing was conducted to investigate whether the mathematical components incorporated into the Optera objective function materially affect the resulting optimisation behaviour. This is important because the proposed objective is composite rather than a conventional single objective profit function. The implemented objective incorporates expected profit, covariance risk, diversification and inventory related considerations. The methodology describes the evolution from a profit oriented objective toward a portfolio inspired formulation incorporating covariance and HHI terms (Markowitz, 1952; Stoilov et al., 2022). Ablation therefore provides a direct experimental mechanism for testing whether removing individual components changes the resulting allocation or objective value.

5.5.2 Ablation Results

Configuration	Profit	Variance	Risk Penalty	HHI	ENC	Fitness
Full Optera IPSO	43.69	39.49	0.0039	0.3338	3.00	42.34
w/o Risk ($\lambda = 0$)	43.69	39.49	0.0000	0.3338	3.00	42.35
w/o Diversification ($\alpha = 0$)	43.69	39.51	0.0040	0.3350	2.99	43.69
w/o Shortage ($\Psi = 0$)	43.69	39.49	0.0039	0.3338	3.00	42.34
Profit Only	43.69	39.51	0.0000	0.3350	2.99	43.69
Unconstrained Simplex	45.46	78.33	0.0078	0.5203	1.92	43.36

Table 5.9: Component Ablation Results for the Optera IPSO Objective

The ablation results reveal two different behaviours. First, removing several components produces only small changes under the primary experimental conditions. For example, removing the covariance risk penalty changes the composite fitness from 42.34 to 42.35, while the expected profit, variance, HHI and effective number of categories remain effectively unchanged.

Similarly, removing the shortage component produces almost identical optimisation statistics. This suggests that, under the particular dataset and capital conditions used in this experiment, the shortage component is not strongly binding during the optimisation stage. This is an important finding rather than a result that should be hidden. It indicates that the contribution of an objective function component depends on whether the underlying operating conditions activate the corresponding constraint or penalty. If the simulation environment provides sufficient inventory availability for a particular category, the shortage penalty may remain numerically small. The strongest result is obtained from the unconstrained simplex experiment. Removing the allocation bounds increases expected profit from 43.69 to 45.46, but portfolio variance increases from 39.49 to 78.33. HHI also increases from approximately 0.334

to 0.520, while the effective number of categories decreases from approximately 3.0 to 1.92. This experiment provides direct evidence for the usefulness of bounded allocation constraints. The unconstrained optimiser can obtain a higher immediate profit by placing substantially more capital into a small number of categories. However, this behaviour simultaneously produces considerably higher portfolio variance and concentration. The result supports the underlying portfolio allocation argument developed in Chapter 2. In the financial interpretation of Markowitz optimisation, an allocation cannot be evaluated solely according to expected return because cross asset covariance influences aggregate risk (Markowitz, 1952). The Optera results demonstrate an analogous behaviour in the inventory setting, relaxing allocation boundaries increases the opportunity for profit concentration but also increases exposure to demand risk.

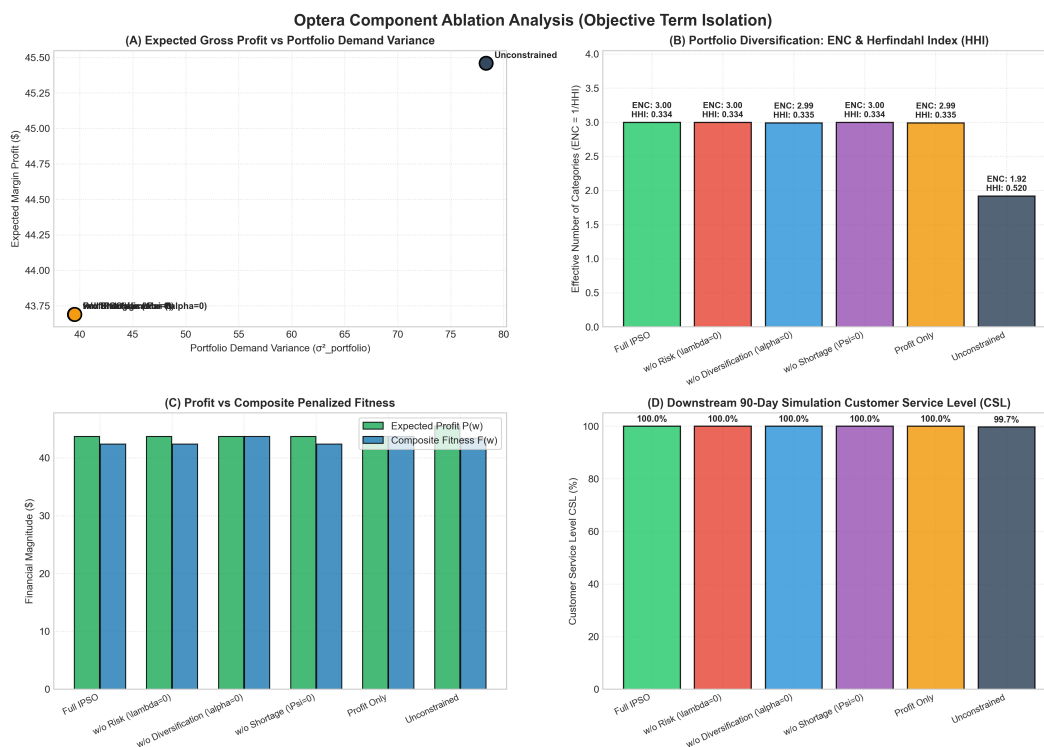


Figure 5.28: Comparison of Optera Objective Function Ablation Variants

Figure 5.28 provides a visual comparison of the objective function variants. The principal distinction is between the bounded formulation and the unconstrained formulation, where the latter permits much greater concentration.

5.6 Hyperparameter Sensitivity Analysis

5.6.1 Purpose of Sensitivity Analysis

Sensitivity analysis was conducted to determine how the behaviour of Optera changes when important optimisation parameters are varied. This is necessary because the risk and diversification coefficients are not universal constants. They represent modelling choices that

control the relative importance of different objectives. The methodology explicitly treats λ and γ -related parameters as tunable quantities associated with decision maker risk preferences. Sensitivity analysis therefore determines whether the observed optimisation behaviour is stable across reasonable parameter values.

Four sensitivity experiments were conducted:

1. covariance risk coefficient λ
2. diversification coefficient α
3. swarm population size
4. optimisation iteration count.

5.6.2 Risk Penalty Sensitivity

λ	Profit	Variance	Risk Penalty	Fitness
0.000000	43.69	39.49	0.0000	42.35
0.000001	43.69	39.49	0.0000	42.35
0.000010	43.69	39.49	0.0004	42.35
0.000050	43.69	39.89	0.0020	42.34
0.000100	43.69	39.50	0.0039	42.34
0.000500	43.69	39.49	0.0197	42.33
0.001000	43.69	39.49	0.0395	42.31
0.005000	43.69	39.48	0.1974	42.15
0.010000	43.69	39.48	0.3948	41.95

Table 5.10: Sensitivity to the Covariance Risk Coefficient λ

The risk penalty sweep demonstrates that increasing λ primarily reduces the composite fitness because the covariance term is increasingly penalised. However, the resulting profit and portfolio variance remain comparatively stable across the tested range. This suggests that the current empirical dataset does not create a sufficiently large trade-off between expected profit and covariance exposure for the risk penalty to substantially change the allocation. The finding is useful because it prevents overinterpretation of the proposed risk term. The mathematical mechanism is implemented correctly, but its practical influence depends on the statistical characteristics of the demand data.

5.6.3 Diversification Sensitivity

α	Profit	HHI	ENC	Fitness
0.00	43.69	0.3350	2.99	43.69
0.02	43.69	0.3350	2.99	43.42
0.05	43.69	0.3338	3.00	43.02
0.08	43.69	0.3350	2.99	42.61
0.10	43.69	0.3350	2.99	42.34
0.15	43.69	0.3338	3.00	41.67
0.20	43.69	0.3338	3.00	41.00
0.25	43.57	0.3208	3.12	40.34
0.30	43.38	0.3033	3.30	39.71

Table 5.11: Sensitivity to the Diversification Coefficient α

The diversification sweep provides stronger evidence of behavioural change. At low and moderate values of α , the allocation characteristics remain relatively stable. At $\alpha = 0.25$ and $\alpha = 0.30$, however, HHI decreases and the effective number of categories increases. This demonstrates the expected effect of the HHI penalty, increasing its relative importance encourages a more diversified allocation. The corresponding decline in expected profit illustrates the trade-off inherent in the proposed objective. A more diversified allocation can reduce concentration, but it may require allocating capital away from categories with higher expected margins. The result is consistent with the theoretical motivation for HHI in the literature review, where concentration and covariance are treated as related but distinct dimensions of portfolio risk (Stoilov et al., 2022).

5.6.4 Swarm Size Sensitivity

Swarm Size	Fitness	Runtime (s)	Convergence Iteration
10	42.34	0.1699	3
20	42.34	0.3125	2
30	42.34	0.4423	3
50	42.34	0.7114	2
75	42.34	1.1537	2
100	42.34	1.4939	2

Table 5.12: Sensitivity to Particle Swarm Size

An important computational result is that increasing the swarm size does not improve the final fitness for this experimental problem. All tested swarm sizes converge to the same fitness value of approximately 42.34. The principal effect of increasing swarm size is therefore computational cost. Runtime increases from approximately 0.17 seconds for ten particles to approximately 1.49 seconds for one hundred particles. This suggests that the

optimisation problem used in the current experiment is relatively easy for the implemented search mechanism under the selected conditions. A very large population therefore provides limited additional benefit while increasing computational cost. This finding is relevant to practical configuration because it indicates that excessively large swarms are not necessary for the tested five category problem.

5.6.5 Iteration Sensitivity

Iterations	Fitness	Runtime (s)
10	42.34	0.0685
25	42.34	0.1501
50	42.34	0.3736
75	42.34	0.4619
100	42.34	1.5741
150	42.34	1.8483
200	42.34	1.1075

Table 5.13: Sensitivity to Optimisation Iteration Count

The iteration sweep demonstrates that computational cost increases with the number of optimisation iterations, while the final fitness remains approximately constant at 42.34 across the tested configurations. This indicates rapid convergence for the present experimental problem. Increasing the iteration limit beyond the early convergence point therefore increases runtime without providing measurable improvement in the final objective. The result should not be interpreted as evidence that the algorithm universally requires only a small number of iterations. The convergence behaviour is dependent on the dimensionality, objective landscape and demand characteristics of the problem. Larger real world catalogues and more complex simulation objectives may require substantially greater search effort.

5.7 Inferential Statistical Evaluation

5.7.1 Purpose of Statistical Testing

Because the framework uses stochastic simulation and stochastic optimisation, individual experimental outcomes should not be interpreted as deterministic proof of superiority. Statistical testing was therefore introduced to determine whether observed differences between strategies are systematic across repeated simulation trials. The statistical evaluation used repeated stochastic trials and examined both distributional assumptions and pairwise differences. Normality was assessed using the Shapiro–Wilk test, while paired comparisons were evaluated using parametric and non parametric methods where appropriate. This approach is consistent with the broader research methodology, which defines the study as quantitative computational experimentation rather than a qualitative investigation. Repeated Monte Carlo

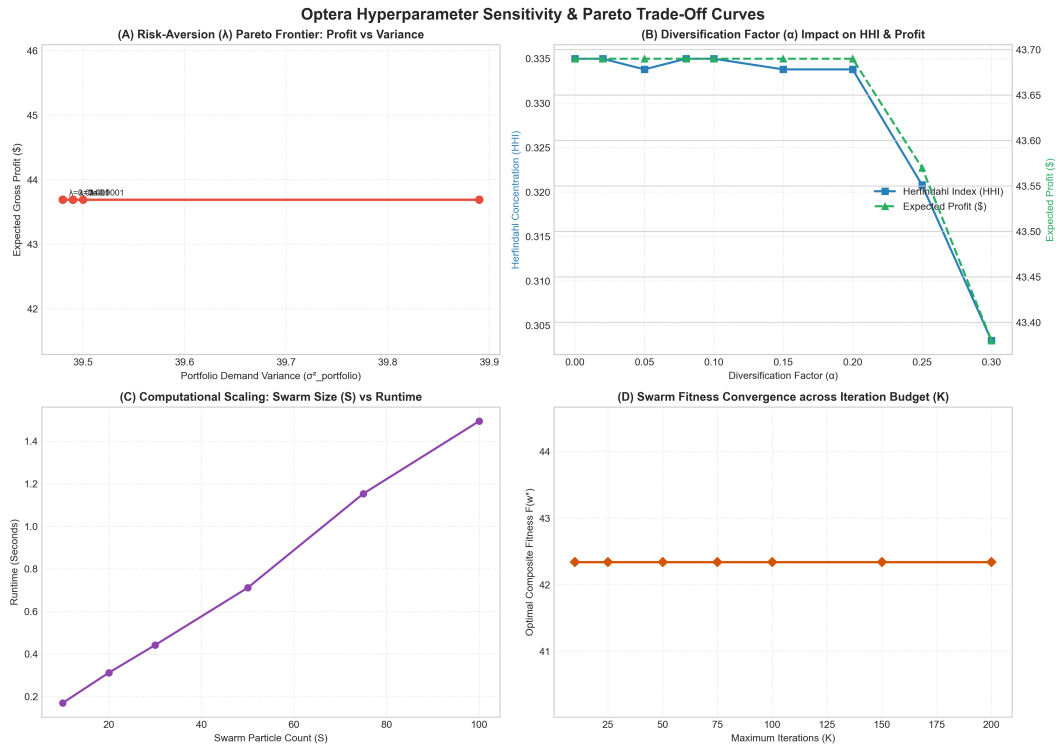


Figure 5.29: Hyperparameter Sensitivity Results for the Optera IPSO Framework

evaluation provides the experimental observations from which statistical comparisons can be performed.

5.7.2 Normality Diagnostics

Strategy	Mean	Std. Dev.	Median	Shapiro–Wilk p	Normal?
Equal Allocation	65,005.84	2,844.34	64,194.68	0.086	Yes
Greedy Allocation	64,735.67	2,808.76	64,011.29	0.124	Yes
Random Search	64,860.94	2,800.77	64,152.12	0.083	Yes
Genetic Algorithm	64,848.08	2,831.71	64,097.81	0.072	Yes
Optera IPSO	64,741.44	2,801.70	64,016.49	0.088	Yes

Table 5.14: Normality Diagnostics Across Optimisation Strategies

The Shapiro–Wilk results do not reject the normality assumption at the conventional $\alpha = 0.05$ level for any of the evaluated strategies. This provides support for the use of parametric statistical comparisons in the subsequent analysis, while non parametric tests were also retained as a robustness check.

5.7.3 Pairwise Statistical Comparison

Baseline	Mean Difference	95% CI	Paired t p	Wilcoxon p	Cohen's d	Result
Equal Allocation	-264.39	[-310.03,-218.76]	< 0.0001	< 0.0001	-0.09	Significant
Greedy Allocation	5.77	[-27.61,39.16]	0.730	0.0618	0.00	Not Significant
Random Search	-119.49	[-164.40,-74.59]	< 0.0001	< 0.0001	-0.04	Significant
Genetic Algorithm	-106.63	[-133.39,-79.88]	< 0.0001	< 0.0001	-0.04	Significant

Table 5.15: Pairwise Statistical Comparison Against Optera IPSO

The statistical results require careful interpretation. The comparisons against Equal Allocation, Random Search and Genetic Algorithm produce statistically significant differences according to both the paired t -test and Wilcoxon signed rank test. However, the effect sizes are small, with Cohen's d values close to zero. This distinction is important. Statistical significance does not necessarily imply practical significance. With repeated observations, even relatively small differences can become statistically detectable. Therefore, the results do not justify describing Optera IPSO as universally superior to all alternative strategies. The comparison with Greedy Allocation is particularly informative. Neither the paired t -test nor the Wilcoxon test identifies a statistically significant difference. This indicates that, under the tested conditions, Optera IPSO and Greedy Allocation produce statistically similar performance. This finding provides a more balanced interpretation of the proposed method. Optera demonstrates a statistically distinguishable behaviour from several baseline strategies, but not from every baseline. The absence of significant difference from Greedy Allocation indicates that the additional risk and diversification modelling does not necessarily translate into superior raw simulation profit under the baseline regime.

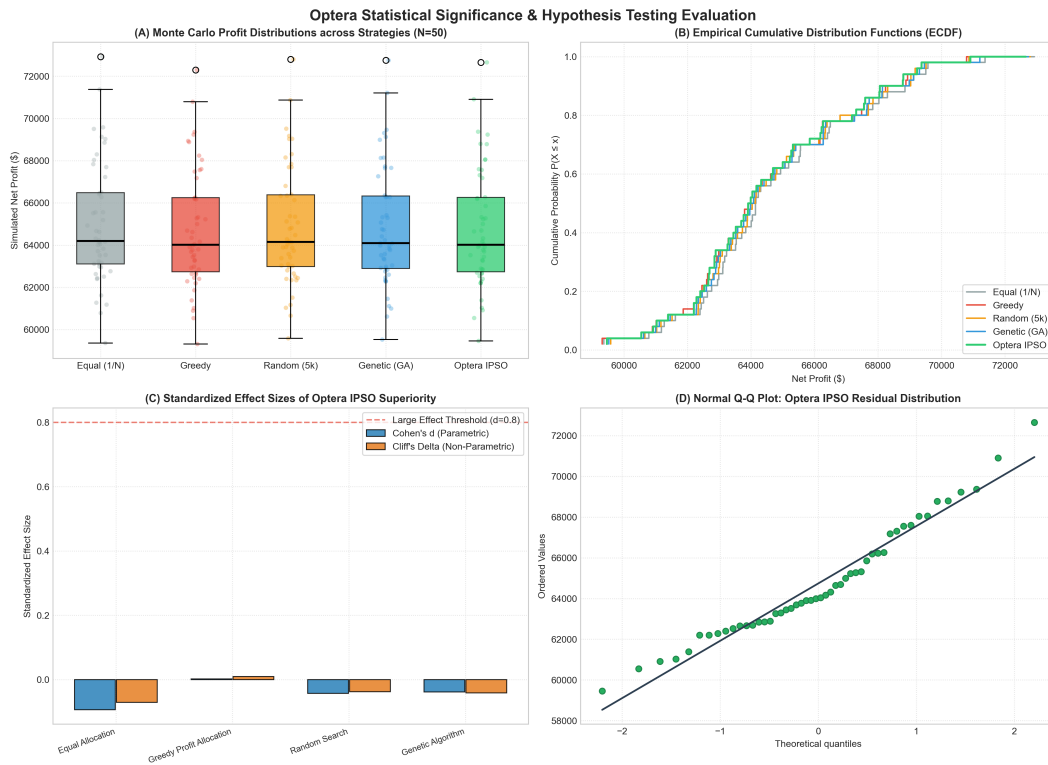


Figure 5.30: Distribution of Stochastic Performance Across Compared Strategies

5.8 Demand Shock Stress Testing

5.8.1 Purpose of Stress Testing

The baseline experiment evaluates performance under a fitted demand regime. However, a central motivation of Optera is the presence of demand uncertainty. An optimisation strategy that performs well only when historical demand assumptions remain unchanged would provide limited practical value. Stress testing was therefore conducted by modifying the demand environment according to predefined scenarios. The five scenarios were,

1. baseline fitted demand
2. high-volatility demand with a +50% increase in standard deviation
3. asymmetric demand surge affecting selected categories
4. macroeconomic recession with a -25% demand reduction
5. combined category disruption and volatility shock.

These scenarios are intended as controlled computational stress conditions rather than forecasts of specific real world events. Their purpose is to examine how allocation strategies behave when the statistical characteristics of demand deviate from the baseline regime.

5.8.2 Stress Test Results

To evaluate the robustness of the allocation strategies under changing demand conditions, five demand shock scenarios were evaluated using the same Monte Carlo simulation framework. The scenarios represent normal demand, increased volatility, asymmetric category growth, recessionary demand contraction, and a combined category disruption and demand surge.

Scenario	Name	Stress Condition
1	Baseline Normal	Original fitted demand distributions
2	High Volatility	+50% demand standard deviation across categories
3	Asymmetric Demand Surge	+50% demand increase in Furniture and Clothing
4	Macroeconomic Recession	−25% demand across all categories
5	Category Disruption and Surge	+75% demand volatility with +40% demand surge in Furniture and Clothing

Table 5.16: Demand Shock Scenarios Used in the Evaluation

The complete cross algorithm stress test matrix compares expected net profit, downside risk, customer service level (CSL), and lost revenue under each demand condition. The results are presented in Table 5.17.

Scenario	Strategy	Net Profit	VaR 5%	CVaR 5%	CSL	Lost Revenue
1	Equal Allocation	33.328M	32.541M	32.322M	63.58%	18.317M
1	Greedy Allocation	33.318M	32.505M	32.313M	63.38%	18.326M
1	Random Search	33.230M	32.442M	32.231M	62.71%	18.415M
1	Genetic Algorithm	33.230M	32.442M	32.231M	62.71%	18.415M
1	Optera IPSO	33.214M	32.374M	32.081M	62.63%	18.430M
2	Equal Allocation	32.533M	31.315M	31.042M	62.21%	19.152M
2	Greedy Allocation	32.514M	31.362M	31.059M	61.90%	19.171M
2	Random Search	32.499M	31.384M	31.088M	61.52%	19.185M
2	Genetic Algorithm	32.499M	31.384M	31.088M	61.52%	19.185M
2	Optera IPSO	32.447M	31.262M	30.989M	61.31%	19.237M
3	Equal Allocation	38.345M	37.429M	37.170M	57.55%	24.510M
3	Greedy Allocation	38.348M	37.418M	37.157M	57.27%	24.506M
3	Random Search	38.345M	37.418M	37.156M	56.88%	24.509M
3	Genetic Algorithm	38.345M	37.418M	37.156M	56.88%	24.509M
3	Optera IPSO	38.348M	37.430M	37.158M	56.85%	24.507M
4	Equal Allocation	27.136M	26.342M	26.148M	69.43%	11.605M
4	Greedy Allocation	27.128M	26.312M	26.095M	68.94%	11.613M
4	Random Search	27.133M	26.311M	26.111M	68.51%	11.608M
4	Genetic Algorithm	27.133M	26.311M	26.111M	68.51%	11.608M
4	Optera IPSO	27.114M	26.294M	26.069M	68.29%	11.626M
5	Equal Allocation	36.060M	34.618M	34.197M	56.76%	24.668M
5	Greedy Allocation	36.064M	34.617M	34.192M	56.44%	24.665M
5	Random Search	36.065M	34.617M	34.191M	56.08%	24.664M
5	Genetic Algorithm	36.065M	34.617M	34.191M	56.08%	24.664M
5	Optera IPSO	36.064M	34.617M	34.191M	55.99%	24.664M

Table 5.17: Complete Cross-Algorithm Performance Under Five Demand Shock Scenarios

The stress test results demonstrate that increasing demand uncertainty reduces both profitability and service performance across all strategies. The effect is particularly visible in Scenario 5, where the combined volatility increase and category specific demand surge reduce CSL to approximately 56% and increase lost revenue to approximately \$24.7M. An important finding is that the proposed Optera IPSO does not consistently produce the highest raw net profit under every stress condition. For example, Equal Allocation achieves the highest net profit in Scenario 1, while Greedy Allocation performs marginally better in Scenario 3. However, the purpose of the proposed optimisation framework is not simply to maximise short term profit. It incorporates portfolio risk, diversification and inventory related considerations into the allocation decision. The results therefore demonstrate an important distinction between *profit maximisation* and *risk aware allocation*. Under severe demand shocks, no allocation strategy can completely eliminate service level degradation when the procurement budget is fixed. Optimisation determines how the scarce capital is distributed, but it cannot create additional inventory capacity. This provides empirical support for evaluating allocation decisions through stochastic simulation rather than relying exclusively on a static profit objective. Such simulation–optimisation approaches are appropriate when operational uncertainty and complex system interactions cannot be adequately represented through a single closed form formulation (Amaran et al., 2016; Fu, 2015).

5.9 Computational Performance

Computational performance was evaluated primarily through the hyperparameter sensitivity experiments. The results show that increasing the swarm population increases execution time while producing no improvement in the final fitness for the tested five category problem. The measured runtime increased from 0.1699 seconds for 10 particles to 1.4939 seconds for 100 particles, while the observed fitness remained constant at 42.34. Therefore, increasing the swarm size beyond the level required to reach the observed solution primarily increases computational cost rather than solution quality. The complete hyperparameter sensitivity results are presented in Section 5.6. The result demonstrates that Optera can execute the tested optimisation workload within a short runtime while retaining a Python oriented interface and compiled computational components. However, these measurements represent the tested problem size and hardware environment and should not be interpreted as universal execution time guarantees. Larger numbers of categories, simulation paths and optimisation iterations may produce substantially different computational requirements.

5.10 Qualitative and Practical Evaluation

The quantitative experiments were complemented by a qualitative assessment of the Optera research artefact. Since Optera is intended as a reusable Python analytical library rather

than a consumer facing inventory application, the evaluation focused on the clarity of the workflow, modularity of the interface, interpretability of generated outputs and practical reuse. The implemented framework separates ETL, analytics, optimisation, simulation and evaluation functionality into modular components. This allows users to configure an experiment, execute the complete analytical workflow and inspect the resulting allocation, risk and simulation outputs without modifying the underlying computational architecture. This modular structure supports the intended role of Optera as a reusable research and experimentation framework.

Figure 5.31 presents an example of the generated Optera analytical artefacts. Additional screenshots of the implemented library, generated outputs and supporting artefacts are provided in the Appendix.

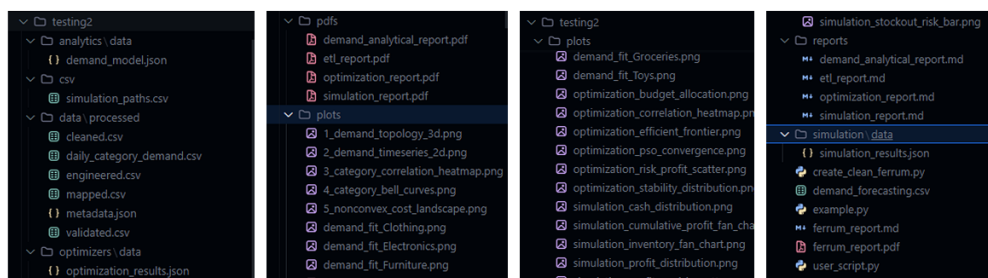


Figure 5.31: Example Optera Analytical Outputs

The qualitative evaluation therefore complements the numerical experiments, the computational experiments assess whether the optimisation and simulation processes produce measurable and repeatable results, while the practical evaluation considers whether the resulting software can reasonably function as a reusable analytical research tool.

5.10.1 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with three external participants with backgrounds in data science and data analytics. The objective was to assess whether the Optera library could be installed, understood, and used successfully across its main analytical workflow, including ETL, demand modelling, optimisation, Monte Carlo simulation, efficient frontier analysis, simulation outputs, and automated reporting. Participants evaluated the system using a five point Likert scale, together with a separate usability checklist.

Participant	Organisation	Role	Modules Evaluated
R. G. Gihan Prasanna Rajapaksha	Builtapps Business Solutions	Data Science	etl(), analytics(), optimize(), simulation(), run()
D. M. D. T. B. Dissanayaka	Builtapps Business Solutions	Data Science	etl(), analytics(), optimize(), simulation(), run()
Nevinka Wickramasinghe	Deloitte	Data Analytics	etl(), analytics(), optimize(), simulation()

Table 5.18: Participants and Optera Modules Evaluated During UAT

The aggregated UAT results are presented in Table 5.19. Demand modelling and PSO optimisation received the highest evaluation, with a cohort mean of 5.00/5.00. ETL, Monte Carlo simulation, and automated reporting each achieved 4.67/5.00, while efficient frontier analysis and simulation outputs achieved 4.33/5.00. All three participants also passed all six usability checklist items, resulting in an overall usability pass rate of 100%.

Evaluation Dimension	Gihan	Damith	Nevinka	Cohort Mean
ETL Pipeline	4/5	5/5	5/5	4.67/5
Demand Modelling	5/5	5/5	5/5	5.00/5
PSO Optimisation	5/5	5/5	5/5	5.00/5
Monte Carlo Simulation	5/5	4/5	5/5	4.67/5
Efficient Frontier	4/5	5/5	4/5	4.33/5
Simulation Output	4/5	5/5	4/5	4.33/5
Automated Reporting	4/5	5/5	5/5	4.67/5
Overall Composite Score	4.43/5	4.86/5	4.71/5	4.67/5
Library Usability	6/6	6/6	6/6	18/18 (100%)

Table 5.19: Quantitative User Acceptance Testing Results

The UAT results indicate that the core Optera workflow was successfully accepted by all three participants. In particular, PSO optimisation and demand modelling achieved full scores from every evaluator. The overall cohort score of 4.67/5.00 corresponds to a 93.4% satisfaction index, while all 18 usability checklist responses were passed. All participants therefore accepted the system with only minor improvements recommended. The qualitative feedback identified several practical areas for future refinement. R. G. Gihan Prasanna Rajapaksha recommended clearer documentation and examples for users working with their own datasets and parameters. D. M. D. T. B. Dissanayaka suggested presenting optimisation results in a simpler, more business oriented manner. Nevinka Wickramasinghe similarly identified the need for clearer documentation, improved error messages for missing parameters, and a dashboard for summarising procurement recommendations. A dashboard was therefore included in the implemented system to provide a consolidated view of the generated procurement and simulation results. Figure 5.32 shows the dashboard artefact produced by the library, providing visual evidence that the analytical outputs can be presented in a form that is easier to interpret beyond the underlying numerical results.



Figure 5.32: Optera Dashboard for Summarising UAT Results

The completed UAT feedback forms, participant evaluation sheets, signed test documentation, and supporting evidence are provided in Appendix. The three participants independently evaluated the implemented library and all returned an accepted verdict with minor improvements recommended. Therefore, the UAT provides supporting evidence that Optera is functionally usable by technically experienced users while also identifying specific areas for improving documentation, interpretability, error handling, and result presentation.

5.11 Evaluation Against the Research Questions

The combined experimental results provide evidence addressing all four research questions defined in Chapter 1.

5.11.1 RQ1: Applicability of Portfolio Optimisation

RQ1: Can portfolio optimization concepts involving expected return, risk, and diversification be effectively adapted to multi category inventory budget allocation under demand uncertainty?

The results provide evidence that portfolio concepts can be operationalised within an inventory allocation framework. Optera combines expected profit, covariance risk and concentration within a bounded allocation space. The ablation experiments demonstrate that removing allocation constraints can increase immediate profit while substantially increasing variance and concentration, demonstrating the trade-off between return and risk. Therefore, the results support the applicability of portfolio inspired concepts to multi category inventory budget allocation, although the evidence is limited to the experimental environments evaluated in this research.

5.11.2 RQ2: Comparison With Baseline Strategies

RQ2: How does the proposed Optera Markowitz–Herfindahl Particle Swarm Optimization approach compare with baseline allocation and optimisation strategies in terms of allocation quality, convergence and stability?

The baseline experiments demonstrate that Optera IPSO produces valid bounded allocations and stable convergence. However, the results do not demonstrate universal superiority in expected profit. Equal Allocation achieves the highest baseline mean profit in the main experiment, while Greedy Allocation was statistically indistinguishable from Optera in the supplied statistical comparison. The principal distinction is therefore methodological, Optera optimises a multi dimensional objective involving profit, risk and diversification rather than maximising expected profit alone. The convergence and sensitivity experiments further demonstrate stable behaviour under the tested configurations.

5.11.3 RQ3: Behaviour Under Stochastic Demand

RQ3: How does the proposed framework perform when optimised allocation strategies are evaluated under stochastic demand and alternative demand shock scenarios?

The Monte Carlo and five scenario stress test results demonstrate that all allocation strategies are affected by demand shocks. High volatility, asymmetric demand surges and severe category disruption reduce customer service levels and increase lost revenue. The severe disruption scenario produces service levels close to 56% across the evaluated strategies. The results demonstrate the value of stochastic simulation for evaluating allocation policies because expected profit alone does not reveal the operational consequences of demand uncertainty. They also identify a fundamental limitation, optimisation can redistribute available capital but cannot eliminate shortages when demand substantially exceeds available inventory funding.

5.11.4 RQ4: Computational Efficiency and Reusability

RQ4: To what extent does the hybrid Python–C++ implementation improve the computational efficiency of repeated simulation workloads while maintaining accessibility through a Python interface?

The computational experiments demonstrate that Optera can execute repeated optimisation and simulation workloads within short execution times for the tested five category problem. The sensitivity experiments show runtimes from sub second execution to approximately one and a half seconds as swarm size increases. The Python interface also preserves accessibility while performance critical components can be executed using compiled code. The results therefore support the feasibility of the hybrid architecture. However, the experiments do not establish a universal Python–C++ speedup for arbitrary problem sizes. Such a conclusion would require controlled benchmarks across different simulation sizes, problem dimensionalities and hardware environments.

5.12 Limitations of the Evaluation

Several limitations should be considered when interpreting the findings.

5.12.1 Dataset Size and Diversity

The principal experiments use a limited number of product categories and datasets. Although the framework is designed for reuse, the empirical results cannot establish universal performance across all retail categories or supply chain structures. Larger datasets with more categories, longer historical periods and different market conditions would provide stronger evidence of generalisability.

5.12.2 Simulation Assumptions

The Monte Carlo results depend on demand distributions and parameters estimated from the available historical data. Consequently, the accuracy of the simulated outcomes is dependent on the suitability of these statistical assumptions. Factors such as temporal dependence, seasonality, supplier lead time uncertainty and correlated demand shocks are not fully represented in the current experimental scope.

5.12.3 Limited Real World Validation

The evaluation is primarily based on controlled computational experiments. The UAT provides useful external feedback from three technically competent users, but the sample is small and does not represent long term industrial deployment. The UAT should therefore be interpreted as supplementary evidence of practical usability rather than proof of operational effectiveness in a live supply chain.

5.12.4 Baseline Scope

The comparison includes Equal Allocation, Greedy Allocation, Random Search and Genetic Algorithm. These provide meaningful comparison points but do not represent every modern optimisation method. Consequently, the research does not claim state-of-the-art superiority across all metaheuristic algorithms. The contribution is instead the adaptation and integration of portfolio inspired risk and diversification concepts within an inventory simulation–optimisation framework.

5.12.5 Statistical and Computational Interpretation

Statistically significant differences should not automatically be interpreted as practically important differences, particularly where effect sizes are small. Similarly, measured execution times depend on the experimental implementation and hardware environment. The conclusions therefore consider statistical evidence together with effect sizes, allocation characteristics, simulation outcomes, stress behaviour and computational performance rather than relying on a single metric.

5.13 Chapter Summary

This chapter evaluated the Optera simulation–optimisation framework from both software engineering and empirical perspectives. The computational results showed stable optimisation behaviour and manageable execution times for the tested problem size, while the detailed hyperparameter sensitivity analysis demonstrated that increasing swarm size primarily increases computational cost without improving the observed fitness. The empirical experiments compared Optera IPSO with Equal Allocation, Greedy Allocation, Random Search and Genetic Algorithm strategies. The results show that Optera does not universally maximise raw expected profit, but instead optimises a composite objective incorporating profit, risk, diversification and inventory considerations. The five scenario stress testing further demonstrated that severe demand shocks negatively affect all strategies and that optimisation cannot overcome fundamental capital scarcity. The qualitative evaluation and UAT provided additional evidence that the implemented library is usable as a research and analytical tool. Three external participants produced a cohort mean score of 4.67/5.00 and all 18 usability checks were passed. The feedback nevertheless identified documentation, business oriented result interpretation, error messaging and dashboard presentation as areas for future improvement. Overall, the findings provide evidence addressing all four research questions. At the same time, the limitations concerning dataset diversity, simulation assumptions, limited real world validation, baseline scope and computational generalisation require the results to be interpreted within the defined experimental environment. The broader implications, contributions and future improvements are discussed in Chapter 6.

CONCLUSION AND FUTURE WORK

6.1 Chapter Overview

This chapter concludes the research by summarising the main findings from the development and evaluation of the Optera simulation–optimization framework. The findings are considered against the research questions, followed by a summary of the research contributions, limitations and potential directions for future work.

6.2 Research Findings

The research investigated multi category inventory budget allocation under demand uncertainty by combining portfolio inspired risk and diversification concepts, Particle Swarm Optimization, Monte Carlo simulation and a hybrid Python–C++ architecture. The results presented in Chapter 5 provide evidence that these components can be combined into a reusable framework for studying inventory allocation decisions under stochastic demand.

6.2.1 RQ1: Portfolio Inspired Inventory Allocation

RQ1: Can portfolio optimization concepts involving expected return, risk, and diversification be effectively adapted to multi category inventory budget allocation under demand uncertainty?

The findings support the feasibility of adapting selected portfolio concepts to inventory budget allocation. Optera considers expected margin profit, demand related risk and allocation concentration when evaluating candidate allocations. The ablation experiments further showed that removing risk or diversification considerations can increase immediate profit while producing less balanced allocations and greater exposure to portfolio level variance. These findings suggest that portfolio inspired modelling provides a useful perspective for inventory allocation when procurement capital is limited. However, the approach should be viewed as an adaptation of selected financial concepts rather than a direct equivalence between financial portfolios and physical inventory systems.

6.2.2 RQ2: Comparison with Alternative Strategies

RQ2: How does the proposed Optera Markowitz–Herfindahl Particle Swarm Optimization approach compare with baseline allocation and optimization strategies in terms of allocation quality, convergence, and stability?

Optera IPSO produced feasible allocations and stable optimization behaviour under the tested configuration. The approach was compared with Equal Allocation, Greedy Allocation,

Random Search and Genetic Algorithm strategies. The results showed that Optera did not consistently produce the highest raw expected profit. This is expected because the proposed objective considers risk and concentration in addition to profit. Several strategies produced closely related results, and the statistical analysis indicated that Greedy Allocation was not statistically distinguishable from Optera under the tested conditions. Therefore, the findings do not support a claim that Optera is universally superior to alternative strategies. Instead, they demonstrate that Optera provides a structured risk aware alternative that remains competitive with the selected baseline approaches.

6.2.3 RQ3: Stochastic Evaluation and Demand Shocks

RQ3: How does the proposed framework perform when optimized allocation strategies are evaluated under stochastic demand and alternative demand shock scenarios?

The stochastic simulation and stress testing experiments demonstrated that inventory allocation performance changes considerably under different demand conditions. Baseline demand, increased volatility, asymmetric demand surges, recession conditions and combined category disruption were evaluated. The results showed corresponding changes in service level, lost revenue and downside risk. Under the severe combined disruption scenario, service levels declined to approximately the mid-50% range across the evaluated strategies. The experiments also demonstrated that optimization cannot remove the effects of severe demand shocks when procurement capital remains fixed. The role of optimization is therefore to determine how scarce capital is distributed rather than to eliminate the underlying operational effects of demand uncertainty.

6.2.4 RQ4: Computational Architecture and Reusability

RQ4: To what extent does the hybrid Python–C++ implementation improve the computational efficiency of repeated simulation workloads while maintaining accessibility through a Python interface?

The results support the feasibility of the hybrid architecture for the experimental problem size. Computationally intensive simulation operations were implemented using compiled C++, while the wider analytical workflow remained accessible through Python. The sensitivity experiments further showed that increasing optimization parameters increased computational cost, while the tested configuration remained practical for repeated experimentation. However, the experiments used a relatively small five category problem and a specific computing environment. The results therefore demonstrate the feasibility of the architecture rather than establishing a universal performance advantage over pure Python implementations.

6.3 Research Contributions

The research provides three main contributions. First, it demonstrates a portfolio inspired formulation of multi category inventory budget allocation in which expected profit, demand related risk and allocation concentration are considered together. Second, it provides a reusable simulation–optimization framework that combines optimization with stochastic evaluation. Candidate allocations can therefore be assessed using not only expected profit but also service level, lost revenue and downside risk measures under changing demand conditions. Third, the research demonstrates a practical hybrid Python–C++ architecture for repeated simulation workloads. The separation between the Python analytical interface and compiled simulation component provides a foundation for further computational experimentation without removing Python level accessibility.

6.4 Limitations

The findings should be interpreted within the scope of the experimental design. The empirical evaluation used a limited number of datasets and product categories, meaning that the results cannot automatically be generalised to larger or substantially different supply chain environments. The simulation outcomes also depend on the demand distributions and assumptions used to represent future demand. More complex temporal dependencies, seasonality and lead time uncertainty were outside the main scope of the current evaluation. Real world validation was also limited. User acceptance testing involved three participants and provided positive evidence regarding functionality and usability, but the sample does not represent long term industrial deployment. Similarly, the algorithm comparison covered the selected baseline strategies rather than the full range of available optimization methods. Finally, computational performance was evaluated for the implemented problem size and hardware environment.

6.5 Future Work

6.5.1 Larger Scale Evaluation

Future studies could evaluate Optera using larger datasets, more product categories and longer historical periods. This would provide a stronger assessment of the scalability of the optimization model and computational architecture.

6.5.2 Enhanced Demand and Lead Time Modelling

Future versions could incorporate seasonality, temporal demand dependence, category level correlations and stochastic supplier lead times. These extensions would allow the simulation to

represent a wider range of real world inventory conditions.

6.5.3 Broader Optimization Benchmarking

Additional experiments could compare Optera IPSO with other established optimization approaches under identical constraints and evaluation budgets. This would help identify the problem conditions under which the proposed objective and optimization mechanism provide the greatest benefit.

6.5.4 Multi Objective Decision Support

Future work could investigate a multi objective formulation that treats profit, risk, service level and concentration as separate objectives. This could produce a Pareto frontier of allocation alternatives rather than a single composite solution.

6.5.5 Improved User Interface and Real World Validation

The Optera framework could be extended with a more business oriented dashboard presenting recommended allocations, expected profit, service level, risk and stress test results. This would address feedback obtained during UAT concerning the interpretation and presentation of optimization results. Further validation with procurement or inventory practitioners and real operational datasets would also provide stronger evidence regarding the practical value of the framework.

6.6 Final Conclusion

This research developed and evaluated Optera as a reusable simulation–optimization framework for multi category inventory budget allocation under demand uncertainty. The results support the feasibility of combining portfolio inspired risk and diversification concepts with Particle Swarm Optimization and Monte Carlo simulation. The hybrid Python–C++ architecture further demonstrated a practical approach for repeated simulation workloads while maintaining a Python accessible analytical workflow. The findings do not establish universal superiority of Optera over alternative optimization methods. Instead, they demonstrate that the proposed framework can provide a structured approach for evaluating inventory allocation decisions across profitability, risk, service performance and demand shock conditions. Within the defined experimental scope, the research therefore achieves its aim and provides a foundation for further investigation into risk aware inventory allocation and larger scale simulation–optimization.

REFERENCES

- Amaran, S., Sahinidis, N.V., Sharda, B. and Bury, S.J., 2016. Simulation optimization: a review of algorithms and applications. *Annals of Operations Research*, 240, pp. 351–380. doi:10.1007/s10479-015-2019-x.
- Arakelian, V., Bolesta, K., Vlah Jeric, S., Liu, Y., Osterrieder, J., Potì, V., Schwendner, P., Sutiene, K. and Weinberg, A.I., 2024. *A Discussion Paper for Possible Approaches to Building a Statistically Valid Backtesting Framework*. Available at: <https://doi.org/10.2139/ssrn.4893677>
- Arboleda, F.J.M., Arias, M.R. and Riveros, J.A.H., 2023. *Performance of Parallelism in Python and C++*.
- Asaduzzaman, A., Telikepalli, V.S.P.T. and Uddin, M.R., 2024. Performance Analysis of C and Python Parallel Implementations on a Multicore System Using Particle Simulation. In: *2024 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*. pp. 1–7. Available at: <https://doi.org/10.1109/ACDSA59508.2024.10467885>
- Balasubramaniam, S., 2026. *Stochastic Optimization in Multi-Supplier Procurement: A Scientific Framework for Quality Risk Management in Textile Spinning Operations*. Available at: <https://doi.org/10.5281/zenodo.20338708>
- Bo, L. and Xu, J., 2024. Enhancing Supply Chain Efficiency Resilience Using Predictive Analytics and Computational Intelligence Techniques. *IEEE Access*, 12, pp. 183451–183465. Available at: <https://doi.org/10.1109/ACCESS.2024.3507161>
- Coşkun, Ş., 2025. Simulation-Based Optimization in Production and Inventory Management: The Impact of Uncertainties, Failures and Customer Demand. *AIEM*, 14, pp. 1–10. Available at: <https://doi.org/10.7508/aiem.01.2025.01.10>
- Erisoglu, Ü., Gunduz, S. and Yaman, M., 2026. An Enhanced Monte Carlo Simulation Framework with Backtest-Driven GMM and Deterministic Initialization for Portfolios Risk Estimation. *Mathematics*, 11, pp. 14096–14120. Available at: <https://doi.org/10.3934/math.2026579>
- Fu, M.C., 2015. *Handbook of Simulation Optimization*. New York: Springer.
- Gao, Q., Xu, H. and Li, A., 2022. The Analysis of Commodity Demand Prediction in Supply Chain Network Based on Particle Swarm Optimization Algorithm. *Journal of Computational and Applied Mathematics*, 400, p. 113760. Available at: <https://doi.org/10.1016/j.cam.2021.113760>
- Gkonos, A., 2023. *Supply Chain Management (SCM)*.
- Goyal, M., Osborne, I. and Pivarski, J., 2026. The Awkward World of Python and C++. *Journal of Physics: Conference Series*, 3206, p. 012062. Available at: <https://doi.org/10.1088/1742-6596/3206/1/012062>

- Guan, Y., Huang, Y. and Qin, H., 2022. Inventory Management Optimization of Green Supply Chain Using IPSO-BPNN Algorithm under the Artificial Intelligence. *Wireless Communications and Mobile Computing*, 2022, p. 8428964. Available at: <https://doi.org/10.1155/2022/8428964>
- Guo, Y., Shi, Q. and Guo, C., 2022. Multi-Period Spare Parts Supply Chain Network Optimization under (T, s, S) Inventory Control Policy with Improved Dynamic Particle Swarm Optimization. *Electronics*, 11, p. 3454. Available at: <https://doi.org/10.3390/electronics11213454>
- Henson, H.P. and Joubert, J.F., n.d. *The Evolution of Walk-Forward Backtesting*.
- Huseyn, R., 2023. *Demand Forecasting Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/raminhuseyn/demand-forecasting-dataset> [Accessed 14 January 2026].
- Imad Eddine, K., Karima, B. and Dekhici, L., 2024. Matrix-Probabilistic Particle Swarm Optimization for Supply Chain Optimization. Available at: <https://doi.org/10.1109/AFR0S62115.2024.11036965>
- Islam, M.T. and Uddin, M.S., 2024. Simulation-Based Forecasting and Inventory Control Models for Consumer Goods Networks: A Quantitative Study Using Monte Carlo Simulation and Time-Series Methods. *Review of Applied Science and Technology*, 3, pp. 165–197. Available at: <https://doi.org/10.63125/a3047d06>
- Jakob, W., Rhineland, J. and Moldovan, D., 2017. *pybind11: Seamless operability between C++11 and Python*. GitHub. Available at: <https://github.com/pybind/pybind11> [Accessed 26 August 2026].
- Jalali, H. and Van Nieuwenhuyse, I., 2015. Simulation optimization in inventory replenishment: a classification. *IIE Transactions*, 47(11), pp. 1212–1230. doi:10.1080/0740817X.2015.1019162.
- Jin, C. and Gao, J., 2025. Managing Risk Across Time: An Intertemporal Spectral Risk Measures Framework for Multi-Period Portfolio Optimization. *Mathematics*, 13, p. 1754. Available at: <https://doi.org/10.3390/math13111754>
- Joubert, J., Sestovic, D., Barziy, I., Distaso, W. and López De Prado, M., 2024. Enhanced Backtesting for Practitioners. *Journal of Portfolio Management*, 51, pp. 12–27. Available at: <https://doi.org/10.3905/jpm.2024.1.637>
- Kartikasari, D.P., Tambak, T.A.T. and Ridwanto, A.R., 2025. Hybrid Demand Forecasting and Monte Carlo Simulation for Retail Supply Chain Inventory Optimization. *JITCoS: Journal of Information Technology and Computer System*, 1, pp. 74–85. Available at: <https://doi.org/10.65230/jitcos.v1i2.40>
- Kennedy, J. and Eberhart, R., 1995. Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*. Perth, Australia: IEEE, pp. 1942–1948. doi:10.1109/ICNN.1995.488968.

- Khoirudin and Sulistyowati, E., 2025. Monte Carlo Simulation as a Predictive Tool in Addressing Demand Volatility. *Jurnal Penelitian Pendidikan IPA*, 11, pp. 996–1005. Available at: <https://doi.org/10.29303/jppipa.v11i11.13413>
- Király, A., Varga, T. and Abonyi, J., 2012. *Constrained Particle Swarm Optimization of Supply Chains*.
- Li, J., 2025. Study of Monte Carlo Simulation: Principles, Methods, and Applications. *Highlights in Science, Engineering and Technology*, 140, pp. 42–51. Available at: <https://doi.org/10.54097/sev38v22>.
- Lu, C.-J., Yang, C.-T., Jiang, D.-Y. and Chen, M.-S., 2025. Mathematical Modeling and Optimization of Sustainable Production–Inventory Systems Using Particle Swarm Algorithms. *Mathematics*, 13, p. 3912. Available at: <https://doi.org/10.3390/math13243912>
- Lu, X., n.d. *Simulation Engine for ML-enhanced Pairs Trading Strategies for Decision Making*.
- Maitra, S., 2024. Inventory Management Under Stochastic Demand: A Simulation-Optimization Approach. *arXiv preprint arXiv:2406.19425*. Available at: <https://arxiv.org/abs/2406.19425v1> [Accessed 3 June 2026].
- Maitra, S., Mishra, V. and Kundu, S., 2023. A Novel Approach with Monte-Carlo Simulation and Hybrid Optimization Approach for Inventory Management with Stochastic Demand. Available at: <https://doi.org/10.48550/arXiv.2310.01079>
- Markowitz, H., 1952. Portfolio selection. *The Journal of Finance*, 7(1), pp. 77–91. doi:10.2307/2975974.
- Mazzi, M., Aboueljinane, L. and Lebbar, M., 2026. Monte Carlo Simulation and Genetic Algorithm for Optimizing Hospital Medicine Inventory Management under Uncertainty. *Procedia Computer Science*, 277, pp. 3093–3102. Available at: <https://doi.org/10.1016/j.procs.2026.02.345>
- Metropolis, N. and Ulam, S., 1949. The Monte Carlo Method. *Journal of the American Statistical Association*, 44(247), pp. 335–341. Available at: <https://doi.org/10.2307/2280232>.
- Mirjalili, S. and Lewis, A., 2016. The Whale Optimization Algorithm. *Advances in Engineering Software*, 95, pp. 51–67. doi:10.1016/j.advengsoft.2016.01.008.
- Mirjalili, S., Mirjalili, S.M. and Lewis, A., 2014. Grey Wolf Optimizer. *Advances in Engineering Software*, 69, pp. 46–61. doi:10.1016/j.advengsoft.2013.12.007.
- Olinto, E., Baffa, A. and Pereira, J.A., n.d. *BACK2SERIES: Designing a Backtesting Framework for Time Series Forecasting into a RESTful API*.
- Oyedepo, O., 2025. Stochastic Simulation of Supply Chain Resilience Under Uncertain Demand and Supply Conditions. Available at: <https://doi.org/10.2139/ssrn.5262096>

- Poudel, P. and Paudel, S., 2025. Quantitative Trading Strategy, Backtesting, and Performance Analysis Using Python: A Data-Driven Analysis. *Quest Journal of Management and Social Sciences*, 7, pp. 219–238. Available at: <https://doi.org/10.3126/qjmss.v7i2.87782>
- Pritchard, A., Taylor, D. and Belford, M., n.d. Teaching Data-Driven Decision Making for Inventory Analysis with Monte Carlo Simulation. Available at: <https://doi.org/10.1111/dsji.12328>
- Raghuwanshi, Y., 2026. Monte Carlo Portfolio Risk Analysis: A Framework for Stress Testing and Risk Measurement. Available at: <https://doi.org/10.2139/ssrn.6517238>
- Raheman, A., Kolonin, A., Glushchenko, A., Fokin, A. and Ansari, I., 2023. Adaptive Multi-strategy Market-Making Agent for Volatile Markets. In: *Artificial General Intelligence*. Lecture Notes in Computer Science. Cham: Springer, pp. 250–259. Available at: https://doi.org/10.1007/978-3-031-19907-3_24
- Sadeghi, A.H., Bani, E.A., Fallahi, A. and Handfield, R., 2023. Grey Wolf Optimizer and Whale Optimization Algorithm for Stochastic Inventory Management of Reusable Products in a Two-Level Supply Chain. *IEEE Access*, 11, pp. 40278–40297. Available at: <https://doi.org/10.1109/ACCESS.2023.3269292>
- Saracoglu, I., 2024. Inventory Optimization with Chance-Constrained Programming Under Demand Uncertainty. *International Journal of Supply and Operations Management*. Available at: <https://doi.org/10.22034/ij som.2024.110359.3078>
- Saunders, M.N.K., Lewis, P. and Thornhill, A., 2023. *Research Methods for Business Students*. 9th ed. Harlow, England: Pearson Education.
- Shah, N., Chuang, P.-Y., Sathre, P. and Feng, W., 2025. On the Landscape of Scientific Computing Libraries in Python. In: *2025 IEEE High Performance Extreme Computing Conference (HPEC)*. pp. 1–9. Available at: <https://doi.org/10.1109/HPEC67600.2025.11196385>
- Simoglou, C.K., Biskas, P.N., Marneris, I.G. and Roumkos, C.G., 2022. Long-term Electricity Procurement Portfolio Optimization. *Electric Power Systems Research*, 202, p. 107582. Available at: <https://doi.org/10.1016/j.epsr.2021.107582>
- Stoilov, T., Stoilova, K. and Dimitrov, S., 2022. Planning Resource Allocation for Husbandry Management by Portfolio Optimization. *Heliyon*, 8. Available at: <https://doi.org/10.1016/j.heliyon.2022.e10841>
- Storn, R. and Price, K., 1997. Differential evolution – A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11, pp. 341–359. doi:10.1023/A:1008202821328.
- Tan, N.D., Kim, H.-S., Long, L.N.B., Nguyen, D.A. and You, S.-S., 2024. Optimization and Inventory Management Under Stochastic Demand Using Metaheuristic Algorithm. *PLoS ONE*, 19, e0286433. Available at: <https://doi.org/10.1371/journal.pone.0286433>

- Turgay, S., Demir, R. and Kavacık, M., 2026. A Monte Carlo-Based Approach to Demand Forecasting and Stochastic Optimization in Supply Chains. *Supply Chain Analytics*, 14, p. 100210. Available at: <https://doi.org/10.1016/j.sca.2026.100210>
- Verma, P., Chaturvedi, B.K. and Malik, A.K., 2022. Comprehensive Analysis and Review of Particle Swarm Optimization Techniques and Inventory System. *IJFRCSC*, 8, pp. 111–115. Available at: <https://doi.org/10.17762/ijfrcsce.v8i3.2107>
- Wambles, C., 2019. *Supermarket Sales Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/chadwambles/supermarket-sales> [Accessed 14 January 2026].
- Wolpert, D.H. and Macready, W.G., 1997. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), pp. 67–82. doi:10.1109/4235.585893.
- Xu, N., Xiao, H., Zhu, Y., Chen, X., Li, Y. and Hu, X., 2024. A Novel End-to-End Framework for A-share Stock Market Portfolio Optimization Considering Risk Measure and Feature Exposure. In: *Proceedings of the 2024 7th International Conference on Big Data Technologies (ICBDT '24)*. New York: ACM, pp. 31–39. Available at: <https://doi.org/10.1145/3698300.3698317>
- You, D.-G., Wang, C.-H. and Li, Y.-T., 2026. Data-Driven Simulation–Optimization for Sustainable (s, S) Inventory Policy Design Under Demand and Lead-Time Uncertainty. *Sustainability*, 18, p. 4305. Available at: <https://doi.org/10.3390/su18094305>
- Yousef, A., Ezzat, R. and El-Rashidy, N., 2026. Grey wolf optimizer and whale optimization algorithm: a systematic review. *Artificial Intelligence Review*, 59, article 162. doi:10.1007/s10462-026-11585-8.
- Attractive Chaos, 2011. Programming Language Benchmarks. Available at: <https://attractivechaos.github.io/plb/>
- Mehta, S., 2026. Data Science Statistics 2026: Facts, Growth, Trends & Market Size. MindInventory. Available at: <https://www.mindinventory.com/blog/data-science-statistics/>
- Zhao, Y., 2025. *Assessment, Mitigation, and Backtesting of Extreme Risks in Insurance and Finance*.

APPENDIX

A.1 UAT testing forms

APRIT Sri Lanka University of Staff Training (SLU) - Final Year Research Project
Optera — User Acceptance Testing Form
Simulation-Based Procurement Optimization Python Library

A — PARTICIPANT DETAILS
Full Name: D.M.D.T.B. Dissanayake
Organisation: Bullapps Business Solutions
☐ Data Analytics ☒ Data Science ☐ Supply Chain ☐ Operations ☐ Software Engineering ☐ Other

B — LIBRARY USAGE
Which Optera modules did you use? (check all that apply)
☒ optera.ets() ☒ optera.analytics() ☒ optera.optimize() ☒ optera.simulation() ☒ optera.run()
Did you use a custom column mapping to load your own dataset?
☐ Yes — my own data ☒ Yes — sample/demo data ☐ No — used default schema

C — FUNCTIONAL ACCEPTANCE (1=poor, 5=excellent)
ETL pipeline — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5 Demand modelling — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
PSO optimization — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5 Monte Carlo simulation — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
Efficient frontier — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☒ 5 Simulation output — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
Report generation — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

Overall, did Optera produce useful procurement optimization insights?
☒ Yes, clearly useful ☐ Partially useful ☐ Needs improvement ☐ Not useful for my use case

D — USABILITY AS A PYTHON LIBRARY
☒ The pip install and import process was straightforward ☒ The column mapping configuration was clear and flexible
☒ The function parameters were well named and intuitive ☒ The optimization results (weights, frontier) were interpretable
☒ The simulation output was easy to read and act on ☒ The library ran without errors on my environment

E — SUGGESTIONS FOR IMPROVEMENT
Please describe any issues encountered, features you expected but did not find, or improvements you would suggest for the Optera library.
The optimization results could also be explained in a simpler, more business focused way.

F — OVERALL VERDICT
I confirm that I have evaluated the Optera Python library and that my responses above reflect my genuine professional assessment.
☐ I accept Optera as a valid and functional procurement optimization library
☒ I accept Optera with minor improvements recommended (see Section E)
☐ I do not accept — significant issues identified (see Section E)

G — SIGNATURE
Name (print): Damith Signature: [Signature] Date: 2026/08/20

Optera — Final Year Research Project APRIT Sri Lanka University of Staff Training (SLU) 2025

(a) UAT form - Damith

APRIT Sri Lanka University of Staff Training (SLU) - Final Year Research Project
Optera — User Acceptance Testing Form
Simulation-Based Procurement Optimization Python Library

A — PARTICIPANT DETAILS
Full Name: P.G. Gihan Prasanna Rajapaksha
Organisation: Bullapps Business Solutions
☐ Data Analytics ☒ Data Science ☐ Supply Chain ☐ Operations ☐ Software Engineering ☐ Other

B — LIBRARY USAGE
Which Optera modules did you use? (check all that apply)
☒ optera.ets() ☒ optera.analytics() ☒ optera.optimize() ☒ optera.simulation() ☒ optera.run()
Did you use a custom column mapping to load your own dataset?
☐ Yes — my own data ☒ Yes — sample/demo data ☐ No — used default schema

C — FUNCTIONAL ACCEPTANCE (1=poor, 5=excellent)
ETL pipeline — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5 Demand modelling — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
PSO optimization — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5 Monte Carlo simulation — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
Efficient frontier — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☒ 5 Simulation output — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☒ 5
Report generation — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☒ 5

Overall, did Optera produce useful procurement optimization insights?
☒ Yes, clearly useful ☐ Partially useful ☐ Needs improvement ☐ Not useful for my use case

D — USABILITY AS A PYTHON LIBRARY
☒ The pip install and import process was straightforward ☒ The column mapping configuration was clear and flexible
☒ The function parameters were well named and intuitive ☒ The optimization results (weights, frontier) were interpretable
☒ The simulation output was easy to read and act on ☒ The library ran without errors on my environment

E — SUGGESTIONS FOR IMPROVEMENT
Please describe any issues encountered, features you expected but did not find, or improvements you would suggest for the Optera library.
Clear documentation and examples would help new users understand what to write their own datasets and parameters.

F — OVERALL VERDICT
I confirm that I have evaluated the Optera Python library and that my responses above reflect my genuine professional assessment.
☐ I accept Optera as a valid and functional procurement optimization library
☒ I accept Optera with minor improvements recommended (see Section E)
☐ I do not accept — significant issues identified (see Section E)

G — SIGNATURE
Name (print): Gihan Signature: [Signature] Date: 2026/08/20

Optera — Final Year Research Project APRIT Sri Lanka University of Staff Training (SLU) 2025

(b) UAT form - Gihan

APRIT Sri Lanka University of Staff Training (SLU) - Final Year Research Project
Optera — User Acceptance Testing Form
Simulation-Based Procurement Optimization Python Library

A — PARTICIPANT DETAILS
Full Name: Nevinka Wickramasinghe
Organisation: Delaite
☒ Data Analytics ☐ Data Science ☐ Supply Chain ☐ Operations ☐ Software Engineering ☐ Other

B — LIBRARY USAGE
Which Optera modules did you use? (check all that apply)
☒ optera.ets() ☒ optera.analytics() ☒ optera.optimize() ☒ optera.simulation() ☐ optera.run()
Did you use a custom column mapping to load your own dataset?
☐ Yes — my own data ☒ Yes — sample/demo data ☐ No — used default schema

C — FUNCTIONAL ACCEPTANCE (1=poor, 5=excellent)
ETL pipeline — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5 Demand modelling — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
PSO optimization — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5 Monte Carlo simulation — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5
Efficient frontier — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☒ 5 Simulation output — ☐ 1 ☐ 2 ☐ 3 ☒ 4 ☒ 5
Report generation — ☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5

Overall, did Optera produce useful procurement optimization insights?
☒ Yes, clearly useful ☐ Partially useful ☐ Needs improvement ☐ Not useful for my use case

D — USABILITY AS A PYTHON LIBRARY
☒ The pip install and import process was straightforward ☒ The column mapping configuration was clear and flexible
☒ The function parameters were well named and intuitive ☒ The optimization results (weights, frontier) were interpretable
☒ The simulation output was easy to read and act on ☒ The library ran without errors on my environment

E — SUGGESTIONS FOR IMPROVEMENT
Please describe any issues encountered, features you expected but did not find, or improvements you would suggest for the Optera library.
The Optera library is useful as it is an ETL, demand modelling, optimisation, simulation and reporting tool. However, user experiences could be enhanced to use the Optera documentation and examples would help new users understand how to use the data, and datasets and set parameters. The optimisation results could be explained in a simpler, more business focused way. There are some suggestions to make the Optera library more user friendly and easy to use. Finally, a simple dashboard for easy procurement optimisation would make for a better user experience.

F — OVERALL VERDICT
I confirm that I have evaluated the Optera Python library and that my responses above reflect my genuine professional assessment.
☐ I accept Optera as a valid and functional procurement optimization library
☒ I accept Optera with minor improvements recommended (see Section E)
☐ I do not accept — significant issues identified (see Section E)

G — SIGNATURE
Name (print): Nevinka Wickramasinghe Signature: [Signature] Date: 2026/08/28

Optera — Final Year Research Project APRIT Sri Lanka University of Staff Training (SLU) 2025

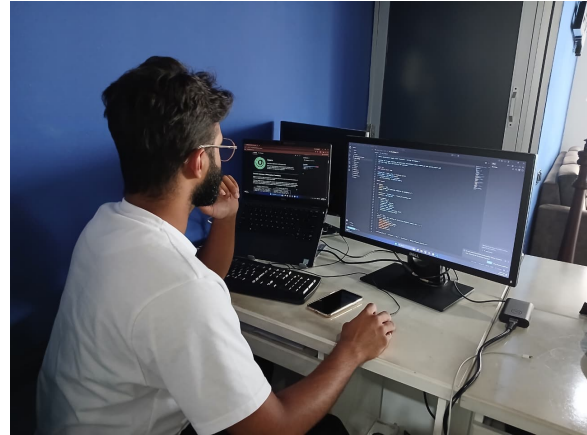
(c) UAT form - Nevinka

Figure 33: UAT Testing Forms

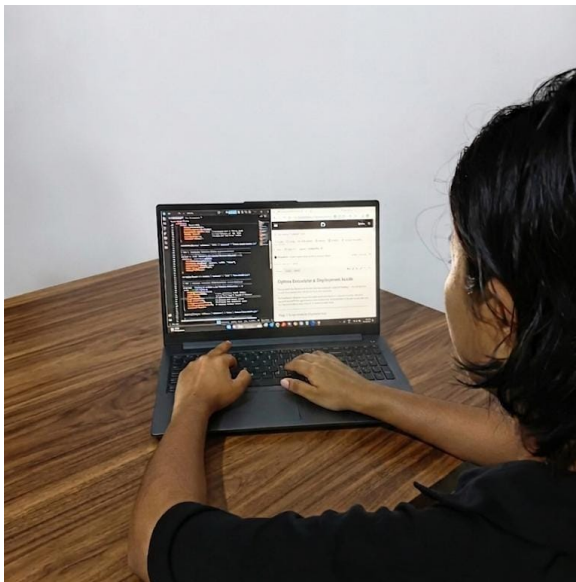
A.2 UAT testing Visit



(a) UAT test - Damith



(b) UAT test - Gihan



(c) UAT test - Nevinka



(d) Approval letter - Builtapps

Figure 34: UAT Testing Visits and Approvals

A.3 Interview Consent Forms

RESEARCH PROJECT CONSENT FORM

Title of Project: Quantitative Backtesting and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Researcher: M.G.L. Niroshan Kumararathna – CB013366

I have read and understood the information sheet provided for the above study.	Yes	☒	No	☐
I have been given the opportunity to ask questions, and I have had any questions answered satisfactorily.	Yes	☒	No	☐
I understand that my participation in this study is entirely voluntary and that I can withdraw at any time without having to give an explanation and without any penalty or loss of benefits to which I am otherwise entitled.	Yes	☒	No	☐
I understand that the evaluation session (including my interaction with the software dashboard) may be screen-recorded and/or audio-recorded for analysis purposes.	Yes	☒	No	☐
I consent that data collected (anonymized feedback and testing metrics) could be used for publication in scientific journals, presented in scientific forums (conferences, seminars), or used for teaching purposes, provided that my personal identity remains confidential.	Yes	☒	No	☐
I agree that data will only be used for this project (Simulation-Based Optimization of Inventory Budget Allocation under Demand Uncertainty), although the anonymized data may be audited for quality control purposes by the university.	Yes	☒	No	☐
All data will be stored safely on a password protected computer (electronic data), or locked away securely (hard copies of data) for 10 years before being destroyed.	Yes	☒	No	☐
I understand that all data will be stored safely on a password-protected computer and will be retained only until the degree has been awarded and the examination board has confirmed the results (approximately 6 months), after which it will be securely destroyed.	Yes	☒	No	☐
I understand that I can withdraw my data from the project up to the point of data anonymization and analysis (typically 2 weeks after the session) without having to give an explanation.	Yes	☒	No	☐
I hereby give consent to take part in this study.	Yes	☒	No	☐

Name Participant: Nevinka Wickramasinghe Job Title / Role: Intern - Control Assurance Organisation: Staffordshire University

Date: 04/06/2026 Signature: 

Name Researcher: M.G.L. Niroshan Date: 04/06/2026 Signature: 

(a) Consent Form – Nevinka Wickramasinghe

RESEARCH PROJECT CONSENT FORM

Title of Project: Quantitative Backtesting and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Researcher: M.G.L. Niroshan Kumararathna – CB013366

I have read and understood the information sheet provided for the above study.	Yes	☒	No	☐
I have been given the opportunity to ask questions, and I have had any questions answered satisfactorily.	Yes	☒	No	☐
I understand that my participation in this study is entirely voluntary and that I can withdraw at any time without having to give an explanation and without any penalty or loss of benefits to which I am otherwise entitled.	Yes	☒	No	☐
I understand that the evaluation session (including my interaction with the software dashboard) may be screen-recorded and/or audio-recorded for analysis purposes.	Yes	☒	No	☐
I consent that data collected (anonymized feedback and testing metrics) could be used for publication in scientific journals, presented in scientific forums (conferences, seminars), or used for teaching purposes, provided that my personal identity remains confidential.	Yes	☒	No	☐
I agree that data will only be used for this project (Simulation-Based Optimization of Inventory Budget Allocation under Demand Uncertainty), although the anonymized data may be audited for quality control purposes by the university.	Yes	☒	No	☐
All data will be stored safely on a password protected computer (electronic data), or locked away securely (hard copies of data) for 10 years before being destroyed.	Yes	☒	No	☐
I understand that all data will be stored safely on a password-protected computer and will be retained only until the degree has been awarded and the examination board has confirmed the results (approximately 6 months), after which it will be securely destroyed.	Yes	☒	No	☐
I understand that I can withdraw my data from the project up to the point of data anonymization and analysis (typically 2 weeks after the session) without having to give an explanation.	Yes	☒	No	☐
I hereby give consent to take part in this study.	Yes	☒	No	☐

Name Participant: A.V.A. Nuwan Pradeep Kumara Job Title / Role: Software Engineer Organisation: TEC Pvt Ltd

Date: 06/02/2026 Signature: 

Name Researcher: M.G.L. Niroshan Date: 06/02/2026 Signature: 

(b) Consent Form – Nuwan Kumara

RESEARCH PROJECT CONSENT FORM

Title of Project: Quantitative Backtesting and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Researcher: M.G.L. Niroshan Kumararathna – CB013366

I have read and understood the information sheet provided for the above study.	Yes	☒	No	☐
I have been given the opportunity to ask questions, and I have had any questions answered satisfactorily.	Yes	☒	No	☐
I understand that my participation in this study is entirely voluntary and that I can withdraw at any time without having to give an explanation and without any penalty or loss of benefits to which I am otherwise entitled.	Yes	☒	No	☐
I understand that the evaluation session (including my interaction with the software dashboard) may be screen-recorded and/or audio-recorded for analysis purposes.	Yes	☒	No	☐
I consent that data collected (anonymized feedback and testing metrics) could be used for publication in scientific journals, presented in scientific forums (conferences, seminars), or used for teaching purposes, provided that my personal identity remains confidential.	Yes	☒	No	☐
I agree that data will only be used for this project (Simulation-Based Optimization of Inventory Budget Allocation under Demand Uncertainty), although the anonymized data may be audited for quality control purposes by the university.	Yes	☒	No	☐
All data will be stored safely on a password protected computer (electronic data), or locked away securely (hard copies of data) for 10 years before being destroyed.	Yes	☒	No	☐
I understand that all data will be stored safely on a password-protected computer and will be retained only until the degree has been awarded and the examination board has confirmed the results (approximately 6 months), after which it will be securely destroyed.	Yes	☒	No	☐
I understand that I can withdraw my data from the project up to the point of data anonymization and analysis (typically 2 weeks after the session) without having to give an explanation.	Yes	☒	No	☐
I hereby give consent to take part in this study.	Yes	☒	No	☐

Name Participant: D. Shashini Harischandra Job Title / Role: Data Analyst Organisation: CBC Finance

Date: 2026/06/12 Signature: 

Name Researcher: M.G.L. Niroshan Date: 2026/06/12 Signature: 

(c) Consent Form – Shashini Harischandrae

RESEARCH PROJECT CONSENT FORM

Title of Project: Quantitative Backtesting and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Researcher: M.G.L. Niroshan Kumararathna – CB013366

I have read and understood the information sheet provided for the above study.	Yes	☒	No	☐
I have been given the opportunity to ask questions, and I have had any questions answered satisfactorily.	Yes	☒	No	☐
I understand that my participation in this study is entirely voluntary and that I can withdraw at any time without having to give an explanation and without any penalty or loss of benefits to which I am otherwise entitled.	Yes	☒	No	☐
I understand that the evaluation session (including my interaction with the software dashboard) may be screen-recorded and/or audio-recorded for analysis purposes.	Yes	☒	No	☐
I consent that data collected (anonymized feedback and testing metrics) could be used for publication in scientific journals, presented in scientific forums (conferences, seminars), or used for teaching purposes, provided that my personal identity remains confidential.	Yes	☒	No	☐
I agree that data will only be used for this project (Simulation-Based Optimization of Inventory Budget Allocation under Demand Uncertainty), although the anonymized data may be audited for quality control purposes by the university.	Yes	☒	No	☐
All data will be stored safely on a password protected computer (electronic data), or locked away securely (hard copies of data) for 10 years before being destroyed.	Yes	☒	No	☐
I understand that all data will be stored safely on a password-protected computer and will be retained only until the degree has been awarded and the examination board has confirmed the results (approximately 6 months), after which it will be securely destroyed.	Yes	☒	No	☐
I understand that I can withdraw my data from the project up to the point of data anonymization and analysis (typically 2 weeks after the session) without having to give an explanation.	Yes	☒	No	☐
I hereby give consent to take part in this study.	Yes	☒	No	☐

Name Participant: Sandeep Dissanayake Job Title / Role: Lead- DevOps Engineer Organisation: Infimum360

Date: 06/03/2026 Signature: 

Name Researcher: M.G.L. Niroshan Date: 06/03/2026 Signature: 

(d) Consent Form – Sandeep Dissanayake

Figure 35: Participant Consent Forms

[illegible]



Register for the course?

PLS V.1.0

Project Log Sheet Supervisory Section

Notes on use of the project log sheet:

- The log sheet is designed for meetings of no more than 15 minutes duration, of which there must be at minimum 10% (15 mins) during the course of the project. (OR mandatory supervisory sessions)
- The student should prepare for the supervisor sessions by deciding which questions to ask the supervisor and what the supervisor will be asked to do. The student should be able to fill in [a] and [b] since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the next session.
- A log sheet must be brought by the STUDENT to each supervisory session.
- At the start of the student's meeting with the supervisor, which should be carried out before the next session should be noted briefly in the relevant sections of the form.
- The student should be asked to copy the contents of the Project Log Sheet with the supervisor and by the administrator at the administrator's request. A copy is retained by the student to be filed in the project file.
- It is recommended that students bring along a copy of previous meeting documents with the project file during the supervisory sessions.
- It must be an important objective for the student to be an important record of a student's organisation and learning experience. The student should hand in the log sheet as an appendix of the final year dissertation, with sheets dated and numbered consecutively.

Student's Name	M.EL. Nivethan Kumaravelu	Date	1/04/2006	Meeting No.	05
Project Title	Research: Improving High Performance Broadband Transmission Through the DMES	Issue			
Supervisor's Name	Dr. Peter Dr. Rajitha Navathana	Supervisor's Signature			

Items for Discussion (noted by student during mandatory supervisory meeting):

- Review the Finalized Project Proposal for final feedback and official approval
- Confirm the exact timeline and specific feedback comments to be used (i.e., person)
- Walk through the complete end-to-end system workflow and business flowchart of the case study
- Present the Finalized high-level Hybrid Architecture diagram for technical validation.

Record of discussion (noted by student during mandatory supervisory meeting):

- Reviewed and discussed all the agenda items, resulting in the approval of the Finalized Proposal and system architecture.
- Confirmed the specific task requests for the development environment.
- Summarized the complete end-to-end system workflow, Review the complete end-to-end system workflow flowchart, Finalized all the business and data requirements.
- Reviewed and officially approved the proposed High-level Hybrid Architecture design.

Action list (to be attempted or completed by student by the next mandatory supervisory meeting):

- Initiate Phase 1, Set up the development environment and create the initial Prototype.
- Conduct Language Integration, Develop the "Zero-Code" Communication layer to provide message sharing between the client and server.
- Initiate Phase 2 and implement the client-server communication layer focusing on high-performance and latency.
- Develop Server-side back-end processing logic to feed the execution layer of the client/server for the next release.

Note: A student should make an appointment to meet with his/her supervisor (in the consultation session) at least ONE (1) week prior to a mandatory supervisory session. Please be punctual or arrived 10 minutes prior. In the event a supervisor cannot be booked for consultations, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

A P P I T
 (open house for everyone)

PSV 1.0

Project Log Sheet Supervisory Session

Notes on use of the project log sheet:

- This log sheet is designed for meeting for more than 1 hour in duration, of which there must be at minimum 15 minutes during the course of the project. (30 minutes mandatory sessioning)
- The student should prepare for the meeting by deciding which questions he/she needs to ask the supervisor and what progress has been made (if any) since the last session, and recording these in the relevant sections of the form, effectively forming an agenda for the session.
- A log sheet is to be brought to the TUTOR for each supervisory session.
- The student by the student (and present to the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
- The student should have a clear agenda for the session, if the student is to discuss with the supervisor and to the administrator at the academic office.
- A copy of the agenda is to be taken to the TUTOR to be filed in the project file.
- It is recommended that students bring along their project log sheet to discuss progress together with the project log during the supervisory session.
- It is recommended that the project log sheet be used as a record of a student's preparation and learning experience.
- The log sheet should be given to the supervisor as an appendix of the final year dissertation, with sheets dated and numbered consecutively.

Student's Name	M.A.I. Kirehman, Nourahmadi	Date	9/31/2006	Meeting No.	03
Project Title	Model Estimating the Role of Informative Content in Social and Economic for SME	Initials			
Supervisor's Name	Dr. Reza Jahanbakhsh	Supervisor's Signature			

Items for Discussion (noted by student) (add mandatory supervisory meeting):

- Final Development Requirements, Class 1, Chapter and Module Data, Configuration for hybrid control development.
- Finalize the data and hardware configuration, completed the data module, the cross-language module.
- Adjusted the directly accessing Python database without data duplication.
- Added and reconfigured some function according over shared database for mission data base.

Record of discussion (noted by student) (add mandatory supervisory meeting):

- Presented the final 4th chapter, successfully finishing a year's work according to the Python.
- Adjusted the data of class 1, module and cross-language module, on the optimal standard for simulation.
- Adjusted according to changing the primary resource and designing effective subroutines.
- Discussed functional enhancing strategies to accomplish research on the industry Python analysis and SFC.

Action List (to be implemented or completed by the student) (add mandatory supervisory meeting):

- Finalize and distribute the design from logging Data Analysis and Software Engineers.
- Document the project on GitHub by adding the development methodology, PYTHON file.
- Draft and format the final technical report following the basic final year technical report.
- Continue development on the C++ core, focusing on the data, final log and module development.

Note: A supervisor should make an appointment to meet with his or her supervisor (use the consultation system) at least one hour prior to a mandatory supervisory session. Please see documents on project time. In the event a supervisor should not be available for supervisory session, the project session should be informed (ONE) week prior to the session for a meeting can be subsequently arranged.



APL
Project Lead for success!

PLS V.1.0

Project Log Sheet Supervisory Session

Notes on use of the project log sheet:

- This log sheet is designed for meetings of over 15 minutes duration, for which there must be at minimum five (5) attendees during the course of the project (20 mandatory supervisory sessions).
- The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what has been learnt from (any) meeting with the supervisor, and noting these in the relevant columns of the form, effectively forming an agenda for the session.
- A log sheet is not brought to the student to each supervisory session.
- The agenda by the student (and, perhaps the supervisor) should be carried before the next session should be noted briefly in the relevant section of the form.
- The student should forward a copy after the project log sheet with the supervisor and to the administrator at the academic centre. A copy is retained by the student to be filed in the project log.
- It is recommended that students bring along lists of previous meetings together with the project file during the supervisory sessions.
- The log sheet is an important document for the project and an important record of the student's organization and learning experience. The student should handle the log sheet in accordance with the project time documentation, with sheets dated and numbered consecutively.

Student's Name:	H. G. Nathan Jhangarath	Date:	8 / 10 / 2016	Meeting No:	01
Project Title:	Design of a new HMI for the Performance Evaluation System for the Technical Education Centre	Initials:			
Supervisor's Name:	Dr. Rajitha Mananayaka	Supervisor's Signature:			

Items for Discussion (noted by student before mandatory supervisory meeting):

- Review the Engaged Single Team for Data Analysts and Software Engineers to get distribution approval.
- Review the structured global variables and the newly added Proprietary Resource File.
- Discuss the completed formal report defining the full Technical Architecture.
- Demonstrate the initial development of the C++ Event Log and State Machine (C++).

Record of discussion (noted during mandatory supervisory meeting):

- Rajitha approved the single team for immediate industry distribution.
- Approved that the framework to be tested and benchmarked against a similar existing tool to quantitatively validate Performance gain.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

- Present benchmarking results.
- Review the industry validation responses to the.

Note: A student should submit an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to the mandatory supervisory session. Please do not overbook an project time slot. In the event a supervisor should not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

Figure 36: Log Sheets (1–4)

A.P.O.I.T
Project Log Sheet Supervisory Session

PLS V 1.0

Notes on use of the project log sheet:

- This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum FIVE (5) during the course of the project (24 mandatory supervisory sessions).
- The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
- A log sheet is to be brought by the STUDENT to each supervisory session.
- The action by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
- The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
- It is recommended that students bring along log sheets of previous meetings together with the project file during the supervisory session.
- The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student should hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name:	M. S. Nishan Kumararathna	Date:	15/10/2024	Meeting No:	05
Project Title:	Revenue Based Procurement Optimization Under Demand Uncertainty	Intake:			
Supervisor's Name:	Dr. Rajitha Narayana	Supervisor's Signature:			

Items for Discussion (noted by student before mandatory supervisory meeting):

- Review initial Google form responses from data analysts regarding the manual analysis workflow.
- Review initial Google form responses from Software Engineers regarding the hybrid architecture.
- Discuss the decreasing data generation speed of the cyber/hybrid engine over time.
- Discuss the performance comparison results between the new C++ engine directly against Ruckh.

Record of discussion (noted by student during mandatory supervisory meeting):

- Conducted a live demonstration of the April benchmark against build-time, accessibility.
- Speculating an approach for execution speed-up.
-
-

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

- Develop integrating Particle swarm optimization into framework to calculate optimal budget allocation weights across the categories (Qualitative analysis of supply chain).
- Prepare and refine a new, more academically rigorous file for the overall benchmark project.

Note: A student should make an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to a mandatory supervisory session. Please see document on project time lines. In the event a supervisor could not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

(a) Log Sheet 5

A.P.O.I.T
Project Log Sheet Supervisory Session

PLS V 1.0

Notes on use of the project log sheet:

- This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum FIVE (5) during the course of the project (24 mandatory supervisory sessions).
- The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
- A log sheet is to be brought by the STUDENT to each supervisory session.
- The action by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
- The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
- It is recommended that students bring along log sheets of previous meetings together with the project file during the supervisory session.
- The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student should hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name:	M. S. Nishan Kumararathna	Date:	24/10/2024	Meeting No:	06
Project Title:	Revenue Based Procurement Optimization Under Demand Uncertainty	Intake:			
Supervisor's Name:	Dr. Rajitha Narayana	Supervisor's Signature:			

Items for Discussion (noted by student before mandatory supervisory meeting):

- Review lead-time feasibility and inventory allocation results.
- Discuss integration of the PSO output with the simulation layer.
- Review the inventory replenishment policy.
- Discuss Monte Carlo Simulation design.

Record of discussion (noted by student during mandatory supervisory meeting):

- PSO allocation weights were linked to initial procurement inventory.
- A base-stock (SS) inventory policy was selected for simulation.
- Demand uncertainty was identified as the main stochastic input.
- Monte Carlo Simulation was selected to evaluate the robustness of optimized allocation.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

- Implement the PSO Monte Carlo simulation engine.
- Integrate PSO allocation with inventory simulation.
- Implement stochastic demand generation and lead-time handling.
- Calculate profit, service level and backlog metrics.

Note: A student should make an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to a mandatory supervisory session. Please see document on project time lines. In the event a supervisor could not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

(b) Log Sheet 8

A.P.O.I.T
Project Log Sheet Supervisory Session

PLS V 1.0

Notes on use of the project log sheet:

- This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum FIVE (5) during the course of the project (24 mandatory supervisory sessions).
- The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
- A log sheet is to be brought by the STUDENT to each supervisory session.
- The action by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
- The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
- It is recommended that students bring along log sheets of previous meetings together with the project file during the supervisory session.
- The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student should hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name:	M. S. Nishan Kumararathna	Date:	26/10/2024	Meeting No:	09
Project Title:	Simulation based Procurement Optimization Under Demand Uncertainty	Intake:			
Supervisor's Name:	Dr. Rajitha Narayana	Supervisor's Signature:			

Items for Discussion (noted by student before mandatory supervisory meeting):

- Present Monte Carlo Simulation Results.
- Discuss evaluation metrics for procurement strategies.
- Review simulation stability and risk measurements.
- Discuss final framework testing and reporting.

Record of discussion (noted by student during mandatory supervisory meeting):

- Multiple stochastic simulation trials were used to evaluate the optimized strategy.
- Profit distribution, var, CVAR, service level and fill rate were selected as evaluation metrics.
- Simulation results were used to assess the robustness of PSO allocation under demand uncertainty.
- Final integration testing of the five-layer framework was discussed.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

- Complete integration testing of all framework layers.
- Compare PSO against baseline allocation strategies.
- Generate final evaluation plots and reports.
- Prepare the framework for final demonstration.

Note: A student should make an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to a mandatory supervisory session. Please see document on project time lines. In the event a supervisor could not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

(c) Log Sheet 9

A.P.O.I.T
Project Log Sheet Supervisory Session

PLS V 1.0

Notes on use of the project log sheet:

- This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum FIVE (5) during the course of the project (24 mandatory supervisory sessions).
- The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
- A log sheet is to be brought by the STUDENT to each supervisory session.
- The action by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
- The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
- It is recommended that students bring along log sheets of previous meetings together with the project file during the supervisory session.
- The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student should hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name:	M. S. Nishan Kumararathna	Date:	06/11/2024	Meeting No:	10
Project Title:	Simulation based Procurement Optimization Under Demand Uncertainty	Intake:			
Supervisor's Name:	Dr. Rajitha Narayana	Supervisor's Signature:			

Items for Discussion (noted by student before mandatory supervisory meeting):

- Demonstrate the completed procurement optimization framework.
- Review final experimental results and comparisons.
- Discuss packaging the framework as a Python library.
- Review final thesis and viva preparation.

Record of discussion (noted by student during mandatory supervisory meeting):

- Complete work was demonstrated and overall workflow was reviewed with the supervisor.
- The final thesis structure, completed chapters and final viva/evaluation were discussed.
-
-

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

- Finalize any remaining exercises and finalizing the thesis based on supervisor feedback.
-
-
-

Note: A student should make an appointment to meet his or her supervisor (via the consultation system) at least ONE (1) week prior to a mandatory supervisory session. Please see document on project time lines. In the event a supervisor could not be booked for consultation, the project manager should be informed ONE (1) week prior to the session so that a meeting can be subsequently arranged.

(d) Log Sheet 10

Figure 37: Log Sheets (5–10)

A.5 Interview Questions

APIT Sri Lanka | University of Staffordshire (UK)
Final Year Research Project — BSc (Hons) Computer Science



SEMI-STRUCTURED INTERVIEW

Quantitative Backtesting and Optimization Framework for Procurement Strategy Evaluation under Demand Uncertainty

Participant Details

Full Name:

Job Title / Role:

Organisation:

Date:

Instructions

This interview is part of an academic research study investigating how organisations currently manage procurement budget allocation under demand uncertainty. Please respond based on your professional experience. The interview will take approximately 15 to 20 minutes.

Researcher: M.G.L. Nishan Kumararathna | Supervisor: Dr. Rajitha Navarathna

Q1. Please briefly describe your current role and the types of decisions you are typically involved in related to procurement, inventory, stock or supply chain planning.
To understand the participant's background and decision-making context.

Q2. When your team decides how much of the procurement budget to allocate across different product categories, what does that process currently look like? Is it data-driven, experience-based, or a combination of both?
Validates whether formal or informal allocation methods are used.

Q3. Before implementing a new allocation or procurement strategy, does your team have a way to test or evaluate how it would have performed using past data? If yes, what tools or methods do you use?
Validates the workflow gap—whether backtesting or simulation is part of the current process.

Q4. How does your team account for unpredictable changes in demand, such as seasonal spikes, sudden drops, or unexpected market shifts, when planning procurement budgets?
Validates whether probabilistic or deterministic methods are used for demand uncertainty.

Q5. If a tool/algorithms could automatically identify the optimal budget allocation across your product categories to maximize profitability while minimizing financial risk, how valuable would that be to your team?
Validates the value of PSO-based optimisation as a capability.

Q6. From a data privacy and compliance perspective, would it matter to your team whether an analytics tool processes your financial data entirely on your own machine rather than uploading it to an external cloud platform?

Q7. Is there anything about your current approach to procurement planning or budget allocation that you feel is genuinely inadequate or that you would like to see improved?

(a) Interview Question - Data Analyst

APIT Sri Lanka | University of Staffordshire (UK)
Final Year Research Project — BSc (Hons) Computer Science



SEMI-STRUCTURED INTERVIEW

Architecture and Performance Validation
Software Engineers, Backend Developers and DevOps Professionals

Participant Details

Full Name:

Job Title / Role:

Organisation:

Date:

Instructions

This interview is part of an academic research project developing a simulation-based backtesting and optimization framework for supply chain strategy evaluation. The system uses a hybrid C++ and Python architecture. Your responses will help validate the technical and architectural decisions from a software engineering perspective. There are no right or wrong answers. The interview will take approximately 15 to 20 minutes.

Researcher: M.G.L. Nishan Kumararathna | Supervisor: Dr. Rajitha Navarathna

Q1. Please briefly describe your current role and the types of systems or applications you typically design or develop.

Q2. In your professional experience, have you encountered situations where Python was too slow for a large-scale computational or simulation task? If yes, how did you approach solving that performance problem?

Q3. What is your professional opinion on hybrid architectures that use Python for user-facing orchestration and a compiled language such as C++ for the performance-critical execution core? Is this a sound and practical engineering approach?

Q4. If you needed to transfer large numerical arrays between Python and C++ with the lowest possible overhead, what approach would you take? Are you familiar with any tools or techniques that address this problem?

Q5. In your professional opinion, is it credible for a simulation engine to achieve 50 to 100 times faster execution than a pure Python equivalent on the same hardware through compiled language integration? What factors would influence that result?

Q6. When building an analytics or simulation system that must handle sensitive financial data, what is your preferred deployment model from a security and data sovereignty perspective? Would on-premise local execution matter to your organisation?

Q7. From a software engineering perspective, what do you consider the most important factors when evaluating or adopting an open-source simulation framework for enterprise use?

Thank you sincerely for your time and contribution to this research.

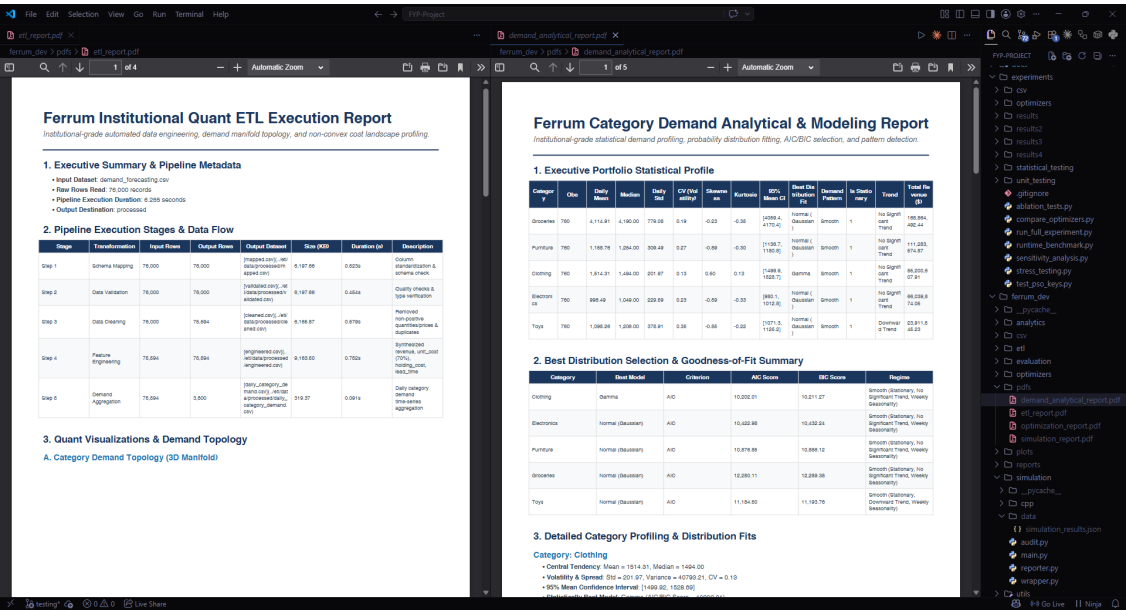
(b) Interview Question - Software Engineer

CB013366

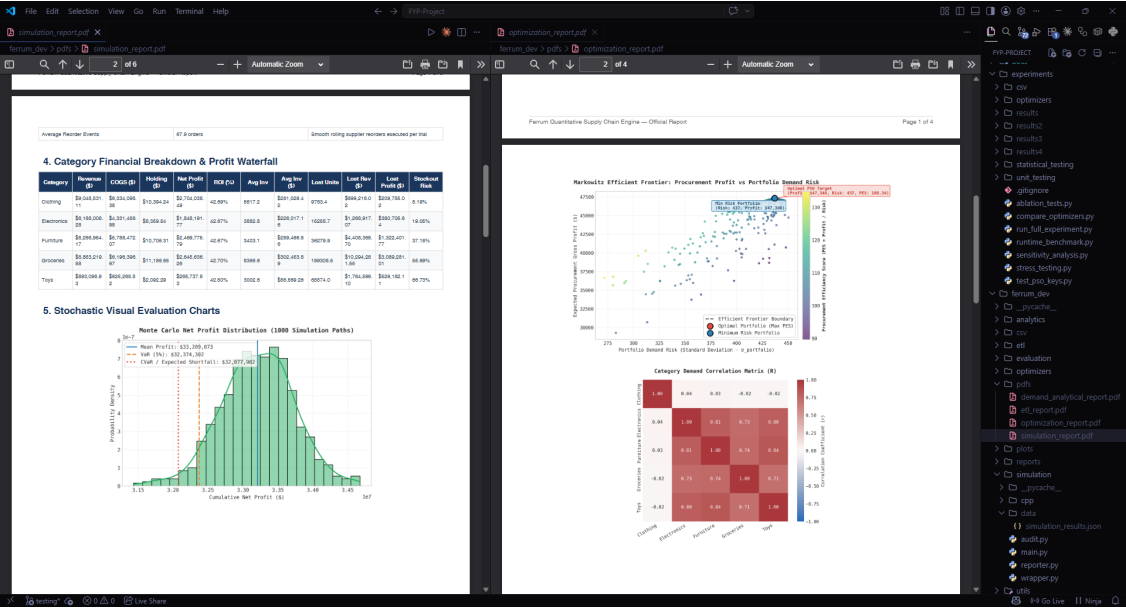
xxiv

Lakshan N. Kumararathna

A.6 Report Outputs



(a) Report output 1



(b) Report output 2